UDIT-PORTFOLIO · PERFORMANCE REVAMP

The Frontend Bible

An end-to-end record of the pixel-identical performance makeover of **gitudit.github.io/udit-portfolio**: audit, every change and why, how parity was proven, the numbers, and a complete interview preparation guide.

PROJECT	Personal portfolio (React single page) · GitHub Pages
WORK COMPLETED	17 October 2026
SCOPE	Toolchain, rendering strategy, bundles, fonts, icons, images, animations, third parties, hygiene, verification
HARD CONSTRAINT	Rendered pixels identical before and after. Nothing that worked may break.
HEADLINE RESULT	Lighthouse mobile 53 → 98–99 (lab); JS on the critical path 258 KB → 69 KB gzipped; 7 origins → 1 (+ analytics after load); 0 differing pixels across 8 viewport/scheme captures

Contents

1 Executive summary

2 Starting point: the audit

3 Principles and constraints

4 Architecture: before and after

5 The changes, area by area

- 5.1 Build toolchain and dependencies
- 5.2 Rendering strategy: build-time pre-render + hydration
- 5.3 Scroll-reveal animations without react-reveal
- 5.4 Lottie: lazy, paused off-screen, layout-stable
- 5.5 Fonts: self-hosted, subset, preloaded
- 5.6 Icons: Font Awesome and font-mfizz without CDNs
- 5.7 Images
- 5.8 Third-party scripts and stylesheets
- 5.9 Dead code and dead dependencies
- 5.10 Runtime behaviour fixes
- 5.11 SEO, PWA manifest and document head
- 5.12 Security hygiene
- 5.13 Quirks deliberately preserved
- 5.14 Repository hygiene and developer experience

6 Verification: how parity and correctness were proven

7 Results

8 Operations: build, deploy, maintain

9 What is next

10 Interview preparation

- 10.1 Telling the story and the numbers to memorise
- 10.2 Foundations: how browsers load and render pages
- 10.3 Foundations: React and the build toolchain
- 10.4 Intermediate: rendering strategy, animation, fonts, images, CSS
- 10.5 Advanced: the mechanics behind the decisions
- 10.6 Expert: debugging stories, measurement and edge cases
- 10.7 System design
- 10.8 Behavioural questions and cross-examination
- 10.9 Rapid-fire revision glossary

11 Appendices: file map, dependency table, glossary, commands

1. Executive summary

The portfolio is a single-page React site generated from the DeveloperFolio template in 2020 and deployed to GitHub Pages. It looked fine but loaded badly: on a simulated mobile connection the page stayed blank for over five seconds because nothing could paint until roughly a quarter megabyte of JavaScript and render-blocking CSS from three third-party origins had arrived. The task was to fix that completely, modernise the stack so a later UI/UX redesign has a clean base, and change *nothing* a visitor can see.

The work was done in three phases. First an audit measured the live site with Lighthouse, attributed every byte of the bundles to a package with source maps, and inspected every file in the repository. Second, the site was rebuilt on Vite 6 and React 18 with build-time pre-rendering, self-hosted subset fonts, inline-free icon fonts, lazy Lottie animations, a from-scratch replacement for the abandoned react-reveal library, and the removal of every dead dependency. Third, the result was proven pixel-identical to a preserved copy of the old build with scripted headless-Chrome captures at six viewports and two colour schemes, functionally exercised end to end, and re-measured with Lighthouse on both builds under identical local conditions.

Headline numbers

Measure	Before	After	Notes
Lighthouse performance, mobile (local, same server)	63	98–99	Live-site baseline was 53; GitHub Pages TTFB adds ~0.8 s on mobile
Lighthouse performance, desktop (local)	97	100	
First Contentful Paint, mobile	3.8 s	1.4 s	Text paints from pre-rendered HTML
Largest Contentful Paint, mobile	5.5 s	2.1 s	Live site measured 8.0 s
Total Blocking Time, mobile	330 ms	60–90 ms	No analytics or Apollo on the critical path
Cumulative Layout Shift	0.001	0	Lottie boxes reserve their exact footprint
Total transfer, first visit	1,136 KB · 33 requests · 7 origins	413 KB (mobile) · 20 requests · 1 origin + GA after load	
JavaScript before first paint	258 KB gzipped	0 KB (HTML paints first); 69 KB to hydrate	Lottie player (54 KB) loads lazily
Fonts	343 KB (TTF + WOFF) + 213 KB icon fonts/CSS from two CDNs	53 KB total, five WOFF2 files, self-hosted	Glyph outlines unchanged
Runtime dependencies	17 packages, 1,648 installed	7 packages, 101 installed	
Pixel difference vs old build	—	0 px outside animations, identical heights	8 viewport/scheme captures

What did not change

Every stylesheet rule that affects layout, every colour, font, size, spacing, animation duration and distance, every piece of content, the dark-mode behaviour, the carousels, tooltips, back-to-top button, contact form and analytics property. Two visual quirks of the original that are clearly bugs were kept on purpose because the brief forbade pixel changes; they are documented in section 5.13 with the one-line deletes for the redesign.

2. Starting point: the audit

2.1 Lab measurements of the live site

Metric (Lighthouse 12.8.2)	Mobile	Desktop
Performance score	53	96
First Contentful Paint	5.5 s	0.9 s
Largest Contentful Paint	8.0 s	1.2 s
LCP breakdown	TTFB 0.8 s, render delay 7.1 s	TTFB 0.16 s, render delay 1.06 s
Total Blocking Time	380 ms	30 ms
Cumulative Layout Shift	0	0.001
Speed Index	5.5 s	0.9 s
Requests / transfer / origins	33 / 1,157 KB / 7	
DOM size	871 elements	

The LCP element was plain text in both cases (the header logo on mobile, the hero subtitle on desktop). Its render delay was entirely time spent waiting for JavaScript to download, parse and execute, plus render-blocking stylesheets from three third-party origins. The page was a pure client-rendered React app: the HTML shipped an empty `<div id="root">`.

2.2 Where the bytes went

Source maps were used to attribute every minified byte of the two main chunks to its originating package.

Resource	Transfer	Composition / observation
Vendor chunk	170 KB gz (642 KB raw)	41% lottie-web (full build with canvas and HTML renderers), 18% react-dom, 14% Apollo client, 6% graphql, 5% react-reveal, 3% babel-runtime, plus react-headroom, react-easy-emoji, colorthief, dotenv, path-browserify, react-twitter-embed
Main chunk	82 KB gz (405 KB raw)	305 KB raw of Lottie animation JSON bundled synchronously; 54 KB of base64-inlined JPEG logos (base64 inflates by a third and compresses poorly); ~45 KB of application code
Google Tag Manager	293 KB, 2 scripts	A Universal Analytics tag (dead since July 2023) that then chained a second GA4 script via a connected-site tag
Font Awesome 5.13 CSS + two fonts	169 KB, 3 requests, render-blocking CSS	14 icons used
font-mfizz CSS + WOFF	44 KB, 2 requests, render-blocking CSS	4 icons used

Resource	Transfer	Composition / observation
Montserrat Regular TTF	113 KB gz (246 KB raw)	One weight shipped; bold is synthesised by the browser
Agustina WOFF	97 KB	Used only for the ten glyphs of the logo name
Google Fonts stylesheet	0.3 KB, render-blocking via @import	Requested Open Sans and Saira for a commented-out “Star Wars crawl”; nothing on the page used them
ambulance.png	139 KB	512 px PNG shown at 250 CSS px
emailjs CDN script	9 KB, render-blocking	Duplicate of the emailjs-com npm copy already bundled
Font preloads	2 × 404	Hard-coded `/static/media/…hash…` paths ignored the repository sub-path and would break on every rebuild
GitHub GraphQL	2 failed requests + preflights	The GitHub section was hidden (`display: false`), yet both components still queried the API on mount with `Bearer undefined`

2.3 Toolchain state

- react-scripts 3.4.3 (webpack 4, Babel 7 with a 2020 caniuse database). Browserslist resolved to IE 11, Chrome 49 and Safari 13, so every class was down-compiled and 17 KB of babel-runtime helpers shipped.
- React 16.13; `ReactDOM.render` ; a class component at the root.
- Build needed `--openssl-legacy-provider` to run on Node 18. Source maps (2.5 MB) were deployed with the site.
- react-reveal 1.2.2 (last published 2018) pinned the project to React 16, injected a global stylesheet at runtime and hid all unrevealed content on the first scroll with a global rule.
- react-lottie 1.2.3 wrapped the full lottie-web build; four animations autoplayed and looped forever, including two that were `display: none` on most screens.
- ColorThief ran on every work-experience logo at runtime to pick the banner colour (canvas read-back per image).

2.4 Bugs found along the way

- **Leaked credential.** The first line of `Header.js` carried a commented-out GitHub personal access token, public in the repository since September 2021. Removed from HEAD; rotation recommended (it remains in history).
- **Stray text node.** `//<Twitter />` inside JSX is not a comment; React rendered a literal “//” above the Contact section. Visible on the live site.
- **Half-size form inputs.** ToggleSwitch.css contained a global `input { transform: scale(0.5) }` . The toggle itself was commented out, but its stylesheet still loaded and shrank the contact form's Name and Email inputs.
- **Header spacer race.** react-headroom measures the header once on mount. The logo font only starts downloading when the logo text first renders, so the measurement always happened with the fallback

font: the spacer was 12 px too short on desktop and the header overlapped the hero until a window resize re-measured it.

- **Contact form reload.** `preventDefault()` was only called when the email validated; an invalid address submitted the form natively and reloaded the page with the field values in the URL.
- **Dead analytics.** The UA property stopped processing hits in 2023 but its script still loaded and then loaded a second GA4 script.
- **Manifest and tile icons** referenced files that did not exist (404s) and still carried the template author's name.
- Sixteen unused images (about 800 KB) and a duplicate AchivementCard component sat in the source tree; `dotenv` was required in browser code; `gsap` was imported and never used.

3. Principles and constraints

1. **Pixel parity is the acceptance test.** Not “looks the same” but “a pixel-diff of full-page screenshots at six widths and both colour schemes shows no difference outside animated regions, and page heights match to the pixel”.
2. **Nothing that worked may break.** Every interactive feature was scripted and exercised on the new build: carousels, tooltips, back-to-top, sticky header pin/unpin, dark mode from storage, form validation, anchor navigation, analytics firing.
3. **Measure, do not guess.** Bundle composition came from source maps, not package sizes. Layout geometry came from the DOM, not from reading CSS. Lighthouse was run on both builds on the same machine behind the same server so the comparison is fair.
4. **Determinism over luck.** Where the old site relied on a race (header spacer, hero animation start), the new one produces the same final layout on every load.
5. **Pay for bytes only when they are seen.** The Lottie player and each animation JSON load only when their box approaches the viewport; hidden boxes never load at all.
6. **Keep the redesign cheap.** Modern toolchain, function components, no abandoned libraries, documented quirks with one-line removals.

4. Architecture: before and after

Concern	Before	After
Build tool	create-react-app 3 / webpack 4 / Babel targeting IE 11	Vite 6 / Rollup / esbuild, ES2020 output (`build.target: 'modules'`)
Framework	React 16.13, `ReactDOM.render`	React 18.3, `hydrateRoot` in production, `createRoot` in dev
Rendering	Client-side only; empty root until JS runs	Static HTML pre-rendered at build time with `renderToString` , then hydrated
Module format	Classic scripts, webpack runtime	Native ES modules, `modulepreload` for the React chunk
Code splitting	One vendor chunk + main; two tiny lazy GitHub cards	App chunk (23 KB gz), React chunk (46 KB gz), lazy Lottie player (54 KB gz), lazy ColorThief (4 KB), lazy GitHub cards; animation JSON as hashed static assets fetched on demand
Scroll animations	react-reveal (runtime-injected keyframes, scroll listeners / IO)	In-repo `Reveal` component: static keyframes, IntersectionObserver, identical timing; hero animates from first paint
Lottie	react-lottie + full lottie-web, JSON inlined in main bundle, always animating	lottie-web 5.7.3 `lottie_svg` build imported on demand, JSON fetched on demand, paused off-screen, inline placeholder preserves geometry
Body/logo fonts	Montserrat TTF (246 KB), Agustina WOFF (97 KB), broken preloads	Subset WOFF2 (38 KB + 9 KB), correct hashed preloads, same `local()` fallbacks and `font-display`
Icon fonts	Font Awesome 5.13 + font-mfizz from two CDNs, render-blocking CSS, full fonts	Same fonts subset to the 18 glyphs used (4.8 KB total), rules copied verbatim, self-hosted, preloaded
Images	PNG project images (142/135/94 KB), JPEG logos base64-inlined into JS	Lossless WebP (77/62/43 KB), JPEG logos as hashed files, lazy loading below the fold
Analytics	UA tag chaining a GA4 tag, 293 KB, async in head	Single GA4 tag injected after `window.load`
GitHub data	Apollo Boost + graphql (~30 KB gz) querying on every load	18-line `fetch` wrapper, only runs when the section is enabled
Banner colours	ColorThief per logo at runtime	Pre-computed values in `portfolio.js` ; ColorThief loaded lazily only if a value is missing
Environment	`REACT_APP_*` + `require('dotenv')` in browser code	`VITE_*` via `import.meta.env`
Deployment	`gh-pages -d build`	Same, plus `--dotfiles` for `.nojekyll` ; no source maps shipped

5. The changes, area by area

5.1 Build toolchain and dependencies

What changed. create-react-app was replaced by Vite 6 with `@vitejs/plugin-react`. The project became an ES module package (`"type": "module"`) with a `vite.config.mjs`. Output still goes to `build/` so the deploy script is unchanged. The `homepage` sub-path is expressed as `base: '/udit-portfolio/'`.

Why Vite. webpack 4 could not be upgraded in place without ejecting; react-scripts 3 is unmaintained and needed the OpenSSL legacy flag on Node 18. Vite gives modern output without the IE 11 transforms (the old browserslist resolved to IE 11 because its caniuse database was from 2020), proper tree-shaking, CSS code-splitting for lazy routes, asset hashing with a configurable inline limit, and a first-class SSR build used for pre-rendering.

Configuration decisions.

- `assetsInlineLimit` is a function: images under 4 KB become data URIs, fonts never do. If a 760-byte icon font were inlined into CSS, the `<link rel="preload">` in the HTML would point at a file nobody uses.
- `manualChunks` isolates react, react-dom and scheduler into a `react` chunk. The app code changes with every content edit; the React runtime does not, so it caches independently.
- `ssr.noExternal` bundles react-headroom, emailjs-com and email-validator into the pre-render build. They are CommonJS with Babel's `exports.default`; imported as externals under Node's ESM loader their default import resolves to the namespace object and React throws “Element type is invalid”.
- Source maps are off for production. The old deploy shipped 2.5 MB of maps to GitHub Pages.

Dependencies.

Removed	Why
react-scripts, babel-runtime transforms	Replaced by Vite
apollo-boost, @apollo/react-hooks, graphql	~30 KB gzipped for two queries against a section that is disabled; replaced by a `fetch` wrapper
react-reveal	Unmaintained, React 16 only, runtime stylesheet injection; replaced by a 150-line component with identical behaviour
react-lottie	Thin class wrapper with legacy lifecycles; replaced by a hook-based component that lazy-loads the player
react-easy-emoji	Converted emoji to Twemoji images from a third-party CDN; no rendered text contained an emoji, so it changed nothing visible
react-twitter-embed	Section disabled and the component was never mounted (the stray `//`)
dotenv	Node-only; in the browser it pulled in path and process polyfills and silently failed
gsap	Imported, never used; Rollup already tree-shook it but it remained an install
gh-pages (moved to devDependencies)	Build tooling, not runtime

Kept / added	Version	Role
react, react-dom	18.3.1	Runtime; hydration API
react-headroom	3.2.1 (from 3.0.0)	Sticky header; React 18 peer support; same markup and class names
lottie-web	5.7.3, pinned	Same renderer version as before so every frame matches; `lottie_svg` build
emailjs-com, email-validator	3.2.0, 2.0.4	Contact form, unchanged API
colorthief	2.7.0	Lazy fallback when a logo has no pre-computed colour
vite, @vitejs/plugin-react, subset-font, prettier	dev	Build, font regeneration, formatting

5.2 Rendering strategy: build-time pre-render + hydration

Problem. A client-rendered page cannot paint text until the framework and the application have executed. On mobile that was 5.5 s to first paint.

Solution. The build runs three steps: `vite build` for the client, `vite build --ssr src/entry-server.jsx` for a Node bundle that exports `render()` (which calls `renderToString(<App/>)`), then `scripts/prerender.mjs` imports that bundle, renders the app once, and injects the 20 KB of markup into `build/index.html` in place of the empty root. The client entry checks whether the root already has children: if so it hydrates, otherwise (dev server) it does a normal render.

Why not a framework (Next, Astro, Gatsby). The site is one page with no routing and no data at build time. A framework would have meant rewriting every component's file conventions, adding a runtime, and risking subtle markup differences; the pixel-parity constraint made a minimal, inspectable pre-render the safer choice. The redesign can still move to a framework later; the components are now ordinary function components.

Hydration correctness. Server and client must produce identical markup on the first render, otherwise React 18 logs a recoverable error and re-renders the mismatched subtree on the client (costing time and possibly flashing). Each potential source of drift was handled:

- *Theme.* The stored preference lives in `localStorage`, which the server cannot read. The component therefore renders light on the server and on the first client render, then applies the stored value in an effect, exactly as the old class component did in `componentDidMount`. (A toggle does not exist on the page, so only system-dark visitors are affected, and the behaviour is unchanged.)
- *Lottie.* Both server and client render an empty box with an inline placeholder SVG; the player attaches in an effect.
- *Reveal state.* Every revealable element renders with `opacity: 0` on both sides, except the hero which uses `appear` and renders in the already-animating state on both sides.
- *Banner colours* come from static data rather than an image-load callback, so they are in the HTML.
- *Browser-only code* (`window.screen` at module scope in the Twitter container, `window.onscroll` in render, `require('dotenv')`) was removed or moved into effects.
- `class=` attributes in JSX were converted to `className`; `fill-rule` to `fillRule`; an invalid string `onClick="null"` was removed. Same DOM, no warnings.

- `onRecoverableError` logs to the console in every environment so drift is visible, not silent. The functional test asserts the console is empty.

Effect on metrics. The hero text and the whole page skeleton paint as soon as HTML and CSS arrive. LCP is no longer gated by JavaScript. The hero's fade-in animation starts from first paint because its keyframes are static CSS and its inline animation styles are in the HTML.

5.3 Scroll-reveal animations without react-reveal

Observable behaviour of the original, extracted from its source. `Fade` and `Slide` cloned their single child, added the class `react-reveal`, and set inline `opacity: 0`. On mount, if any part of the element was in the viewport it animated immediately; otherwise an IntersectionObserver with `threshold = 0.2` revealed it when the callback reported a positive intersection ratio. Revealing set inline styles `animation-fill-mode: both; animation-duration: Nms; animation-delay: 0ms; animation-iteration-count: 1; opacity: 1; animation-name: <generated>`, with keyframes inserted into a runtime stylesheet: for Fade, `from { opacity: 0; transform: translate3d(x, y, 0) } to { opacity: 1; transform: none }` where the distance string (or 100%) sets x/y by direction; for Slide the same without opacity. No timing function was set, so the browser default `ease` applied.

Replacement. `src/components/reveal/Reveal.jsx` exports `Fade` and `Slide` with the same prop names, so call sites changed only their import path. It reproduces the same class, the same inline styles, the same in-viewport test (the same arithmetic, including the 20% fraction), the same observer threshold and the same reveal-on-positive-ratio rule. Keyframes for the six combinations used on the site are static in `reveal.css` (so pre-rendered HTML can animate before JS); any other combination is inserted at runtime like the original did.

The `appear` prop is the one addition: it renders the revealed/animating state immediately. The hero uses it so the fade starts at first paint rather than after hydration. Final pixels are identical because the final keyframe is `opacity: 1; transform: none` either way.

5.4 Lottie: lazy, paused off-screen, layout-stable

Renderer choice. The four animation files were inspected: they contain 25 expressions (JavaScript evaluated per frame by the player) and two custom-slider effects. The 157 KB `lottie_light` build has no expression engine and would have animated differently, so the SVG-only `lottie_svg` build (204 KB raw, 54 KB gzipped, with expressions) was used, and lottie-web was pinned to the exact version the site already shipped (5.7.3) so frame rendering is unchanged.

Loading. The player module and the animation JSON are imported only when the animation's box comes within 300 px of the viewport. The JSON is a hashed static asset (Vite `?url` import) fetched with `fetch()` rather than a JavaScript module: JSON.parse is faster than evaluating an object literal, the file caches independently of code, and it never enters the JavaScript bundle. Two of the four boxes are `display: none` below 1456 px and 1380 px; they never intersect, so they never download anything, and the DOM shrinks by roughly 280 elements on mobile.

Pausing. A second observer pauses the animation when its box leaves the viewport and resumes it when it returns. Previously four SVG animations updated the DOM sixty times a second regardless of visibility, including the two invisible ones.

Layout stability. The measured geometry of the old implementation was: wrapper height = SVG width + 4–5 px. The extra pixels are the inline-box descender gap, because lottie-web's `<svg>` is `display: inline` and sits on the text baseline. A CSS `aspect-ratio: 1/1` placeholder would have been 4–5 px short and the page would have shifted when the player mounted. Instead the box contains an empty inline `<svg viewBox="0 0 1 1">` with the same `width: 100%; height: 100%` styles, so the pre-mount box is byte-for-byte the same height as the post-mount one. React removes the placeholder and lottie-web inserts the real SVG in the same layout pass (the player is given `animationData` so it builds the SVG synchronously inside a layout effect), so there is no frame with an empty box. CLS stayed at zero and the measured boxes match the baseline to the pixel at every viewport.

Markup parity. The wrapper carries the same inline styles and attributes react-lottie emitted (`width/height 100%; overflow hidden; margin 0 auto; outline none; role="button"; aria-label="animation"; tabindex="0"; title=""`).

5.5 Fonts: self-hosted, subset, preloaded

Font	Before	After	Subset
Montserrat Regular	245,708 B TTF	38,460 B WOFF2	Basic Latin, Latin-1, Latin Extended-A, General Punctuation, Currency, ™, Arrows
Agustina Regular	96,932 B WOFF	9,424 B WOFF2	Only the glyphs of the logo text “Udit Singh”
Font Awesome 5 Brands	76,576 B (CDN)	2,684 B	js, react, sass, html5, node, google, linkedin-in, github, kaggle, hackerrank
Font Awesome 5 Free (solid)	79,464 B (CDN)	760 B	database, quote-left, map-marker, hand-point-up
font-mfizz	42,044 B WOFF (CDN)	1,324 B	aws, git, mongodb, redis

Why this is pixel-safe. WOFF2 is a lossless container (Brotli-compressed, transformed glyph tables); subsetting with HarfBuzz keeps the retained glyphs' outlines, hinting and kerning intact. The same font files render the same glyphs. The subset boundary is the only risk: a character outside the retained ranges would fall back to the next font in the stack. The ranges cover every character on the page with wide margins (the site's text is English with “©”, en dashes and curly quotes).

What was kept on purpose. `local("Montserrat")` and `local("Agustina Regular")` remain first in `src` , and `font-display: swap` is unchanged, because both affect what a given visitor sees and the brief was parity. Only the Regular weight ships, as before, so bold text is still synthesised by the browser; shipping a real Bold would have changed pixels.

Preloads fixed. The two preload tags pointed at `/static/media/<hash>` paths without the `/udit-portfolio/` prefix and returned 404 on every load. The new `index.html` preloads all five fonts via `/src/assets/fonts/…` paths that Vite rewrites to the final hashed URLs, so the preload and the CSS reference always agree. Fonts are fetched as CORS requests, so the tags carry `crossorigin` even though they are same-origin; without it the browser would fetch twice.

Removed. Two identical `@import url(fonts.googleapis.com/…Open+Sans…Saira…)` lines. They cost a render-blocking cross-origin round trip (Lighthouse estimated 833 ms on mobile) for fonts used only by a Star Wars crawl that was commented out years ago.

Reproducibility. The original font files live in `tools/fonts-src/` and `npm run fonts:subset` regenerates every WOFF2 with `subset-font` (HarfBuzz compiled to WebAssembly). Changing the logo text or adding an icon is a one-line edit in that script.

5.6 Icons: Font Awesome and font-mfizz without CDNs

Two stylesheets (59 KB and 9 KB raw) from jsDelivr and cdnjs were render-blocking and pulled three font files for 18 icons. The CSS rules that those 18 icons actually need were copied verbatim into `src/styles/icons.css`: the `@font-face` declarations (same families, weights and `font-display: block`), the shared base rule (`display: inline-block; font-style: normal; font-variant: normal; text-rendering: auto; line-height: 1` and the font-smoothing properties), the family/weight rules for `.fa/.fas/.fab`, the per-icon `:before { content }` codepoints, and font-mfizz's base rule (which differs: `text-rendering: optimizeLegibility`, `text-decoration: inherit`). The fonts were subset from the exact files the CDN served. An alternative, inline SVG sprites, would have been smaller still but renders vector paths instead of font glyphs, which can differ at the sub-pixel level; the brief ruled it out.

The icon rules are selector-exact (`.icon-aws:before, .icon-git:before, …`) rather than attribute-prefix selectors, because the site also has an unrelated `.icon-button` class.

5.7 Images

- Three PNG project images were re-encoded as **lossless** WebP: 142,688 → 76,536 B, 135,065 → 61,628 B, 93,789 → 42,932 B. Decoded pixels were compared byte-for-byte: every visible pixel is identical; the only differences are RGB values under fully transparent pixels, which libwebp normalises and which cannot be seen. Lossy WebP or AVIF would have saved more but would not have been pixel-identical.
- The images were not resized: they are 512 px and display at 250 CSS px, which is exactly what a 2× screen needs.
- JPEG logos that CRA inlined as base64 (10 KB limit) are now hashed files (Vite limit 4 KB). Base64 inflates by a third, compresses poorly and blocks on the JavaScript it sits in.
- Project card images below the fold have `loading="lazy"` and `decoding="async"`. Their container has a fixed 250 px height, so late loading cannot shift layout.
- One file named `droneApp.png` is actually a WebP; browsers sniff the format, so it works and was left alone.
- Sixteen unused images were deleted from the source tree; they were never shipped but inflated the repository.

5.8 Third-party scripts and stylesheets

Before	After
gtag.js for UA-135618960-2 (dead) → chained gtag.js for G-001DL7NH6R; 293 KB; two long tasks on mobile	One GA4 tag. `gtag('config', 'G-001DL7NH6R')` is queued inline so nothing is lost; the script element is created in a `load` listener so it never competes with first paint or hydration
emailjs-com from jsDelivr as a synchronous head script plus `emailjs.init()` inline	Removed; the npm copy in the bundle already performs `send()` with the user id
Font Awesome CSS + fonts (jsDelivr)	Self-hosted subsets (5.6)
font-mfizz CSS + font (cdnjs)	Self-hosted subset (5.6)
Google Fonts @import	Removed (5.5)
api.github.com GraphQL (2 failing calls)	Only called when the section is enabled
twemoji.maxcdn.com image URLs (react-easy-emoji)	Removed

Net: seven origins on first load became one, plus Google Tag Manager after the page has fully loaded. Every removed origin saved a DNS lookup, a TCP handshake and a TLS negotiation on the critical path.

5.9 Dead code and dead dependencies

- **Apollo.** Both GitHub components called the GraphQL API on mount even with the section disabled. Replaced with `src/lib/github.js`, an 18-line `fetch` POST that throws on HTTP or GraphQL errors; the components call it only when `openSource.display` is true. The lazily loaded card components are unchanged.
- **ColorThief.** The banner colour of each experience card was computed in the browser by quantising the logo on a canvas after the image loaded. The exact values ColorThief produced in Chrome were captured once from the old build and stored as `bannerColor: [r, g, b]` in `portfolio.js`. The card renders in its final colour immediately, including in the pre-rendered HTML. If a future logo has no value, the component imports ColorThief on demand and computes it as before.
- **react-easy-emoji.** Every call site was audited: the only rendered strings passed through it contained no emoji characters, so unwrapping them changes nothing. Hidden sections keep their emoji as plain Unicode.
- Removed files: `serviceWorker.js` (never registered), `logo.svg`, `App.test.js` (jest came from react-scripts), the duplicate `AchivementCard.js`, the Twitter container, `public/index.html` (Vite uses the root `index.html`), `browserconfig.xml` (referenced a missing tile).

5.10 Runtime behaviour fixes

- **Back-to-top button.** The old code assigned `window.onscroll` (clobbering any other handler) and read and wrote the DOM on every scroll event. The new hook uses a passive scroll listener and coalesces work with `requestAnimationFrame`: at most one style write per frame. Same 20 px threshold, same visibility rule.

- **Tooltip.** The timeout handle was a local variable recreated on every render and cleared with `clearInterval`; it is now a ref cleared with `clearTimeout`, and cleared on unmount.
- **Sticky header spacer.** After `document.fonts.ready` the header dispatches a synthetic `resize` event, which react-headroom already handles by re-measuring. The spacer is now always the header's real height (section 2.4). This was the one deliberate visual deviation: the old overlap was a timing bug, differed by operating system (fallback font metrics) and corrected itself on any window resize, so it was not a stable pixel state to preserve.
- **Contact form.** `preventDefault()` is now unconditional, so the "Invalid Email!" state is actually shown instead of the page reloading. The pending reset timer is cleared on unmount. Comparisons use `===`. EmailJS identifiers come from `VITE_*` variables.
- **Theme.** Same logic as before (read storage after mount, default from `prefers-color-scheme`), expressed as a function component so hydration is predictable.
- **Carousels.** Unchanged logic; keys now include the start index so React does not reuse a card instance for a different company when paging.

5.11 SEO, PWA manifest and document head

- A real `<meta name="description">` (the old tag was `name="Portfolio"`, which no crawler reads), canonical URL, `og:url`, `og:description`, `og:image` (a 1200 px JPEG generated from the repository screenshot) and a Twitter card.
- `manifest.json` now names the site correctly, points at the existing favicon, and no longer references two missing icons (two 404s per load gone).
- `robots.txt`, favicon and `theme-color` unchanged. A `.nojekyll` file is deployed so GitHub Pages does not run the output through Jekyll.

5.12 Security hygiene

- The committed GitHub personal access token was removed from the source. It should be revoked in GitHub settings; GitHub's secret scanning usually revokes its own tokens found in public pushes, but that must not be assumed.
- `.env` remains tracked, as before, because the EmailJS service, template and public ids are by design shipped to the browser (they were in the old bundle too). The keys were renamed to the `VITE_` prefix. Abuse of a public EmailJS key is mitigated on the EmailJS dashboard (allowed origins, reCAPTCHA), not in the repository.
- External links keep `rel="noopener noreferrer"`.

5.13 Quirks deliberately preserved

Quirk	Where	Why kept	Removal
Literal "//" text above the Contact section	`Main.jsx` renders `{"//"}` with a comment	Visible on the live site; the brief forbade pixel changes	Delete one line

Quirk	Where	Why kept	Removal
Contact Name/Email inputs rendered at 50% scale	`ToggleSwitch.css` global `input` rule, imported from `Header.jsx`	Same	Delete the import or the rule
`StarWars.css` **imported after** `Greeting.css`	`Greeting.jsx`	Its unconditional duplicate `.greeting-main { display: flex }` comes later in the cascade than Greeting.css's `max-width: 768px { display: block }` rule and overrides it. That is why the hero stays side-by-side on phones. Deleting the file changed the mobile layout by 185 px; it was restored (minus its Google Fonts import)	Fold the intended rule into Greeting.css during the redesign
Empty bullet under the education entry	`descBullets: [""]`	Content data; renders a bullet with no text	Empty the array
Both carousel arrows have `alt="left"`	Data/markup	Not visual	Fix alt text

5.14 Repository hygiene and developer experience

- All components are `.jsx`; `portfolio.js`, `StyleContext.js` and `lib/github.js` stay `.js` because they contain no JSX. Vite's esbuild transform needs the extension to decide.
- Images are imported at the top of `portfolio.js` instead of `require()` calls inside object literals.
- `npm run dev` starts Vite with hot module replacement at `/udit-portfolio/`; `npm run preview` serves the production build; `npm run build` performs client build, SSR build and pre-render; `npm run fonts:subset` regenerates fonts; `npm run deploy` builds and pushes to `gh-pages`.
- A README documents the architecture, the quirks and the maintenance tasks. The install footprint fell from 1,648 packages to 101.

6. Verification: how parity and correctness were proven

The acceptance test was not a visual glance. Every claim in this document about “identical” is backed by a scripted measurement that can be re-run.

6.1 Preserving the baseline

Before the first new build, the old `build/` directory (which byte-for-byte matched what GitHub Pages was serving, verified against the live hashed filenames) was copied aside. Both builds were served by the same small Node static server under the same `/udit-portfolio/` sub-path with gzip for text types, so the only variable was the build itself.

6.2 Screenshot capture protocol

- Headless system Chrome driven by puppeteer-core 23, flags `--font-render-hinting=none --hide-scrollbars`, device scale factor 1.
- Six viewports: 1920×1080, 1366×768, 1280×720, 768×1024, 412×915, 375×667. Light scheme for all; dark scheme additionally at 1366 and 412 (the site switches to dark when `prefers-color-scheme: dark` is set and nothing is stored).
- A fresh incognito browser context per capture, so `localStorage` is empty and the first-visit theme logic runs.
- Wait for `networkidle0` and `document.fonts.ready`; force `scroll-behavior: auto` so programmatic scrolling is instant.
- Scroll the page in half-viewport steps with 130 ms pauses so every IntersectionObserver fires and every reveal animation runs; wait 2.2 s for the longest animation (the 2 s Slide); return to the top; dispatch a synthetic `resize` (see 6.3); wait.
- Record the bounding boxes of the four Lottie containers; take a full-page screenshot; save the DOM; scroll to 700 px and take a viewport screenshot to compare the sticky header (unpinned, translated −90 px) and the back-to-top button (visible).
- Compare with pixelmatch (threshold 0.1, anti-aliasing detection off) after painting the Lottie boxes black in both images, because animation frames legitimately differ with time. Report differing pixels and height differences.

6.3 Discrepancies found and resolved

Symptom	Diagnosis	Resolution
Tablet (768 px) page 185 px taller; hero stacked instead of side-by-side; 18.5% pixels differ	Per-section geometry probe showed `.greeting-text-div` at full width. `StarWars.css`, deleted as “duplicate”, had an unconditional `.greeting-main { display: flex }` that came after Greeting.css's 768 px media query in the cascade and overrode it	File restored (minus its Google Fonts import) and imported after Greeting.css; comment explains the dependency

Symptom	Diagnosis	Resolution
Desktop pages 12 px taller (17 px at 1920, 2 px on phones); zero differing pixels in content	Three independent probes (section offsets, footer position, header spacer through the capture steps) showed the old build's react-headroom spacer measuring 78 px before the logo font loaded and 90 px after any resize. The new build measured 90 px from the start	Classified as a timing bug in the original, not a layout; new build re-measures after fonts load; baseline capture normalised with a synthetic resize so the comparison isolates real differences
Contact form test showed "Send" instead of "Invalid Email!" and all reveal opacities reset	Native form submission reloaded the page because `preventDefault()` was conditional (original behaviour)	`preventDefault()` made unconditional
Lottie boxes measured 4–5 px taller than their SVG	Inline SVG descender gap	Placeholder is an inline SVG with the same sizing, so pre- and post-mount heights match exactly (605/609, 580/584, 854/859, 671/675, 351/355 at the tested widths)

6.4 Final pixel-diff results

Capture	Baseline size	New size	Differing pixels outside animations
1366×768 light, full page	1366×4621	1366×4621	0
1366×768 dark, full page	1366×4621	1366×4621	0
1920×1080 light, full page	1920×5870	1920×5870	0
1280×720 light, full page	1280×4632	1280×4632	0
768×1024 light, full page	768×5993	768×5993	842 (0.021%), all inside the hero animation where it overflows its masked box
412×915 light, full page	412×7129	412×7129	0
412×915 dark, full page	412×7129	412×7129	0
375×667 light, full page	375×7241	375×7241	0
All eight, scrolled to 700 px (viewport)	—	—	0 in every case

Icon glyph boxes were also compared element by element (ten skill icons and five social icons): widths and heights match to the pixel between the CDN fonts and the self-hosted subsets. Loaded font families match: Agustina Regular, Font Awesome 5 Brands, Font Awesome 5 Free, FontMfizz, Montserrat.

6.5 Functional sweep

Check	Result
Hydration: React fiber attached to pre-rendered root; console has no errors or warnings	Pass

Check	Result
Experience carousel: right ×2 shows Scaler, Altair, BYJU'S with pre-computed banner colours; left returns one step; arrows appear/disappear at the ends	Pass
Projects carousel paging	Pass
Tooltip appears after 400 ms hover with the right text and disappears on leave	Pass
Back-to-top: hidden at top, visible after scrolling, click returns to 0	Pass
Sticky header: unpinned (−90 px) when scrolling down, pinned when scrolling up; same class names as before	Pass
Contact form: labels float on input; invalid email shows "Invalid Email!" without a page reload; no email is sent during testing	Pass
Lottie: two visible animations load, two `display: none` boxes never download; box geometry equals baseline	Pass
Anchor navigation targets exist: #skills, #experience, #projects, #contact	Pass
Dark mode from stored preference applies to wrapper and header	Pass
Network: no 4xx/5xx; only first-party plus Google Tag Manager/Analytics	Pass
Dev server (`npm run dev`) serves the app at the sub-path with JSX and `?url` imports transformed	Pass

6.6 Lighthouse under equal conditions

Lighthouse 12.8.2, headless Chrome, default mobile emulation (Moto G Power class CPU slowdown, simulated slow 4G) and the desktop preset, both builds served from the same local gzip server. Local numbers exclude GitHub Pages' network latency, which is why the live mobile baseline (53) is lower than the local baseline (63); the relative improvement is what the comparison is for.

7. Results

7.1 Lab performance, same machine, same server

Metric	Old build, mobile	New build, mobile	Old build, desktop	New build, desktop
Performance score	63	98–99	97	100
First Contentful Paint	3.8 s	1.4 s	1.0 s	0.3 s
Largest Contentful Paint	5.5 s	2.1 s	1.0 s	0.5 s
Speed Index	3.8 s	1.4 s	1.0 s	0.3 s
Total Blocking Time	330 ms	60–90 ms	20 ms	0 ms
Time to Interactive	7.4 s	4.0 s	1.2 s	0.5 s
Cumulative Layout Shift	0.001	0	0.001	0
Total byte weight	1,136 KB	413 KB	1,136 KB	497 KB
Requests	33	20	33	24
JavaScript execution	1.7 s	0.6 s	0.4 s	0.2 s
Main-thread work	3.5 s	1.8 s	0.9 s	0.7 s
DOM elements	871	592	871	831

Desktop byte weight is higher than mobile because two more Lottie animations are visible at desktop widths and load.

7.2 Payload by category (new build, as served)

Category	Files	Raw	Transfer	When
HTML, pre-rendered	1	23.6 KB	9.5 KB	First
CSS	1	34.9 KB	7.3 KB	Render-blocking by design
Fonts (five WOFF2)	5	52.7 KB	52.7 KB	Preloaded, high priority
Application JS	1	55.3 KB	23.2 KB	Deferred module
React runtime JS	1	143.1 KB	45.8 KB	modulepreload
Lottie player	1	203.8 KB	53.6 KB	Only when an animation box is near
Hero animation JSON	1	168.1 KB	21.1 KB	Same
Skills animation JSON	1	125.9 KB	9.8 KB	Same
First-screen and fold images	5	115.7 KB	115.7 KB	Logos immediately, project image lazily
Google Analytics 4	1	~150 KB	~150 KB	After `window.load`

Before: 258 KB of gzipped JavaScript and 343 KB of fonts had to arrive before anything painted, plus 213 KB of icon CSS and fonts from two other origins and 293 KB of analytics competing for bandwidth. After: 17 KB of HTML and CSS paints the page, 69 KB of JavaScript hydrates it, and the rest arrives when it is needed.

7.3 What moved the needle most

1. Pre-rendering: removed roughly 7 s of mobile render delay from LCP by letting HTML paint before JavaScript.
2. Removing render-blocking third-party CSS (Font Awesome, font-mfizz, Google Fonts) and the synchronous emailjs script: Lighthouse attributed about 4.3 s of blocking to these on mobile.
3. Lazy Lottie: 98 KB gzipped of player and JSON moved off the critical path, 700 ms of script evaluation deferred, four perpetual animations reduced to the visible ones.
4. Deleting Apollo, dotenv, emoji conversion, Twitter embed and the UA analytics chain: ~45 KB gzipped of JavaScript and ~150 KB of analytics gone, two long tasks gone.
5. Fonts: 343 KB to 53 KB, correctly preloaded, so text swaps to the real face almost immediately.
6. Modern output: no IE 11 transforms, no babel-runtime, no source maps, React 18.

7.4 Repository and maintenance

Files changed	106 (34 added, 39 deleted, 24 renamed, 9 modified)
Runtime dependencies	17 → 7
Installed packages	1,648 → 101
Build output	2.0 MB + 2.5 MB source maps → 1.3 MB, no maps
Build time	~2 s client + 0.5 s SSR + pre-render

8. Operations: build, deploy, maintain

8.1 Commands

Command	Effect
`npm run dev`	Vite dev server with HMR at `http://localhost:5173/udit-portfolio/` (client-rendered; no pre-render in dev).
`npm run build`	Client build → SSR build → pre-render into `build/index.html`; the temporary `.ssr-build` directory is removed.
`npm run preview`	Serves `build/` locally for a production check.
`npm run deploy`	Runs the build, then pushes `build/` (including `.nojekyll`) to the `gh-pages` branch.
`npm run fonts:subset`	Regenerates the five WOFF2 files from `tools/fonts-src`.

8.2 Environment

`.env` holds `VITE_serviceId`, `VITE_templateId`, `VITE_userId` (EmailJS) and optionally `VITE_GITHUB_TOKEN`. Vite inlines `VITE_`-prefixed variables at build time; nothing else from the environment reaches the bundle.

8.3 Hosting notes

GitHub Pages serves gzip (not Brotli) and sets `Cache-Control: max-age=600` on every file, including hashed assets, so returning visitors revalidate after ten minutes. The build is compatible with any static host; moving to Cloudflare Pages, Netlify or Vercel would add Brotli, immutable caching for hashed assets and HTTP/3 with no code change, and would typically drop the 0.8 s mobile TTFB measured on Pages.

8.4 Maintenance recipes

- **Add an icon.** Add its `.fa-name:before { content }` (or `.icon-name:before`) rule in `src/styles/icons.css`, add the codepoint to the matching list in `scripts/subset-fonts.mjs`, run `npm run fonts:subset`.
- **Change the logo text.** Edit `greeting.username` and `LOGO_TEXT` in the subset script, then regenerate; otherwise new letters fall back to the next font.
- **Add a work-experience logo.** Import the image in `portfolio.js`. Either omit `bannerColor` (computed in the browser, ColorThief loads lazily) or run ColorThief once and store the triple for an instant, pre-rendered colour.
- **Enable the GitHub section.** Set `VITE_GITHUB_TOKEN`, set `openSource.display: true`, test with a real token: the fetch path replaced Apollo and was verified only statically because the section is disabled.
- **Re-verify pixels after CSS changes.** The capture/diff scripts live in the session scratchpad and are described in section 6; the approach (preserve old build, capture both, mask animations, pixelmatch) is reproducible in a few dozen lines.

9. What is next

9.1 Ready for the UI/UX revamp

All components are function components on React 18 with no abandoned libraries. Content is in one data file. Fonts, icons and animations have documented regeneration paths. The two visual quirks have one-line removals. Dark mode is a wrapper class plus per-element `isDark` class names; a redesign should move theming to CSS custom properties on the root so components stop branching on a prop and the theme can be set before paint by a tiny inline script without any hydration concern.

9.2 Remaining opportunities (deliberately not taken because they change pixels or behaviour)

- Lossy WebP/AVIF for the project images (another ~60% smaller) and an LCP-image preload if the redesign introduces a hero image.
- A real Montserrat Bold instead of synthesised bold, and `font-display: optional` or `size-adjust` metric-matched fallbacks for a swap-free load.
- Respecting `prefers-reduced-motion` by showing the first Lottie frame statically.
- Inlining the 7 KB of CSS into the HTML to save one round trip on first visit, at the cost of caching it separately.
- Preloading the hero animation JSON so the illustration appears sooner after hydration.
- Real-user monitoring with the `web-vitals` library reporting LCP, INP and CLS to GA4, so lab gains are confirmed in the field.
- A Lighthouse CI budget and a pixel-regression job in GitHub Actions on every pull request.
- Hosting with Brotli and immutable caching.
- Accessibility: meaningful alt text on the carousel arrows, the half-size inputs, heading order, visible focus styles, the Lottie box's `role="button"` (kept for parity, but it is not a button).

10. Interview preparation

This section turns the project into interview material. Every answer is written as a short lesson: first the idea in plain language, then what actually happens inside the browser or React, then how it showed up in this project with real numbers, then the follow-up questions an interviewer is likely to ask, and finally one or two sentences you can say out loud. Read it in order the first time; afterwards use the rapid-fire glossary at the end (10.9) for revision.

10.1 Telling the story

THE PROJECT IN ONE PARAGRAPH

"My portfolio was a 2020 create-react-app site. It looked fine but on a mobile connection it stayed blank for five seconds, because it was fully client-rendered behind about 250 KB of JavaScript and render-blocking CSS from three content-delivery networks. I audited it with Lighthouse and source-map attribution, then rebuilt it on Vite and React 18 with build-time pre-rendering so the HTML paints before any script runs. I self-hosted and subset every font including the icon fonts, made the Lottie animations download and play only when visible, deleted a GraphQL client, an emoji converter and a dead analytics tag, and wrote a small replacement for an abandoned animation library. My constraint was zero pixel change, and I proved it with headless-Chrome screenshot diffs at six viewports in both colour schemes. Lighthouse mobile went from 53 to 98, LCP from 8 seconds to about 2, the critical JavaScript from 258 KB to 69 KB gzipped, and the dependency tree from about 1,600 packages to 100."

THE SHAPE OF A LONGER ANSWER

1. **Situation.** Template-generated React 16 site, webpack 4 toolchain that needed an OpenSSL legacy flag, 17 runtime dependencies, deployed to GitHub Pages.
2. **Diagnosis.** Lighthouse LCP breakdown showed 7.1 s of "render delay" on mobile: the page could not paint until JavaScript ran. Source maps showed 41% of the vendor bundle was the Lottie player, 20% an Apollo GraphQL client for a disabled section. Four third-party stylesheets and a synchronous script were render-blocking. Fonts were 343 KB with broken preloads.
3. **Plan, in order of impact.** Pre-render HTML; remove render-blocking third parties; make Lottie lazy; remove dead dependencies; subset and self-host fonts; fix images and analytics; modernise the toolchain so the redesign starts clean.
4. **Constraint and proof.** Pixel parity. Preserved the old build, captured both with a deterministic script, masked animated regions, diffed with pixelmatch, probed geometry when the diff disagreed.
5. **What the proof caught.** A stylesheet that looked duplicate but held a cascade-order dependency (tablet layout changed by 185 px when deleted), a header spacer that depended on a font-loading race, a form that reloaded the page on invalid input.
6. **Result and next.** The numbers above; next is field data with the web-vitals library, CI budgets, and a host with Brotli and immutable caching.

NUMBERS WORTH MEMORISING

Number	What it is
53 → 98–99	Lighthouse mobile performance (live baseline vs lab after; 63 → 98–99 on the same local server)
8.0 s → 2.1 s	Mobile LCP (live before vs local after; 5.5 s → 2.1 s like-for-like)
7.1 s	LCP “render delay” before: time the text waited for JavaScript
258 KB → 69 KB	Gzipped JavaScript needed before interaction (Lottie player, 54 KB, is now lazy)
1,136 KB → 413 KB	Total first-visit transfer on mobile
7 → 1	Origins contacted on the critical path (analytics loads after `load`)
343 KB → 53 KB	Fonts (plus 213 KB of icon CDN assets gone)
41% / 20%	Share of the old vendor bundle: lottie-web / Apollo + graphql
1,648 → 101	Installed npm packages
0 px	Differing pixels outside animations across 8 viewport/scheme captures
12 px, 185 px	Header spacer race; StarWars.css cascade dependency

10.2 Foundations: how browsers load and render pages

What are the Core Web Vitals, what does each one measure, and what are the thresholds?

SIMPLE EXPLANATION

Core Web Vitals are Google's three user-centred measurements of a page: how fast the main content shows up (Largest Contentful Paint, LCP), how quickly the page reacts when the user does something (Interaction to Next Paint, INP), and how much the layout jumps around while loading (Cumulative Layout Shift, CLS). They were chosen because they map to what a person actually feels: “is it there yet”, “did it respond”, “did it move under my finger”.

HOW THEY WORK

LCP is the time, from navigation start, at which the largest text block or image inside the viewport finished rendering. Good is 2.5 seconds or less, poor is more than 4 seconds. INP looks at every click, tap and key press during the whole visit, measures how long each took until the next frame was painted, and reports roughly the worst one (ignoring one outlier per fifty interactions). Good is 200 ms or less, poor is above 500 ms. INP replaced First Input Delay in March 2024 because FID only measured the delay before the first handler started, not the whole interaction. CLS adds up unexpected layout shift scores inside the worst five-second window of the visit. Good is 0.1 or less, poor is above 0.25. Google evaluates all three at the 75th percentile of real page loads, separately for mobile and desktop, so one slow user out of four is what the score reflects.

IN THIS PROJECT

LCP was the broken vital: 8 seconds on mobile in the live Lighthouse run. CLS was already essentially zero (0.001) and had to stay there while I made content load lazily, which is why the Lottie placeholder work mattered. INP is a field metric and cannot be measured in Lighthouse; its lab proxy, Total Blocking Time, fell from 330 ms to under 90 ms.

FOLLOW-UPS

- "Why the 75th percentile?" It is a balance: strict enough to represent most users, lenient enough not to be dominated by outliers like a user on a train.
- "Can Lighthouse measure INP?" No, INP needs real interactions. Lighthouse reports TBT as a proxy and some tools simulate interactions.
- "What about FCP and TTFB?" They are not Core Web Vitals but they are diagnostic: a slow TTFB caps how good LCP can be.

How to say it: "The three vitals are LCP for loading, INP for responsiveness and CLS for stability, judged at the 75th percentile. My problem was LCP at 8 seconds, caused entirely by waiting for JavaScript."

Explain FCP, LCP, Speed Index, TTI and TBT and how they relate to each other.

SIMPLE EXPLANATION

These are the loading metrics you see in a Lighthouse report. Think of a page load as a film: FCP is the first frame with anything on it, Speed Index is how quickly the whole picture fills in, LCP is when the main subject is on screen, TTI is when the page is reliably ready to respond, and TBT is how much of the time between FCP and TTI the main thread was too busy to respond.

HOW THEY WORK

First Contentful Paint is the first time any text, image, canvas or SVG is painted. **Speed Index** is computed from a filmstrip: it measures the average time at which visible parts of the page are displayed, so a page that shows most of its content early scores well even if a small part comes late. **Largest Contentful Paint** is the render time of the largest element in the viewport; the candidate can change as bigger things appear, and the browser reports the final one. **Time to Interactive** is the point after FCP where the main thread has had no long task for five seconds and there are at most two in-flight network requests. **Total Blocking Time** adds up, for every task longer than 50 ms between FCP and TTI, the part beyond 50 ms; a 120 ms task contributes 70 ms. Lighthouse 10 to 12 weights the performance score as TBT 30%, LCP 25%, CLS 25%, FCP 10% and Speed Index 10%.

IN THIS PROJECT

Before, mobile FCP and LCP were 3.8 s and 5.5 s on the local server (5.5 s and 8.0 s live) and almost the same thing, because nothing painted until React rendered everything. After, FCP is 1.4 s (the pre-rendered HTML and CSS), LCP 2.1 s (the hero text in its final web font), Speed Index 1.4 s, TBT 60–90 ms (down from 330 ms), TTI 4.0 s (down from 7.4 s; it waits for the lazily loaded Lottie player to settle).

FOLLOW-UPS

- "Why is TBT weighted highest?" Because it predicts whether taps will feel laggy, which users punish most.
- "Why would TTI still be 4 s when LCP is 2 s?" Lazy work after first paint (the Lottie player) keeps the main thread briefly busy; TTI waits for a quiet window.

How to say it: "FCP is first paint, LCP is main content, Speed Index is how fast the picture fills, TBT is how blocked the main thread was before interactive. I attacked LCP first because it was 8 seconds and entirely render delay."

Walk me through the critical rendering path: what happens between the HTML arriving and pixels appearing?

SIMPLE EXPLANATION

The browser has to build two trees from your files, combine them, work out where everything goes, draw it, and then put the drawn layers together. Anything that stops one of those steps delays the first pixel.

HOW IT WORKS

1. **Parse HTML into the DOM.** The parser builds nodes as bytes arrive. A *preload scanner* races ahead to discover `<link>`, `<script>` and `<img>` URLs so downloads start early.
2. **Parse CSS into the CSSOM.** Stylesheets in the head are *render-blocking*: the browser will not paint until it has them, because painting with partial styles would be wrong and flashy. A stylesheet that contains `@import` blocks until the imported file is also fetched, one round trip later.
3. **Scripts.** A classic `<script>` without `async` / `defer` is *parser-blocking*: parsing stops until it downloads and runs, because the script might call `document.write`. It also waits for pending stylesheets, since the script might read styles.
4. **Render tree.** DOM plus CSSOM, minus anything `display: none`.
5. **Layout (reflow).** Compute the geometry of every box.
6. **Paint.** Turn boxes into drawing commands, rasterise into layers.
7. **Composite.** The compositor thread stacks layers and sends a frame to the GPU. Properties like `transform` and `opacity` can be animated here without redoing layout or paint.

Web fonts do not block rendering, but text that needs a font that has not arrived may be hidden briefly depending on `font-display`. Images do not block the first paint either.

IN THIS PROJECT

The old page had three render-blocking stylesheets from other origins (Font Awesome from jsDelivr, font-mfizz from cdnjs, Google Fonts via `@import` inside the main CSS), one synchronous script (emailjs from jsDelivr), and an empty DOM until 255 KB of gzipped JavaScript ran. So the render tree was empty for seconds. The new page has one 7 KB same-origin stylesheet, a deferred module script, and a complete DOM in the HTML, so layout and paint happen as soon as the CSS arrives.

FOLLOW-UPS

- "Where does hydration fit?" After the first paint: the DOM exists; React attaches to it later.
- "Why do stylesheets block but fonts not?" A missing stylesheet would change every box; a missing font changes only glyphs, and the browser can fall back temporarily.

How to say it: "DOM, CSSOM, render tree, layout, paint, composite. CSS and sync scripts block the path; my old page had four blocking third-party resources and an empty DOM, the new one has one small stylesheet and pre-rendered HTML."

What is the difference between a normal script, async, defer and type="module"?

SIMPLE EXPLANATION

These attributes tell the browser when a script may run relative to parsing the page. The wrong choice makes the browser stop building the page to wait for JavaScript.

HOW IT WORKS

- **Plain `<script src>`** : parsing pauses, the script downloads and executes immediately, then parsing resumes. Blocks everything below it.
- **`async`** : downloads in parallel while parsing continues, then executes as soon as it arrives, pausing the parser at that moment. Order between async scripts is not guaranteed. Right for independent third-party code that must run early, such as analytics (though even that can be deferred).
- **`defer`** : downloads in parallel, executes after parsing finishes, in document order, before `DOMContentLoaded` . The DOM is complete when it runs.
- **`type="module"`** : deferred by default, strict mode, its own scope, supports `import` ; dependencies are fetched as a graph. `async` can be added to run as soon as the graph is ready.

IN THIS PROJECT

The old HTML had a synchronous emailjs script in the head, which blocked parsing while it fetched from a third-party CDN; it was redundant because the npm copy was in the bundle. Vite emits a `type="module"` script for the app plus a `modulepreload` for the React chunk, so hydration code never blocks the pre-rendered HTML. The GA4 script is created by a small inline listener on `window.load` , which is later than `defer` , so analytics cannot compete with first paint or hydration.

FOLLOW-UPS

- "Where should scripts go, head or body end?" With `defer` /modules it does not matter for blocking; head lets the preload scanner find them earlier.
- "Does `async` guarantee execution before `load` ?" Yes, async scripts still delay the `load` event.

How to say it: "Plain scripts block parsing, async runs on arrival, defer runs after parsing in order, modules are deferred by default. I removed the one sync third-party script and load analytics after the load event."

What does "render-blocking" mean exactly, and how do you find and remove render-blocking resources?

SIMPLE EXPLANATION

A render-blocking resource is one the browser insists on finishing before it will draw anything. The user looks at a blank screen for as long as the slowest one takes.

HOW IT WORKS

Stylesheets referenced in the head block rendering (not parsing). Each `@import` inside a stylesheet is another blocking fetch discovered late. Synchronous scripts block parsing, which indirectly blocks rendering of everything after them. Inline scripts that appear before content run immediately. A stylesheet with a non-matching `media` attribute (for example `media="print"`) is fetched with low priority and does not block. Lighthouse's "Eliminate render-blocking resources" audit lists them with an estimated saving; the Network panel shows their priority as Highest and the Performance panel shows the paint waiting for them.

Fixes, in order of preference: delete it; self-host it so there is no extra connection; inline the critical part and load the rest asynchronously (the `media="print" onload="this.media='all'"` trick or `rel="preload" as="style"` swapped to stylesheet); split CSS so only above-the-fold rules block; turn a sync script into `defer` .

IN THIS PROJECT

Lighthouse estimated 4.3 seconds of savings on mobile from render-blocking resources: the Google Fonts import (833 ms), font-mfizz CSS (784 ms), Font Awesome CSS (911 ms), the emailjs script (151 ms), and the two JavaScript bundles. I deleted the Google Fonts import (its fonts were never used), replaced both icon stylesheets with a self-hosted 2 KB rule set, removed the duplicate script, and moved the application to a deferred module. The only remaining blocking resource is the site's own 7 KB stylesheet, which is as it should be.

FOLLOW-UPS

- "Why not inline all the CSS?" With one small same-origin file over HTTP/2 the extra round trip is small, and the file caches separately; inlining 7 KB into every HTML response is a reasonable alternative, listed as a follow-up.
- "Do fonts block?" Not rendering, but with `font-display: block` they hide text for up to three seconds.

How to say it: "Blocking means paint waits. I found four third-party blockers with Lighthouse, deleted two, self-hosted two, and left only my own small stylesheet."

Why does each additional origin cost time, and what do dns-prefetch and preconnect do about it?

SIMPLE EXPLANATION

Before a browser can ask a new server for a file, it has to find the server's address, open a connection, and agree on encryption. That takes several round trips, and on a phone each round trip can be 100 ms or more. Every extra domain you load from pays that tax.

HOW IT WORKS

DNS lookup (one round trip to a resolver, often cached), TCP three-way handshake (one round trip), TLS handshake (one round trip with TLS 1.3, two with 1.2). Then the connection starts with a small congestion window, so the first few packets are slow until it ramps up. On mobile with 150 ms RTT, a new origin costs roughly 300 to 450 ms before the first byte, plus slow-start. HTTP/2 lets one connection carry many requests, but only to the same origin. `<link rel="dns-prefetch">` does the lookup early; `<link rel="preconnect">` does lookup plus TCP plus TLS, which is more useful but holds a connection open, so use it for at most a few origins you will certainly use.

IN THIS PROJECT

The old page touched seven origins on the critical path: GitHub Pages, jsDelivr (two different paths), cdnjs, Google Fonts, Google Tag Manager, Google Analytics, api.github.com. Lighthouse flagged about 300 ms of preconnect savings on mobile. Instead of adding preconnects, I removed the origins: everything the first paint needs now comes from the page's own origin, and analytics is the only third party, loaded after `load`.

FOLLOW-UPS

- "Isn't a CDN faster than my origin?" For a static site on a CDN-backed host (GitHub Pages is on Fastly), your own origin is already a CDN; a second CDN only adds a connection.
- "When would you preconnect?" For a third party you cannot remove and will definitely use early, like a video CDN on a video page.

How to say it: "DNS plus TCP plus TLS is three round trips per new domain, half a second on mobile. I went from seven origins to one on the critical path rather than preconnecting to six."

Explain preload, prefetch, modulepreload and preconnect, and when each is the right tool.

SIMPLE EXPLANATION

These are hints you put in the HTML to tell the browser about resources before it would otherwise discover them. The browser normally learns about a font only when it parses the CSS and finds text that needs it; a hint lets it start the download much earlier.

HOW IT WORKS

- **`<link rel="preload" href as="font" crossorigin>`:** fetch this now at high priority because the current page needs it soon. Requires `as` so the browser knows the priority and the request headers to use. For fonts it must carry `crossorigin` because font requests are always CORS requests; without it the browser fetches the font twice and warns that the preload was unused. Typical targets: fonts, the LCP image, critical scripts discovered late.
- **`rel="prefetch"`:** fetch at lowest priority when idle, for something a *future* navigation will need. Stored in the HTTP cache.
- **`rel="modulepreload"`:** fetch and also parse and compile an ES module ahead of its import so the import resolves instantly; Vite emits one for each statically imported chunk of the entry.
- **`rel="preconnect"`:** warm a connection to another origin; no resource is fetched.

Misusing preload hurts: preloading things that are not used wastes bandwidth and logs a console warning; preloading too much competes with the LCP resource.

IN THIS PROJECT

The old HTML preloaded two fonts at paths that returned 404 on every load: they used `/static/media/<hash>` without the repository sub-path, and hard-coded a hash that would change at the next build. So the browser wasted two requests and still discovered the fonts late. The new HTML preloads all five WOFF2 files using source paths that Vite rewrites to the final hashed URLs at build time, so the preload and the CSS reference always agree, and emits a `modulepreload` for the React chunk.

FOLLOW-UPS

- "Should you preload the Lottie JSON?" It would speed up the illustration but compete with fonts and CSS for bandwidth on slow connections; I chose to prioritise text and listed it as a follow-up.
- "What about `fetchpriority`?" An attribute on `img`, `link` and `script` to raise or lower priority; `fetchpriority="high"` on an LCP image is the modern replacement for many preloads.

How to say it: "Preload is 'I need this now', prefetch is 'I will need this later', modulepreload is preload plus compile for ES modules, preconnect just opens a connection. The old preloads 404ed; the new ones are generated from the build so they cannot drift."

How do web fonts load, and what do the font-display values mean?

SIMPLE EXPLANATION

A web font is downloaded only when the page actually needs it for some text. While it downloads, the browser has to decide whether to show the text in a temporary font or hide it. `font-display` is how you tell it which.

HOW IT WORKS

The browser parses `@font-face` rules but does not fetch any font until the style resolution of some element selects that family and a character needs a glyph from it. That makes font loading lazy and late, which is why preload exists. Once a fetch starts, the font goes through a *block period* (text invisible, drawn with an invisible fallback so layout proceeds) and a *swap period* (fallback visible, swapped when the font lands):

- `auto` : browser default, usually like `block` .
- `block` : about 3 s invisible, then fallback, swap whenever the font arrives (Flash of Invisible Text, FOIT).
- `swap` : near-zero block, fallback shown immediately, swap when loaded (Flash of Unstyled Text, FOUT, and a possible layout shift).
- `fallback` : about 100 ms block, about 3 s swap window, after which the fallback stays.
- `optional` : about 100 ms block, no swap; use the web font only if it is already cached or arrives almost instantly. Most stable, least brand-consistent on first visit.

Layout shift on swap is avoided with metric overrides on the fallback (`size-adjust` , `ascent-override` , `descent-override` , `line-gap-override`) so the fallback occupies the same space as the real font.

IN THIS PROJECT

Montserrat and Agustina use `swap` , unchanged from the original, because switching to `optional` would change what slow-connection visitors see. Font Awesome uses `block` , copied verbatim from its stylesheet, so icon glyphs never show as placeholder characters. The lazy-fetch rule is the root cause of the header bug: the logo text only existed after React rendered, so the font request started at mount and the header was measured with the fallback font.

FOLLOW-UPS

- "How do you know when fonts are ready in JavaScript?" `document.fonts.ready` is a promise; `document.fonts.load()` can trigger a specific face.
- "Why not `optional` for everything?" It is a great default for body text on new designs; here parity forbade it.

How to say it: "Fonts load lazily when text needs them; font-display controls whether text hides or swaps meanwhile. I kept swap for parity and fixed the preloads so the swap window is tiny."

What are WOFF2 and font subsetting, why are they safe for pixel parity, and how much did they save?

SIMPLE EXPLANATION

A font file is a database of glyph outlines plus tables for spacing, kerning and ligatures. WOFF2 is a compressed container for that database. Subsetting deletes glyphs the page will never need, like Cyrillic letters on an English page. Neither changes the shape of the glyphs you keep.

HOW IT WORKS

WOFF2 applies a reversible transform to the glyph table (storing coordinates more compactly) and compresses everything with Brotli; decoding gives back the original outlines, so it is lossless. Subsetting, done with HarfBuzz here, computes the *glyph closure* for the requested code points: the glyphs they map to through the character map, plus any glyphs reachable through substitution rules (ligatures, alternates) and composite references. It then rebuilds the tables and prunes kerning and positioning data to the retained glyphs. Risks: dropping hinting (if the tool strips it), dropping layout tables (kerning changes spacing), or omitting a character the page actually uses, which falls back to the next font in the stack.

IN THIS PROJECT

Montserrat Regular	245,708 B TTF → 38,460 B WOFF2 (Latin, Latin-1, Latin Extended-A, punctuation, currency, ™, arrows)
Agustina Regular	96,932 B WOFF → 9,424 B WOFF2 (only "Udit Singh")
Font Awesome Brands / Solid	76,576 / 79,464 B → 2,684 / 760 B (10 and 4 glyphs)
font-mfizz	65,864 B TTF → 1,324 B (4 glyphs)

Icon glyph boxes were measured element by element on both builds and matched to the pixel. The originals and a regeneration script live in the repository so a new icon or a changed name is a one-line edit.

FOLLOW-UPS

- "What about a visitor whose name has an accent?" Content is fixed; the subset covers Latin-1 and Extended-A anyway. For user-generated text you would split by `unicode-range` instead of guessing.
- "Did you keep hinting?" HarfBuzz keeps it by default; rendering was compared on Windows. A cross-platform check would also run on macOS, which ignores hints.

How to say it: "WOFF2 is lossless compression; subsetting removes unused glyphs. Same outlines, same pixels, 85% fewer font bytes."

Compare gzip and Brotli. What compresses well, what does not, and what did GitHub Pages serve?

SIMPLE EXPLANATION

Both are lossless compressors a server applies to text responses so fewer bytes travel over the network; the browser decompresses on arrival. Brotli is newer and usually smaller.

HOW IT WORKS

gzip (DEFLATE) finds repeated byte sequences within a 32 KB window and Huffman-codes them. Brotli uses a much larger window, context modelling, and a built-in 120 KB dictionary of common web strings, so it typically produces 15 to 25% smaller output for HTML, CSS and JavaScript at comparable speed. Text compresses 3 to 8 times; JSON with repetitive keys even more. Already-compressed formats (WOFF2, WebP, AVIF, JPEG, PNG, MP4) gain nothing and may grow slightly, so servers skip them. The browser advertises support with `Accept-Encoding: gzip, br` and the server answers with `Content-Encoding`.

IN THIS PROJECT

GitHub Pages returned `content-encoding: gzip` only, with `cache-control: max-age=600` . So I measured JavaScript and CSS by gzipped size and fonts and images by raw size, and reproduced gzip in my local test server so Lighthouse transfer sizes were realistic. The Lottie JSON compressed about 8× (168 KB to 21 KB), the vendor bundle about 3.7× (642 KB to 172 KB). Moving to a host with Brotli would shave roughly another 15% off the text assets with no code change.

FOLLOW-UPS

- "Static or dynamic compression?" For a static site, precompress at build time at Brotli level 11 and let the server pick the file; dynamic compression uses lower levels to stay fast.
- "Why does base64 compress badly?" It spreads 6 bits across 8-bit characters with an alphabet that defeats pattern matching, which is one reason inlined images are a poor trade.

How to say it: "Both are lossless; Brotli is 15–25% smaller on text thanks to a bigger window and a web dictionary. Pages only did gzip, so I compared gzipped numbers and listed Brotli as a hosting win."

How does HTTP caching work for a static site, and what did GitHub Pages do?

SIMPLE EXPLANATION

Caching lets a returning visitor reuse files they already downloaded. The server tells the browser how long each file is safe to reuse. The trick for static sites is to make file names change when content changes, so you can cache them for a year without ever serving stale code.

HOW IT WORKS

`Cache-Control: max-age=N` says the response is fresh for N seconds; `immutable` tells the browser not to revalidate even on reload; `no-cache` means store but revalidate every time (using `ETag` or `Last-Modified` and getting a cheap 304 when unchanged); `no-store` means never store. The standard static-site pattern: hashed assets (`app-D3qm7iry.js`) get `max-age=31536000, immutable` ; the HTML, whose name never changes, gets `no-cache` or a short max-age so a new deploy is picked up immediately and then pulls the new hashed files. `stale-while-revalidate` lets the browser use a slightly stale copy while fetching a fresh one in the background. Since Chrome 86 the cache is partitioned by top-level site, so a file cached while visiting one site is not reused on another, which removed the old argument for shared CDN URLs.

IN THIS PROJECT

GitHub Pages sets `max-age=600` on every file and ignores anything you might want to configure, so returning visitors revalidate everything after ten minutes. The build still follows the pattern (hashed assets, stable HTML name, a separate long-lived React chunk) so it benefits immediately on any host that honours it. The old deploy also shipped 2.5 MB of source maps; they are not fetched unless DevTools opens, but they should not be public anyway.

FOLLOW-UPS

- "How do you invalidate a CDN cache?" You mostly do not: hashed names make invalidation unnecessary; only the HTML needs a short TTL or a purge on deploy.
- "What about a service worker cache?" A second layer under your control, useful for offline, but it adds an update-flow problem (users seeing an old version until all tabs close).

How to say it: "Hash the assets, cache them forever, keep the HTML short-lived. Pages forces ten minutes on everything, so that is a hosting limitation I documented rather than a code one."

What is the difference between HTTP/1.1, HTTP/2 and HTTP/3, and does HTTP/2 make request count irrelevant?

SIMPLE EXPLANATION

HTTP/1.1 handles one request at a time per connection, so browsers open about six connections per domain and queue the rest. HTTP/2 lets one connection carry many requests at once. HTTP/3 does the same over a new transport that recovers from packet loss better. Fewer requests still help, but fewer bytes and fewer domains help more.

HOW IT WORKS

HTTP/2 multiplexes streams over one TCP connection, compresses headers (HPACK), and supports prioritisation. But because TCP delivers bytes in order, one lost packet stalls every stream on that connection (transport head-of-line blocking). HTTP/3 runs over QUIC on UDP: each stream recovers independently, the handshake is faster (0 or 1 round trip), and connections survive network changes (Wi-Fi to cellular). All versions still pay per-origin connection costs, per-request header and scheduling overhead, and bandwidth for bytes.

IN THIS PROJECT

Request count fell from 33 to 20 on mobile mostly as a side effect of removing origins and dead resources, not as a goal. I did not combine files for the sake of it; the app chunk, React chunk, CSS and fonts are separate files because they cache and prioritise differently. GitHub Pages serves over HTTP/2; a host with HTTP/3 would help users on lossy mobile networks most.

FOLLOW-UPS

- "Should you bundle less with HTTP/2?" Somewhat: dozens of small files are fine, hundreds still hurt because of per-request overhead and compression efficiency.
- "Is HTTP/2 push useful?" It was removed from Chrome; `103 Early Hints` and preload replaced it.

How to say it: "HTTP/2 multiplexes one connection, HTTP/3 fixes loss recovery with QUIC. Requests still cost something, but bytes and origins cost more, so I attacked those."

What is the main thread, what is a long task, and why do they matter for responsiveness?

SIMPLE EXPLANATION

The browser runs your JavaScript, style calculation, layout and most painting on one thread per page. If a piece of work takes a long time, the page cannot respond to a tap until it finishes. Anything over 50 ms is called a long task because humans start to notice delays around 100 ms, and the tap itself needs some of that budget.

HOW IT WORKS

The event loop takes one task at a time: run a script, handle an event, run a timer. Between tasks, the browser can render a frame (style, layout, paint) and handle input. A 300 ms task means any input during those 300 ms waits. Lighthouse's TBT sums the excess over 50 ms; INP in the field measures what the user actually waited. The Performance panel shows long tasks with red corners; the `PerformanceObserver`

for `longtask` entries reports them in code. Mitigations: do less (remove code), do it later (defer, lazy load), do it in pieces (yield with `scheduler.yield()` or `setTimeout` between chunks), or do it elsewhere (Web Worker).

IN THIS PROJECT

Lighthouse found four long tasks on the old mobile load: the main chunk evaluating 300 KB of inline Lottie JSON (344 ms), the two gtag scripts (138 ms and 70 ms), and the initial HTML parse. After: the Lottie JSON is fetched as data rather than parsed as code, gtag loads after `load` and is a single script, and the remaining long task is the lazily loaded player initialising, outside the TBT window in most runs. TBT fell from 330 ms to 60–90 ms.

FOLLOW-UPS

- "Why is parsing JSON as JavaScript slower than JSON.parse?" JSON grammar is simpler; V8 documented that `JSON.parse` of a string is faster than parsing an equivalent object literal, especially for large payloads.
- "When would you use a Web Worker?" For CPU-heavy work that does not touch the DOM: image processing, parsing large data, some Lottie renderers via OffscreenCanvas.

How to say it: "One thread does everything; tasks over 50 ms block input. I removed the two biggest long tasks by fetching Lottie JSON as data and deferring analytics."

What causes layout shift, how is CLS calculated, and how do you prevent shifts when content loads late?

SIMPLE EXPLANATION

Layout shift is when something on the page moves after it has been shown, usually because something else appeared above it or changed size. CLS is a score for how much and how far things moved.

HOW IT WORKS

Each shift gets a score equal to *impact fraction* (the share of the viewport affected by elements that moved, before and after) multiplied by *distance fraction* (how far they moved relative to the viewport's larger dimension). Shifts within 500 ms of a user interaction are excluded as expected. Scores are grouped into session windows (shifts less than a second apart, up to five seconds) and CLS is the largest window. Common causes: images without dimensions, late-loading fonts with different metrics, ads or banners inserted at the top, components that mount after hydration with a different height than their placeholder, and animating `top` / `height` instead of `transform`. Prevention: always give images width and height (or `aspect-ratio`), reserve space for anything that arrives late, use metric-matched fallback fonts, and animate only transform and opacity.

IN THIS PROJECT

The biggest risk I introduced was lazy Lottie: four boxes that previously had their SVG at first render now get it later. The old wrapper measured SVG width plus 4–5 px (the inline SVG descender gap). A CSS `aspect-ratio: 1/1` placeholder would have been square and 4–5 px short, shifting everything below when the player mounted. I used an empty inline SVG with the same sizing as a placeholder, so the box is identical before and after, and swapped it in a layout effect so no frame has an empty box. CLS stayed 0. Project images below the fold sit in fixed-height containers, so their lazy loading cannot shift anything either.

FOLLOW-UPS

- "How do you debug CLS?" Lighthouse's layout-shift elements audit, the Performance panel's Experience track, and the `layout-shift` PerformanceObserver with `sources`.
- "Does `font-display: swap` cause CLS?" Yes if the fallback has different metrics; fix with `size-adjust` overrides or `optional`.

How to say it: "CLS is impact times distance over the worst five-second window. Lazy loading is where you introduce shifts, so my placeholder reproduced the exact final geometry and CLS stayed at zero."

WebP versus PNG and JPEG, lossless versus lossy, and how do you prove an image conversion did not change pixels?

SIMPLE EXPLANATION

PNG is lossless and good for graphics with flat colours and transparency; JPEG is lossy and good for photos. WebP does both jobs with better compression; AVIF compresses even better for photos but is slow to encode. "Lossless" means decoding gives back exactly the same pixels; "lossy" means some detail is thrown away in exchange for size.

HOW IT WORKS

Lossless WebP predicts each pixel from its neighbours, encodes the differences with entropy coding, and can reuse colour palettes and transforms; it typically beats PNG by 25 to 45%. Lossy WebP is a video-codec keyframe (VP8) with chroma subsampling and quantisation. To prove a lossless conversion, decode both images to raw RGBA buffers and compare byte by byte. One subtlety: for fully transparent pixels, libwebp by default normalises the hidden RGB values (controlled by its `exact` flag), so buffers can differ in pixels with alpha 0 that contribute nothing to the composited image.

IN THIS PROJECT

Three PNG project images became lossless WebP: 142,688 → 76,536, 135,065 → 61,628 and 93,789 → 42,932 bytes. The decode comparison found zero differences in visible pixels and tens of thousands of differences only in alpha-0 pixels, which I reported honestly rather than claiming bit identity. I did not resize: 512 px images shown at 250 CSS px are exactly right for 2× screens. Lossy WebP at quality 80 would have been about a quarter of the size again but not pixel-identical, so it is a follow-up for the redesign.

FOLLOW-UPS

- "Why not AVIF?" Better lossy compression, weak lossless mode, slow encoding; wrong tool for a parity requirement.
- "How do you serve the right size?" `srcset` with width descriptors and `sizes`, or `<picture>` for format fallbacks.

How to say it: "I used lossless WebP and proved it by decoding both files and comparing pixels; the only differences were under fully transparent pixels. Roughly half the bytes, zero visible change."

How does native lazy loading of images work, and when is it the wrong choice?

SIMPLE EXPLANATION

`loading="lazy"` tells the browser not to download an image until the user scrolls near it. It saves bandwidth for images the user may never see, but it delays images the user needs immediately.

HOW IT WORKS

The browser computes a distance threshold from the viewport (Chrome uses a few hundred to a few thousand pixels depending on connection type) and starts the fetch when the image enters that margin. The image must have a reserved size (width/height attributes or CSS) or the page shifts when it loads. `decoding="async"` allows the browser to decode the image off the critical path so inserting it does not block the next frame. Never lazy-load the LCP image or anything in the first viewport: the browser would otherwise have fetched it at high priority immediately, and lazy loading delays it until layout is known, which measurably worsens LCP.

IN THIS PROJECT

Only the project card images below the fold are lazy, with `decoding="async"`. Their containers have a fixed 250 px height with overflow hidden, so even if they load late nothing moves. The hero contains no image element (it is a Lottie SVG), and the work-experience logos are small and needed for the banner colour fallback, so they load normally.

FOLLOW-UPS

- "Does lazy loading work inside an opacity-0 reveal wrapper?" Yes, visibility is based on geometry, not opacity.
- "What about `fetchpriority`?" Use `fetchpriority="high"` on the LCP image and `low` on decorative ones to shape the download order.

How to say it: "Lazy loading is for below-the-fold images with reserved space; never for the LCP image. I applied it to the project cards, whose containers are fixed-height."

What is the DOM size cost, and why did the element count drop from 871 to 592 on mobile?

SIMPLE EXPLANATION

Every element in the page costs memory, style calculation time and layout time. A huge DOM makes every style change and every layout slower, and makes selectors slower to match.

HOW IT WORKS

Style recalculation visits elements whose computed style may have changed; layout visits boxes whose geometry may have changed; both scale with tree size and depth. Lighthouse warns above roughly 800 elements and flags depth and child counts. Lottie SVGs are DOM-heavy: each shape layer becomes groups and paths, and the player updates their attributes every frame.

IN THIS PROJECT

The old page had 871 elements with four Lottie SVG trees always present, including two inside `display: none` containers that still existed in the DOM and still animated. The new page renders only the visible animations' SVGs, so on mobile (two visible) the DOM is 592 elements; on desktop at widths where all four are visible it is 831. Fewer nodes means less work per frame while the visible animations run.

FOLLOW-UPS

- "Does `display: none` content cost anything?" It is excluded from layout and paint but still occupies memory and is still mutated by scripts, which is why the hidden Lotties were pure waste.

How to say it: "DOM size scales style and layout cost. Not rendering hidden animations cut the mobile DOM by a third."

Lab data versus field data: what is the difference, and which did you use?

SIMPLE EXPLANATION

Lab data is measured by a tool in a controlled environment: reproducible, detailed, good for debugging, but not what any particular user experienced. Field data is collected from real visitors' browsers: representative and noisy, and it needs traffic to be meaningful.

HOW IT WORKS

Lighthouse loads the page in a fresh profile with simulated or applied throttling and reports metrics plus audits. The Chrome User Experience Report (CrUX) aggregates anonymised Core Web Vitals from opted-in Chrome users for sites with enough traffic; PageSpeed Insights shows both. Real-user monitoring (RUM) is your own collection, typically with the `web-vitals` library sending beacons to analytics or a database, letting you segment by device, network, country and build version.

IN THIS PROJECT

All numbers are lab. The live site baseline came from Lighthouse against GitHub Pages; the before/after comparison came from Lighthouse against both builds served by the same local server so the only variable was the code. CrUX had no data (too little traffic), and I did not add RUM, which is my stated next step so the lab improvement can be confirmed in the field.

FOLLOW-UPS

- "Why not compare live-before with local-after?" That would mix network effects into the code comparison; GitHub Pages added about 0.8 s of TTFB on mobile, which is a hosting fact, not a code fact.

How to say it: "Lab is reproducible, field is true. I have lab evidence under identical conditions; adding web-vitals RUM is how I would close the gap."

How does Lighthouse simulate a slow phone, and how is the performance score computed?

SIMPLE EXPLANATION

Lighthouse records a load on your fast machine and then estimates how long it would have taken on a mid-range phone on a slow connection using a model. Each metric is then converted to a 0–100 score against real-world distributions and the scores are weighted together.

HOW IT WORKS

The default mobile preset emulates a Moto G Power class device: simulated slow 4G (about 150 ms round-trip time, 1.6 Mbps down, 750 Kbps up) and a 4× CPU slowdown. "Simulated" throttling (Lantern) replays the recorded network and CPU activity through a dependency graph model; "applied" throttling actually slows the browser. The desktop preset uses 40 ms RTT, 10 Mbps and no CPU slowdown. Each metric value is mapped through a log-normal curve calibrated on HTTP Archive data so that, for example,

an LCP of 2.5 s scores about 90 and 4 s about 50. Weights in Lighthouse 10–12: TBT 30%, LCP 25%, CLS 25%, FCP 10%, Speed Index 10%. The score has run-to-run variance of a few points, so compare medians of several runs for small differences.

IN THIS PROJECT

Mobile scores went 63 → 98–99 locally and the live baseline was 53. The 98 versus 99 across runs is normal variance. Desktop went 97 → 100. I reported ranges rather than a single lucky run.

FOLLOW-UPS

- "Why did the live mobile score differ from local?" Network latency to GitHub Pages (TTFB 0.8 s) and real variance; the code is identical.
- "Is 100 meaningful?" It means every metric is in the good range on the modelled device; it says nothing about a real user on a 2G network.

How to say it: "Lighthouse models a mid-range phone on slow 4G and weights TBT, LCP and CLS most. I compared both builds under the same model and reported ranges."

What is the difference between display: none, visibility: hidden and opacity: 0, and why did it matter here?

SIMPLE EXPLANATION

All three make something invisible, but they differ in whether the element takes space, whether it can be clicked, whether it is read by screen readers, and whether observers see it.

HOW IT WORKS

- `display: none` : no box at all; takes no space; not in the render tree; not focusable; hidden from assistive technology; IntersectionObserver never reports it intersecting; children cannot override it.
- `visibility: hidden` : box exists and takes space; not painted; not clickable or focusable; hidden from assistive technology; children can set `visibility: visible` .
- `opacity: 0` : box exists, takes space, is painted (transparently), still receives clicks, still exposed to assistive technology, animatable on the compositor; Chrome does not treat an opacity-0 element as an LCP candidate until it becomes visible.

IN THIS PROJECT

Two Lottie boxes are `display: none` on common screen sizes, which is exactly why the new lazy loader never loads them: there is no box to intersect. react-reveal used `opacity: 0` on unrevealed sections so they keep their space (no layout shift when they appear) and can animate on the compositor; my replacement does the same. The hero's opacity-0 start on the old site also delayed LCP until JavaScript ran the fade.

FOLLOW-UPS

- "Is opacity 0 content accessible?" Yes, screen readers still read it, which is why purely visual hiding should pair with `aria-hidden` if the content is not meant to be read yet.

How to say it: "display none removes the box, visibility hides it but keeps space, opacity paints it transparent and keeps it interactive. Hidden boxes never intersect, which is why hidden animations never load now."

How do IntersectionObserver thresholds and rootMargin work, and what is the gotcha with the first callback?

SIMPLE EXPLANATION

An IntersectionObserver tells you when an element crosses into or out of view, without you listening to scroll events and measuring geometry yourself. Thresholds say at what visible fraction you want to be told; rootMargin grows or shrinks the area that counts as "in view".

HOW IT WORKS

The browser computes intersections off the main thread's critical path and delivers callbacks asynchronously with entries containing `isIntersecting`, `intersectionRatio` and bounding rectangles. With `threshold: [0.2]` you get a callback when the visible fraction crosses 20% in either direction. With `rootMargin: "300px 0px"` the viewport is treated as 300 px taller on both ends, so an element 250 px below the fold counts as intersecting. Two gotchas: first, a callback fires once immediately after `observe()` with the current state, even if no threshold was crossed; second, an element that goes from 0% to 15% visible never triggers a 0.2 threshold, so a tall element on a short viewport may never reach the fraction.

IN THIS PROJECT

Reveal animations use threshold 0.2 and reveal when the callback reports a ratio greater than zero, exactly as react-reveal did, which means an element already 5% visible at observe time reveals immediately (initial callback) while one scrolled slowly to 15% does not (no crossing). My functional test confirmed this quirk is pre-existing: jumping past a section with `scrollIntoView` leaves it unrevealed in both builds. Lazy Lottie uses `rootMargin: "300px 0px"` so the 54 KB player and the JSON usually arrive before the box scrolls in, and a second observer with threshold 0 pauses and resumes playback.

FOLLOW-UPS

- "Why not scroll listeners?" They run on every scroll event on the main thread and require layout reads; observers are batched and asynchronous.
- "How did you choose 300 px?" A compromise: early enough to usually finish loading before visibility, late enough not to download animations for sections the user never reaches.

How to say it: "Thresholds fire on crossings plus one initial callback; rootMargin expands the viewport. I matched react-reveal's exact semantics and used a 300 px margin to prefetch animations just in time."

Which CSS properties are cheap to animate and why?

SIMPLE EXPLANATION

Animating position or size forces the browser to re-measure and redraw every frame on the main thread. Animating transform or opacity can be done by the compositor on its own thread and often on the GPU, so it stays smooth even when JavaScript is busy.

HOW IT WORKS

Changing `width`, `height`, `top`, `left`, `margin` or font size triggers layout, then paint, then composite. Changing `color` or `box-shadow` skips layout but repaints. Changing `transform` or `opacity` on an element that has its own compositor layer only recomposites: the layer's existing pixels are moved or

blended. The browser promotes elements to layers when they are animated with these properties; `will-change: transform` promotes early but costs memory, so apply it just before an animation and remove it after. Filters like `blur` are expensive even on the compositor.

IN THIS PROJECT

The reveal animations use `translate3d` plus opacity, inherited from react-reveal and kept. Lighthouse flagged the contact form's `transition: 150ms ease` (all properties, including padding and left) as six non-composited animations; it is a parity-preserved pattern I listed to narrow to transform and colour in the redesign. The experience cards use a 5 px blur layer, which is why hovering them is heavier than it looks.

FOLLOW-UPS

- "What is the frame budget?" About 16.7 ms at 60 Hz, 8.3 ms at 120 Hz; the browser needs a few of those milliseconds itself.
- "How do you see which animations are non-composited?" The Performance panel's Animations track and Lighthouse's audit.

How to say it: "Transform and opacity are compositor-only; everything else triggers layout or paint. The reveal uses translate and opacity; the form's transition-all is on my follow-up list."

What is the CSS cascade, and how did a "duplicate" stylesheet change the mobile layout?

SIMPLE EXPLANATION

When several rules apply to the same element and property, the cascade decides the winner: first by origin and importance, then by specificity, then by source order. Media queries do not add specificity; they only decide whether a rule applies at all.

HOW IT WORKS

Order of resolution: user-agent styles, user styles, author styles; `!important` flips the author/user order; then specificity (inline, ids, classes/attributes/pseudo-classes, elements); then the later rule wins. A rule inside `@media (max-width: 768px)` has the same specificity as the same selector outside it. So if stylesheet A has `.x { display: flex }` and `@media (max-width: 768px) { .x { display: block } }`, and stylesheet B, loaded later, has `.x { display: flex }`, then on phones the winner is B's unconditional flex, because it comes last among equal-specificity rules. Cascade layers (`@layer`) are the modern tool to make order explicit.

IN THIS PROJECT

StarWars.css duplicated several Greeting.css rules, including `.greeting-main { display: flex }`, and was imported after Greeting.css. Greeting.css's phone media query set `display: block`, but StarWars.css's later unconditional rule overrode it, which is why the hero stays side-by-side on phones on the live site. Deleting the "duplicate" file switched phones to the stacked layout: the tablet page grew by 185 px and 18.5% of pixels changed. The pixel diff caught it; I restored the file (minus its Google Fonts import) with a comment explaining the dependency.

FOLLOW-UPS

- "How would you prevent this class of bug?" Cascade layers, CSS modules or a linter for duplicate selectors, and a visual regression test on every change.
- "Did restoring it cost performance?" About 1 KB gzipped of CSS; parity won.

How to say it: "Media queries don't add specificity, so a later unconditional rule beats an earlier media-query rule. A file that looked duplicate was holding the phone layout; the screenshot diff caught it when I deleted it."

Why does an inline SVG make its container taller than the SVG itself?

SIMPLE EXPLANATION

An `<svg>` is an inline element by default, like an `<img>` or a letter. Inline things sit on the text baseline, and a line of text reserves room below the baseline for letters like "g" and "y". So an inline SVG gets a few pixels of empty space under it.

HOW IT WORKS

In an inline formatting context the line box height comes from the font metrics of the parent (its `line-height` and the strut). A replaced inline element's bottom edge sits on the baseline; the descender space below it belongs to the line box, so the containing block is taller than the element by the descent of the current font at the current size. Fixes when you want it gone: `display: block`, `vertical-align: middle` or `bottom`, or `line-height: 0` on the container.

IN THIS PROJECT

Measured on the old build: Lottie wrapper 609 px for a 605 px SVG, 584 for 580, 859 for 854. The 4–5 px gap is the descender at the 19/16 px body font. The lazy placeholder had to reproduce it or the page would shift when the player mounted, so the placeholder is itself an empty inline SVG with the same width and height styles. The measured boxes matched the baseline exactly at every viewport.

How to say it: "Inline SVGs sit on the baseline, so the line box adds descender space below them. I reproduced that gap with an inline SVG placeholder instead of guessing with aspect-ratio."

10.3 Foundations: React and the build toolchain

How does React turn components into DOM updates? Explain reconciliation in plain terms.

SIMPLE EXPLANATION

You describe what the screen should look like for a given state; React works out the smallest set of DOM changes to get there. It keeps a lightweight description of the UI in memory (the element tree, often called the virtual DOM), compares the new description with the previous one, and only touches the real DOM where something differs.

HOW IT WORKS

Rendering a component returns React elements (plain objects: type, props, children). React's reconciler walks the new tree against the previous fiber tree. If an element's type changed (`div` to `span` , or a different component), React throws the old subtree away and mounts a new one. If the type is the same, it keeps the DOM node and updates only changed attributes, then recurses into children. Children lists are matched by index unless they have `key` s, in which case they are matched by key, which is what lets React reorder, insert and remove list items without rebuilding them. React 18's concurrent renderer can pause and resume this work, but the commit phase, where DOM mutations happen, is always synchronous. Real DOM operations are the expensive part; reconciliation exists to minimise them.

IN THIS PROJECT

Most components are static, so reconciliation cost is trivial. Where it mattered: the Reveal component changes only an inline `style` and a `className` on an existing element when it reveals, so React patches two attributes and nothing else re-renders. Theme changes flip `isDark` at the root, which re-renders the whole tree once but only changes class names in the DOM.

FOLLOW-UPS

- "Is the virtual DOM fast?" It is a convenience, not a speed trick; a hand-written DOM update is faster. Its value is that you describe state and React handles the diff correctly.
- "What triggers a re-render?" State changes, parent re-renders, context value changes.

How to say it: "React diffs the new element tree against the previous one and applies minimal DOM mutations; type decides replace versus update, keys decide list matching."

Why do keys matter in lists, and why did you change the carousel keys to include the start index?

SIMPLE EXPLANATION

A key tells React which item is which between renders. With the wrong key, React may keep a component instance alive and hand it different data, so internal state or effects from one item leak into another.

HOW IT WORKS

When a list re-renders, React matches old and new children by key. Index keys (`key={i}`) say "the item in slot 0 is still the item in slot 0" even if a different item moved into that slot. The component instance in slot 0 keeps its state (refs, hooks state, uncontrolled inputs) and just receives new props. That is fine for stateless cards, and wrong for anything with per-item state.

IN THIS PROJECT

The experience and project carousels show three of five or six items and page by changing a start index, rendering `items.slice(start, start + 3)` with `key={i}`. The ExperienceCard has internal state (a computed banner colour and an image ref). With index keys, paging right would hand the Cars24 card instance the Cerebry data; since banner colours are now static props this would still display correctly, but the lazy ColorThief fallback could run on the wrong image. Keys of `start + i` make each visible slot map to a specific data item, so instances are replaced rather than reused when paging. The DOM output is identical.

FOLLOW-UPS

- "Why not use the company name as the key?" Also fine; any stable identity of the item works. Avoid `Math.random()` keys, which force a remount on every render.

How to say it: "Index keys reuse instances across different data. I keyed carousel cards by their data position so paging replaces instances instead of mutating them."

Explain render, useEffect and useLayoutEffect: when does each run and what is each for?

SIMPLE EXPLANATION

Render is the pure function that computes what the UI should be; it must not touch the DOM or start side effects. Effects are where you do things to the outside world: subscribe, measure, fetch, start a timer. The difference between the two effect hooks is whether they run before or after the browser paints.

HOW IT WORKS

During a render React calls your component and collects the result. During commit it applies DOM mutations. `useLayoutEffect` callbacks run synchronously right after mutations and before the browser paints, so you can measure layout and make a correction the user never sees; blocking here delays the frame. `useEffect` callbacks run after paint, asynchronously, so they do not delay the frame; measuring and then changing the DOM in a `useEffect` can show a one-frame flicker. Both run their cleanup before the next run and on unmount. On the server neither runs; older React versions warn that `useLayoutEffect` does nothing during server rendering, which is why libraries use an isomorphic alias that picks `useEffect` when `window` is undefined.

IN THIS PROJECT

DisplayLottie uses `useEffect` to set up the proximity observer (no visual impact) and a layout effect to call `loadAnimation` with in-memory data the moment React has removed the placeholder, so the real SVG exists before the next paint and no frame shows an empty box. Reveal uses `useEffect` for the in-viewport test and observer because its result only changes styles that start an animation; one frame of delay is invisible. Top uses `useEffect` to add a passive scroll listener and remove it on unmount.

FOLLOW-UPS

- "Why are effects run twice in development?" StrictMode mounts, unmounts and remounts to surface missing cleanup; production runs them once.
- "When would a layout effect be wrong?" For anything slow or network-bound, because it blocks painting.

How to say it: "Render is pure; useLayoutEffect runs before paint for measure-and-correct; useEffect runs after paint for everything else. I used a layout effect to swap the Lottie placeholder without a visible empty frame."

How does a wrapper component like Reveal add behaviour to a child without adding a DOM element?

SIMPLE EXPLANATION

Instead of rendering its own `<div>` around the child, the wrapper takes the child element and returns a copy of it with extra props: a class, a style and a ref. The DOM stays exactly as the child defined it, which is essential for CSS that targets direct children or flex layouts.

HOW IT WORKS

`React.Children.only(children)` asserts there is exactly one child element. `React.cloneElement(child, newProps)` returns a new element of the same type with merged props. To attach a ref without breaking a ref the child already had, you pass a callback ref that stores the node in your own ref and also forwards it to the child's original ref (function or object). Class names are concatenated; styles are merged with the wrapper's inline style taking precedence for the properties it sets. react-reveal did exactly this, which is why the live DOM shows `class="react-reveal main contact-margin-top"` on the section element itself rather than on an extra wrapper.

IN THIS PROJECT

Reveal clones the child with `className: "react-reveal " + child.props.className`, a merged `style` (`opacity: 0` before reveal; the animation properties after), and a merging ref. Layout parity depends on this: several revealed sections are flex children of their parents, and an extra wrapper div would have changed `flex: 1` distribution and heights.

FOLLOW-UPS

- "What are the alternatives?" A render-prop or hook (`useReveal(ref)`) that the child applies itself; more explicit, more code per call site.
- "Is cloneElement discouraged?" The React team prefers explicit props, but it remains supported and is the right tool for a drop-in replacement that must not change markup.

How to say it: "Reveal clones its single child with an extra class, style and merged ref, so the DOM is identical to react-reveal's output and no wrapper div disturbs the flex layouts."

How does React Context work here, and what are its re-render implications?

SIMPLE EXPLANATION

Context lets a value defined high in the tree be read anywhere below without passing it through every component. When the value changes, every component that reads it re-renders.

HOW IT WORKS

`createContext` makes a Provider/Consumer pair. `useContext(Ctx)` subscribes the component to the nearest Provider's value. React compares the new value with `Object.is`; a new object literal every render counts as changed, so providers often memoise the value. Consumers re-render even if their parents are memoised, because context bypasses props.

IN THIS PROJECT

StyleContext carries `{ isDark, changeTheme }`. The theme changes at most once per visit (after mount, from storage), and when it does the whole tree re-renders anyway because every section branches on `isDark` for class names. So memoising the value would buy nothing here. In the redesign, moving theming to CSS custom properties on the root would remove the need for any component to read the theme, eliminating this re-render and the hydration hazard at once.

How to say it: "Context is a subscription to a value; consumers re-render on change. The theme context changes once per visit, so it is not a hot path, but the redesign should move theming into CSS."

Controlled versus uncontrolled inputs: how is the contact form built and what are the trade-offs?

SIMPLE EXPLANATION

A controlled input's value lives in React state: every keystroke updates state and React writes the value back into the input. An uncontrolled input keeps its own value in the DOM and you read it when you need it. Controlled gives you validation and formatting on every keystroke; uncontrolled is simpler and cheaper.

HOW IT WORKS

Controlled: `value={state}` plus `onChange`; React re-renders on each change. The floating-label effect here depends on knowing whether the field is focused or non-empty, which is state, so controlled is natural. Uncontrolled: `defaultValue` and a ref, or `FormData` on submit. Submission: the browser's native submit navigates to the form's action (the current URL if none) with the fields as query parameters; a React handler must call `preventDefault()` to stop that.

IN THIS PROJECT

The form is a controlled class component (kept as a class, which React 18 fully supports) with name, email and message state, a focus flag per field for the floating labels, and a status for the button text. Email validation uses the email-validator package; on success EmailJS sends with the service, template and public ids from `VITE_` variables; on invalid input the button shows "Invalid Email!". The original only prevented the default on the valid path, so an invalid email reloaded the page; the fix was to prevent it unconditionally, which the functional test then verified.

FOLLOW-UPS

- "Why not convert the class to hooks?" No behavioural benefit and extra parity risk; classes are not deprecated.
- "How would you stop spam?" EmailJS origin allowlist and reCAPTCHA, or a serverless endpoint with rate limiting (see system design).

How to say it: "Controlled inputs drive the floating labels and validation; the bug was a conditional preventDefault that let invalid submissions reload the page."

What changed between React 16 and React 18 that affected this migration?

SIMPLE EXPLANATION

React 18 introduced a new root API that unlocks concurrent rendering, batches state updates everywhere, and ships better server-rendering tools. Most code runs unchanged, but the entry point and a few patterns need attention.

HOW IT WORKS

- **Root API.** `ReactDOM.render` becomes `createRoot(container).render()`; `ReactDOM.hydrate` becomes `hydrateRoot(container, element, options)` with an `onRecoverableError` option.
- **Automatic batching.** Multiple `setState` calls inside promises, timeouts and native handlers now cause one render instead of several.
- **Concurrent features.** `startTransition`, `useDeferredValue`, `useTransition`; Suspense for data fetching; streaming SSR with `renderToPipeableStream` and selective hydration.
- **New hooks.** `useId` (stable ids across server and client), `useSyncExternalStore` (subscribe to external stores without tearing), `useInsertionEffect` (CSS-in-JS).
- **StrictMode** double-invokes effects in development to find missing cleanup.
- **Legacy lifecycles** (`componentWillReceiveProps`) still work but warn; react-reveal used them, one more reason to replace it.

IN THIS PROJECT

The entry became `hydrateRoot` in production and `createRoot` on the dev server (detected by whether the root already has children). `onRecoverableError` logs hydration problems in every environment so the test harness can assert a clean console. react-headroom moved from 3.0.0 to 3.2.1 for React 18 peer support with the same markup. Nothing else needed to change because the components used only stable APIs.

How to say it: "New root API, automatic batching, streaming SSR and hydration error hooks. The migration touched the entry point and one dependency version; the components were already compatible."

Explain hydration in depth: what React actually does, what it requires, and what happens when it fails.

SIMPLE EXPLANATION

The HTML already contains the rendered page. Hydration is React reading that DOM, building its internal tree to match, and attaching event handlers, so the page becomes interactive without being rebuilt. It only works if React's first render would have produced the same HTML.

HOW IT WORKS

`hydrateRoot` renders the component tree in "hydration mode": instead of creating DOM nodes, React walks the existing child nodes of the container in document order and tries to claim each one for the corresponding fiber. For each element it checks the tag name and, in development, warns about attribute and text differences; in production it compares text content and structure. Event handling in React 18 is delegated at the root container, so attaching handlers is mostly bookkeeping. When a mismatch is found, React 18 reports a recoverable error (`onRecoverableError`, visible as "Hydration failed…" or minified error codes 418/423/425) and falls back to client rendering from the nearest Suspense boundary, or the whole root if there is none: it discards the server DOM for that subtree and creates it afresh. The user sees a flash and pays the render cost twice. Attribute differences are not patched in production; the server's attribute stays.

IN THIS PROJECT

Every source of potential drift was neutralised: the theme renders light on both sides and applies the stored value in an effect; Lottie boxes render identical placeholders; reveal elements render `opacity: 0` except the hero which renders the animating state on both sides; banner colours come from static data; browser-only reads were moved into effects; `class` became `className` and `fill-rule` became `fillRule` so React does not warn about unknown attributes. The functional test confirms React attached to the pre-rendered root and the console stayed empty.

FOLLOW-UPS

- "What about text that must differ, like a timestamp?" `suppressHydrationWarning` on that element, and update it in an effect.
- "How is hydration different from `createRoot().render()` on an empty container?" Rendering into an empty container builds all DOM from scratch; hydration adopts existing DOM and is faster when the markup matches.

How to say it: "React walks the existing DOM and claims nodes for its fibers; a mismatch means a recoverable error and a client re-render of that subtree. I made the first render deterministic on both sides and asserted a clean console."

How do React.lazy and Suspense work, and where did you use dynamic imports?

SIMPLE EXPLANATION

`React.lazy` wraps a dynamic `import()` so a component's code is downloaded only when it is first rendered. `Suspense` shows a fallback while that download is in flight.

HOW IT WORKS

A dynamic `import()` returns a promise; the bundler turns it into a separate chunk and a loader. `React.lazy(() => import('./X'))` returns a component that, when rendered the first time, throws the pending promise; the nearest `Suspense` catches it, renders the fallback, and re-renders when the promise resolves. Code for the chunk is then cached. For non-component code (a library, data) you call `import()` yourself inside an effect or event handler and store the result in state.

IN THIS PROJECT

Component-level: the GitHub profile and repo cards are `React.lazy` with a `Loading` fallback (unchanged). Library-level: the Lottie player (`import("lottie-web/build/player/lottie_svg")` , 54 KB gzipped) and ColorThief (4 KB) are imported inside effects only when needed, and the animation JSON is fetched as data in parallel with the player import so both arrive together. Vite emitted each as its own chunk and generated the loader code.

FOLLOW-UPS

- "What is the downside of lazy loading?" A delay when the code is first needed, and an extra request; mitigate by prefetching on hover or proximity, which is what the 300 px root margin does.
- "Can Suspense fetch data?" In React 18 only through frameworks or libraries that integrate with it; `use()` arrives in React 19.

How to say it: "lazy plus Suspense defers component code; plain dynamic import defers libraries. The Lottie player and its JSON load only when the box is near the viewport."

What does a bundler do, and why did you move from create-react-app to Vite?

SIMPLE EXPLANATION

A bundler takes your source files (JSX, CSS, images, modules) and produces the optimised files a browser can load: transpiled JavaScript, hashed assets, split chunks. create-react-app wrapped webpack 4 with a fixed configuration; Vite is a newer tool with a near-instant dev server and a Rollup-based production build.

HOW IT WORKS

In development Vite serves each source file as a native ES module and transforms it on request with esbuild, pre-bundling dependencies once; the browser does the module graph work, so start-up is sub-second and hot updates are per module. In production Vite runs Rollup: resolves the module graph, tree-shakes, splits dynamic imports into chunks, minifies with esbuild, hashes filenames, rewrites asset URLs in HTML and CSS, emits `modulepreload` hints, and (with the SSR flag) can build a Node bundle of the same code. webpack 4 in CRA 3 did similar things but with a slower pipeline, an outdated Babel target, no first-class SSR mode, and a configuration you could not change without ejecting.

IN THIS PROJECT

The move removed the OpenSSL legacy flag, the IE 11 transforms and babel-runtime, enabled the pre-render step via `vite build --ssr`, and brought a configurable asset inline limit and chunking. Build time is about two seconds. The install dropped from 1,648 packages to 101 largely because react-scripts pulled in jest, eslint, workbox and dozens of webpack loaders.

FOLLOW-UPS

- "Why not webpack 5 or Rspack?" Both fine; Vite's SSR build and minimal config suited a small site. Rspack/Turbopack matter at monorepo scale.
- "What is esbuild versus Rollup?" esbuild (Go) is extremely fast at transforming and minifying; Rollup does the careful tree-shaking and chunking. Vite uses both; Rolldown is merging them.

How to say it: "Vite serves native modules in dev and uses Rollup for production; it gave me a modern target, an SSR build for pre-rendering, and a 16× smaller install."

ES modules versus CommonJS, and what interop problem did you hit?

SIMPLE EXPLANATION

CommonJS (`require` / `module.exports`) is Node's original module system, dynamic and synchronous. ES modules (`import` / `export`) are the standard, static (analysable before running), and what browsers understand. Mixing them is where surprises live.

HOW IT WORKS

ESM imports are resolved and linked before execution, enabling tree shaking and `modulepreload`. CJS exports are an object built at runtime. When Node imports a CJS module from ESM, the default import is `module.exports` as a whole; Node uses a lexer to guess named exports but does not honour Babel's `__esModule` flag, so a package compiled with `exports.default = X` gives you `{ default: X, __esModule: true }` as the default import. Bundlers handle this interop; Node does not.

IN THIS PROJECT

The SSR build externalised dependencies by default, so Node loaded react-headroom (Babel-compiled CJS). `import Headroom from 'react-headroom'` resolved to the exports object, and React threw "Element type is invalid: expected a string or class/function but got: object". The fix was `ssr.noExternal: ['react-headroom', 'emailjs-com', 'email-validator']` so Vite bundles them into the SSR output and applies interop. Making the package `"type": "module"` let the SSR output and the scripts run as ESM in Node.

How to say it: "Node exposes CommonJS module.exports as the default import and ignores Babel's __esModule flag, so a compiled default export arrives as an object. I bundled those packages into the SSR build instead of externalising them."

What is tree shaking, what defeats it, and why did an unused CSS import stay in the bundle?

SIMPLE EXPLANATION

Tree shaking drops exports nobody imports. It relies on the bundler being able to see, without running the code, which exports are used and which modules have no side effects.

HOW IT WORKS

Rollup builds the ESM graph, marks used bindings, and removes unused ones, then removes modules whose remaining code has no side effects. Things that defeat it: CommonJS (dynamic exports), top-level code with side effects (a module that registers something on import), re-export barrels that import everything, and `/*#__PURE__*/` annotations missing on helper calls. A `sideEffects: false` field in package.json tells the bundler modules are safe to drop. CSS imports are side effects by definition (importing injects styles), so a module that imports a CSS file is kept even if none of its exports are used.

IN THIS PROJECT

The `gsap` import was dropped by webpack's tree shaking already, which is why it contributed zero bytes to the old bundle despite being installed. The ToggleSwitch component is not rendered, but Header imports its stylesheet to keep the global `input { transform: scale(0.5) }` rule for parity; the import is a side effect, so Rollup keeps the CSS. I made that import explicit (importing the CSS file directly with a comment) rather than relying on an unused component import.

How to say it: "Tree shaking needs static ESM and side-effect-free modules. CSS imports are side effects, which is how a stylesheet for an unrendered component intentionally stays in the build."

What are source maps, why should they not be deployed to production, and how did you use them in the audit?

SIMPLE EXPLANATION

A source map links positions in the minified output back to your original files so stack traces and DevTools show real code. They are useful to you and useless to users; deploying them publishes your source and adds weight to the deploy.

HOW IT WORKS

A map file holds `sources`, optional `sourcesContent` and `mappings`: base64 VLQ-encoded segments that say "generated line L column C came from source S line L' column C'". Browsers fetch maps only when DevTools is open, via a `//# sourceMappingURL` comment. For error monitoring you upload maps to

the monitoring service at build time instead of serving them publicly.

IN THIS PROJECT

The old deploy shipped 2.5 MB of maps to GitHub Pages. I used those very maps for the audit: decoding the mappings, sorting segments by generated position, and measuring the span to the next segment gives the number of minified bytes each source file produced; grouping by `node_modules/<package>` showed lottie-web at 261 KB (41% of the vendor chunk), react-dom at 117 KB, Apollo plus graphql at about 127 KB, react-reveal at 35 KB. That is far more accurate than looking at package sizes on npm, because it reflects what survived tree shaking and minification. The new build sets `sourcemap: false`.

How to say it: "Maps are for debugging, not users; I turned them off in production, but I used the old ones to attribute every minified byte to a package before deciding what to cut."

How do environment variables work in Vite versus CRA, and what is safe to put in them?

SIMPLE EXPLANATION

Build tools replace references like `import.meta.env.VITE_X` with the literal value at build time, so the value ends up inside the JavaScript bundle. Anything you put there is public. The prefix requirement exists to stop you leaking server secrets by accident.

HOW IT WORKS

CRA exposed `process.env.REACT_APP_*`; Vite exposes `import.meta.env.VITE_*` plus built-ins like `BASE_URL`, `MODE`, `DEV`, `PROD`. Files `.env`, `.env.local`, `.env.production` are loaded in order; only prefixed keys are inlined. There is no `process` object in the browser; the old code's `require('dotenv').config()` pulled a Node library and polyfills into the bundle and silently failed.

IN THIS PROJECT

The EmailJS service id, template id and public key moved from `REACT_APP_` to `VITE_` names; they are designed to be public (they were in the old bundle too) and are protected on the EmailJS side by origin allowlists. The GitHub token for the disabled section is `VITE_GITHUB_TOKEN` and should be a fine-grained, read-only token if ever used, because it will be public in the bundle. The Header's home link uses `import.meta.env.BASE_URL` so the sub-path is not hard-coded.

How to say it: "VITE_-prefixed values are inlined into the bundle, so they are public by definition; EmailJS ids are built for that, a GitHub token is not, which is why that section stays off."

What is browserslist, what did the old target cost, and what does the new build target?

SIMPLE EXPLANATION

Browserslist is a query like ">0.2%, not dead" that tools resolve against a browser-usage database to decide how much to transpile and polyfill. If the database is years old, the query still includes browsers that are gone, and you ship code for them.

HOW IT WORKS

Babel's preset-env looks at the resolved browser list and transforms syntax those browsers lack: classes, arrow functions, async/await (which needs the regenerator runtime), optional chaining. Each transform adds code and often a helper from babel-runtime. The resolved list is only as current as the installed `caniuse-lite` package. Modern tools instead target a fixed ES year or a "baseline" set.

IN THIS PROJECT

The installed caniuse database was from 2020, so `>0.2%` still resolved to IE 11, Chrome 49 and Safari 13. Every class in React and the app was down-compiled and 17 KB of babel-runtime helpers shipped; Lighthouse flagged legacy JavaScript. Vite 6's default target (`modules` : ES2020, Chrome 87, Safari 14, Firefox 78, Edge 88) emits native classes, async functions and optional chaining, which is why the app chunk is 23 KB gzipped including all component code.

How to say it: "An outdated browserslist database made the old build target IE 11. The new build targets ES2020-era browsers, which removed the transforms and helpers."

10.4 Intermediate: rendering strategy, animation, fonts, images, CSS

Compare CSR, SSR, SSG and ISR in depth. Which fits a portfolio and why?

SIMPLE EXPLANATION

The question is where and when HTML gets produced. In the browser on every visit (CSR), on a server for every request (SSR), once at build time (SSG), or at build time with periodic refreshes (ISR).

HOW IT WORKS

Strategy	First paint	Freshness	Infrastructure	Typical cost
CSR	After JS downloads and runs	Live (fetched by the client)	Any static host	Slow first paint, poor SEO without extra work, cheap hosting
SSR	After server renders (TTFB includes render)	Per request	Node server or edge runtime	Fast paint if server is fast, server cost, caching complexity, hydration still needed
SSG	As soon as HTML arrives	At build time	Any static host plus a build step	Fastest and cheapest, needs rebuild on content change
ISR	Like SSG	Regenerated per page on a timer or on demand	A host that supports it (or your own cache layer)	Fresh enough for most content without full rebuilds

Hybrid approaches are common: SSG for most pages with islands of client interactivity, SSR for personalised pages, CSR for logged-in dashboards. All server-produced HTML needs hydration if it has interactive components, and all of them benefit from the same asset discipline (hashing, compression, lazy loading).

IN THIS PROJECT

Content is a single data file known at build time; there is one page; nothing is personalised. SSG is the obvious fit, implemented as a pre-render step: build the client, build a Node bundle of the same components, call `renderToString` once, inject the markup into the HTML, deploy static files. The visitor gets a complete page in the first response and React hydrates afterwards.

FOLLOW-UPS

- "When would SSG be wrong?" When content changes faster than you can rebuild, or depends on the user.
- "Does SSG need hydration?" Only if there is interactivity; a pure document would not even need React at runtime.

How to say it: "Where HTML is produced decides first paint and freshness. A portfolio is build-time content on one page, so I pre-render at build and hydrate."

Describe your pre-render pipeline step by step, including how the client knows whether to hydrate.

SIMPLE EXPLANATION

Three steps: build the browser bundle, build a Node version of the same app, run it once to produce HTML, and paste that HTML into the page before deploying.

HOW IT WORKS

1. `vite build` produces `build/index.html` with hashed script, style and preload tags, plus all assets.
2. `vite build --ssr src/entry-server.jsx --outDir .ssr-build` bundles the components for Node. The entry exports `render()` which returns `renderToString(<App/>)`. Asset imports resolve to the same hashed URLs as the client build because hashing is content-based and the base path is shared; CSS imports are stripped from the server bundle.
3. `scripts/prerender.mjs` imports the server bundle, calls `render()`, asserts the output is non-trivial, replaces `<div id="root"></div>` with the filled root, writes the HTML back, and deletes the temporary SSR directory.
4. `src/entry-client.jsx` checks `container.hasChildNodes()`: true in production (hydrate), false on the dev server (create a root and render). `hydrateRoot` gets an `onRecoverableError` that logs.

Pitfalls handled: packages that break under Node's ESM loader (`ssr.noExternal`), browser globals at module scope (moved into effects), and anything that would differ between server and client first render.

IN THIS PROJECT

The injected markup is about 20 KB (9.5 KB gzipped with the rest of the HTML). It contains the hero already in its animating state, static banner colours, and placeholders for the Lottie boxes. The whole page skeleton paints when CSS arrives, before any JavaScript.

FOLLOW-UPS

- "Why not react-snap or a headless-browser snapshot?" A snapshot captures post-animation DOM including Lottie SVGs that React did not render, guaranteeing hydration mismatches. Rendering with React itself produces exactly what hydration expects.
- "How do you keep server and client assets consistent?" Both builds run from the same source and config; the asset URLs are content hashes, so they agree.

How to say it: "Client build, SSR build, renderToString once, inject into index.html; the client hydrates if the root already has children. Thirty lines of script instead of a framework."

Why did you not adopt Next.js, Astro or Gatsby for the pre-rendering?

SIMPLE EXPLANATION

Frameworks are excellent when you need routing, data fetching conventions, image components and deployment integrations. This was one page with build-time content and a rule that nothing visible may change. The smallest tool that achieved the goal carried the least risk.

HOW IT WORKS

Adopting Next.js means its file-based routing, its `next/image` and `next/font` conventions, its own compiler and dev server, and a runtime that expects to own the page. Astro means rewriting components as Astro components or wrapping React islands, with its own hydration directives. Each migration changes markup in small ways (wrapper elements, attribute order, class injection) that a pixel-parity test would flag, and each adds a learning and maintenance surface for a one-page site. The custom pre-render is thirty lines using only React's public `renderToString` and Vite's SSR build, both standard.

IN THIS PROJECT

The result has the same first-paint advantage a framework would give. The components are now plain React 18 function components without framework-specific APIs, so when the redesign adds pages or a blog, moving to Astro (islands, zero JS by default) or Next (if server features are needed) is straightforward and can be judged on its own merits.

How to say it: "The constraint was zero visible change on a single page; a framework migration risks markup drift and adds surface area for no routing or data benefit. I kept the door open by leaving plain function components."

How exactly does your Reveal component reproduce react-reveal, including the viewport test and the keyframes?

SIMPLE EXPLANATION

I read react-reveal's source and copied its observable behaviour precisely: what it renders at first, when it decides to reveal, and what styles it applies. The component has the same prop names so call sites only changed an import path.

HOW IT WORKS

1. **Initial render.** Clone the single child, add class `react-reveal`, set inline `opacity: 0`. (With `appear`, render the revealed state immediately.)
2. **On mount.** Compute whether the element is in view the way react-reveal did: `s = pageYOffset − offsetTop(el)`, `o = min(height, innerHeight) × 0.2`, in view if `s > o − innerHeight && s < height − o`. If so, reveal now.
3. **Otherwise observe** with `IntersectionObserver({ threshold: 0.2 })` and reveal on the first callback whose `intersectionRatio > 0`, then disconnect.
4. **Reveal styles** (inline): `animation-fill-mode: both; animation-duration: Nms; animation-delay: 0ms; animation-iteration-count: 1; opacity: 1; animation-name: <name>`. No timing function, so the browser default `ease` applies, as before.
5. **Keyframes.** Fade: `from { opacity: 0; transform: translate3d(x, y, 0) } to { opacity: 1; transform: none }`; Slide: the same without opacity. Direction and distance map exactly as in the original (`bottom` with distance 40px gives `translate3d(0, 40px, 0)`; `left` without distance gives `translate3d(-100%, 0, 0)`). The six combinations used are static in CSS; others are inserted at runtime like the original did.

IN THIS PROJECT

Call sites: Greeting (Fade bottom 40px, 1000 ms, `appear`), Skills (Fade left and right), progress/experience/projects/contact (Fade bottom 20px), Footer (Fade bottom 5px), EducationCard (Fade left 1000 ms and Slide left 2000 ms). The final state is identical (`opacity: 1; transform: none`), which the screenshot diff confirmed, and the functional test confirmed the reveal timing quirks match.

FOLLOW-UPS

- "Why not CSS scroll-driven animations?" Browser support is still incomplete and they change the motion model; the goal was identical behaviour.
- "What did you drop?" react-reveal's cascade, collapse and out-animations, none of which the site used.

How to say it: "Same class, same initial opacity, same 20% viewport test, same observer rule, same inline animation properties and keyframes. Only the import path changed at call sites."

Why define the animation keyframes statically in CSS when react-reveal inserted them at runtime?

SIMPLE EXPLANATION

If keyframes only exist after JavaScript runs, the pre-rendered hero cannot animate before hydration. Putting the keyframes the page uses into the stylesheet lets the hero start fading in the moment CSS arrives.

HOW IT WORKS

A CSS animation starts when an element with an `animation-name` matching an existing `@keyframes` rule is styled. The pre-rendered hero carries its animation styles inline in the HTML. With the keyframes in the stylesheet, the browser starts the animation at first paint, before any script. The component still has a runtime fallback: if a prop combination has no static keyframes, it inserts a rule into a dedicated `<style>` element via `insertRule`, which is what react-reveal always did.

IN THIS PROJECT

Six keyframe sets are static (fade bottom 40/20/5 px, fade left/right 100%, slide left 100%). The hero fades in from first paint on the pre-rendered page; everything else waits for hydration and the observer, as before. Because Chrome does not count opacity-0 paints toward LCP, starting the fade early is also what lets LCP register early.

How to say it: "Static keyframes let the pre-rendered hero animate before JavaScript; a runtime insertRule fallback keeps the component general."

What is inside a Lottie JSON, what are expressions, and why did that decide which player build you could use?

SIMPLE EXPLANATION

A Lottie file is an After Effects animation exported as JSON: layers, shapes, and keyframes describing how properties change over time. Some animations also include small JavaScript programs, called expressions, that compute property values every frame. A player without an expression engine shows those animations wrong.

HOW IT WORKS

The JSON has global width, height, frame rate, in/out points, an `assets` array (images, precomposed layers) and a `layers` array. Each layer has a type (shape, solid, image, precomp, text), transform properties (anchor, position, scale, rotation, opacity) with either static values or keyframe arrays with easing, and for shape layers a tree of shape items (paths, fills, strokes, trim paths, repeaters). Any animatable property may carry an `x` field holding an expression string; effects (`ef`) can expose sliders and controls that expressions read. The player evaluates all of this per frame and writes the result into SVG attributes (SVG renderer), draws it to a canvas (canvas renderer), or positions DOM elements (HTML renderer). lottie-web ships several builds: full (all renderers plus expressions, 261 KB minified), `lottie_svg` (SVG plus expressions, 204 KB), `lottie_light` (SVG without expressions, 157 KB), and canvas variants.

IN THIS PROJECT

Inspecting the four files found 21 expressions in the coding animation, 3 in the engine, 1 plus three slider effects in the hero, none in the email animation. So `lottie_light` was ruled out; `lottie_svg` was the smallest correct build, 11 KB gzipped less than the full build the site had shipped. I pinned 5.7.3, the version already in use, so every frame is produced by identical code.

How to say it: "Lottie is layers, shapes and keyframes, sometimes with per-frame JavaScript expressions. My files had 25 expressions, so I needed the SVG-plus-expressions build, pinned to the version already shipped."

Design the lazy Lottie component: what are the moving parts and why is each there?

SIMPLE EXPLANATION

The component shows an empty box of the right size, waits until the box is near the viewport, then downloads the player and the animation together, swaps them in without the box ever collapsing, and pauses the animation whenever the box scrolls out of view.

HOW IT WORKS

1. **Markup parity.** An outer div and an inner box carrying the same inline styles and attributes react-lottie produced, so layout is unchanged.
2. **Placeholder.** Inside the box, an empty inline `<svg viewBox="0 0 1 1">` with `width/height: 100%` , so the box has exactly the height the real SVG will have, descender gap included. Rendered on the server and in the client's first render, so hydration matches.
3. **Proximity observer.** `rootMargin: 300px` , threshold 0. On first intersection it disconnects and starts loading.
4. **Parallel loading.** `Promise.all([import(player), fetch(jsonUrl).then(r => r.json())])` . When both resolve, state becomes `{ lottie, data }` .
5. **Swap in a layout effect.** React's commit removes the placeholder; the layout effect immediately calls `loadAnimation({ container, renderer: 'svg', loop: true, autoplay: true, animationData })` . Because the data is in memory, the player builds the SVG synchronously, before the browser paints. No empty frame, no shift.
6. **Visibility observer.** Threshold 0; `pause()` when not intersecting, `play()` when intersecting.
7. **Cleanup.** Disconnect observers, `destroy()` the animation on unmount, and ignore late promise results if unmounted.

IN THIS PROJECT

Two of the four boxes are `display: none` at most widths and never load. Measured box geometry matched the old build at every viewport (for example 605 wide / 609 tall). CLS stayed 0. The player chunk (54 KB gz) and the hero JSON (21 KB gz) load after hydration instead of before first paint.

How to say it: "Placeholder with the exact final geometry, load player and JSON in parallel on proximity, swap synchronously in a layout effect, pause when off-screen, destroy on unmount."

Why fetch the animation JSON as a static asset instead of importing it as a module?

SIMPLE EXPLANATION

Importing JSON makes it part of your JavaScript: it is bundled, parsed as code, and cached as code. Fetching it keeps it as data: smaller to parse, cached on its own, and only downloaded when needed.

HOW IT WORKS

A JSON import becomes an ES module exporting an object literal; the engine must parse that literal as JavaScript (full grammar, possible identifiers and expressions) which V8 has measured to be slower than `JSON.parse` of an equivalent string, noticeably for hundreds of kilobytes. Vite's `?url` suffix instead emits the file as a hashed asset and gives you its URL; `fetch(url).then(r => r.json())` downloads it when you ask, the browser's HTTP cache stores it independently of code, and the player can receive it as `animationData` for a synchronous build. Letting the player fetch via its `path` option would have introduced frames with an empty box because the SVG would only appear when the player's own request finished.

IN THIS PROJECT

The old main chunk carried 305 KB of raw Lottie JSON (33 KB gzipped) that every visitor parsed as JavaScript before anything rendered. Now the four files are separate assets (21 / 9.8 / 1.6 / 1.1 KB gzipped) fetched on demand; most mobile visitors download two.

How to say it: "Data should travel as data: JSON.parse is faster than parsing an object literal, the file caches separately, and I can hand it to the player synchronously."

How does ColorThief compute a dominant colour, why did it need crossOrigin, and how did you make it static?

SIMPLE EXPLANATION

ColorThief draws the image onto an invisible canvas, reads the pixels, and runs a colour quantisation algorithm that groups similar colours and picks the biggest group. Browsers only let you read canvas pixels from images that are same-origin or served with CORS headers; otherwise the canvas is "tainted" and reading throws.

HOW IT WORKS

`getColor(img, quality)` creates a canvas of the image's natural size, `drawImage` s it, calls `getImageData` , samples every *quality*-th pixel (default 10), discards near-white and transparent pixels, and runs the modified median cut quantisation (MMCQ) algorithm to build a palette, returning the most populated bucket's average as `[r, g, b]` . The `crossorigin="anonymous"` attribute on the `<img>` makes the browser request the image with CORS so that, if the server answers with `Access-Control-Allow-Origin` , the canvas is not tainted. For same-origin images it is harmless and was already present.

IN THIS PROJECT

The old card computed the colour in an `onLoad` handler and set `style.background`, so every card painted with a gradient first and snapped to its colour after the logo loaded. I ran the same library once in headless Chrome against the same five logos, confirmed the values equalled what the rendered banners showed (for example Cars24 → rgb(70, 54, 247)), and stored them as `bannerColor` in the content file. The card now renders its final colour immediately and in the pre-rendered HTML, and ColorThief is only imported on demand if a logo has no stored colour.

FOLLOW-UPS

- "Could results differ between browsers?" Slightly, because JPEG decoders differ; that is another reason to compute once in the browser you ship for and store the value.

How to say it: "Canvas read-back plus median-cut quantisation. I computed the five values once with the same library in Chrome, stored them, and kept a lazy fallback for new logos."

Icon fonts versus SVG sprites versus inline SVG: trade-offs, and why did you keep icon fonts?

SIMPLE EXPLANATION

Icon fonts draw icons as text glyphs; SVG sprites draw them as vector shapes referenced from one file; inline SVG puts the shapes directly in the HTML. Vectors are the modern recommendation, but fonts render differently at the sub-pixel level, and my brief was to match pixels.

HOW IT WORKS

Approach	Pros	Cons
Icon font	Simple CSS classes; colour via `color`; subsettable	Text rendering (anti-aliasing, hinting) differs from vector paths; FOIT; accessibility needs `aria-hidden`; one weight per file
SVG sprite (`<use href="#id">`)	One cached file; crisp vector rendering; multi-colour possible; no font loading	Needs the sprite in the DOM or an external file with same-origin constraints; styling internals is limited
Inline SVG	Full CSS control; no extra request; crisp	Bytes repeat per use in HTML; large DOM if many icons

IN THIS PROJECT

18 icons justified neither of the two CDN stylesheets nor three full font files (213 KB). The pixel-safest route was to keep fonts but subset them to the 18 glyphs (4.8 KB total) and copy the exact CSS rules, including `font-display: block`, `text-rendering: auto` for Font Awesome and `optimizeLegibility` for font-mfizz. Per-icon glyph boxes matched the originals exactly. For the redesign I would switch to an SVG sprite and drop the icon-font files entirely.

How to say it: "SVG is the better default, but fonts render glyphs differently at the sub-pixel level; for parity I subset the same fonts to 18 glyphs and copied the rules verbatim."

Why is self-hosting now preferred over public CDNs like jsDelivr or cdnjs?

SIMPLE EXPLANATION

The old argument was that a popular library on a shared CDN might already be in the visitor's cache from another site. Browsers no longer share cache across sites, so that benefit is gone, and the costs of an extra origin remain.

HOW IT WORKS

Since Chrome 86 (2020), Safari earlier and Firefox 85, the HTTP cache is partitioned by the top-level site, to prevent cross-site tracking via cache timing. A file cached while visiting site A is fetched again on site B even if the URL is identical. What remains of a shared CDN is: a new origin (DNS, TCP, TLS), a third party on your critical path (availability and security dependency), a privacy leak of your visitors' IPs, and no ability to subset or set cache headers. Self-hosting from your own CDN-backed origin reuses the connection the HTML came over, lets you subset and hash, and keeps you in control.

IN THIS PROJECT

Two icon stylesheets and fonts from jsDelivr and cdnjs, a Google Fonts stylesheet, and an emailjs script were replaced by same-origin assets or removed. Render-blocking third-party time estimated by Lighthouse at over two seconds on mobile disappeared.

How to say it: "Cache partitioning killed the shared-CDN argument; what is left is extra connections and a dependency. I self-hosted everything on the critical path."

What is the trade-off in inlining small assets as data URIs, and how did you set the limit?

SIMPLE EXPLANATION

Inlining a tiny image into the CSS or JavaScript saves a request but makes the bytes bigger and un-cacheable on their own. Below a few kilobytes the request saving wins; above it the bytes win.

HOW IT WORKS

Base64 encodes 3 bytes as 4 characters (+33%) and the result compresses poorly because the alphabet hides repetition. An inlined asset also cannot be cached separately, cannot download in parallel, and delays the file it lives in. Over HTTP/2 the per-request cost is small, so the break-even is low. Vite's default limit is 4 KB; CRA's was 10 KB. Vite accepts a function so you can decide per file type.

IN THIS PROJECT

The old main chunk carried 54 KB of base64 JPEG logos, all of which every visitor downloaded as part of the critical JavaScript. The new config inlines only images under 4 KB (the two arrow icons, two tiny project images) and never fonts, because inlining a 760-byte icon font into CSS would have left the HTML preload tag pointing at a file the CSS no longer referenced. Hashed JPEGs load in parallel and cache independently.

How to say it: "Base64 costs a third more and compresses badly; I inline only under 4 KB and never fonts so preloads stay accurate."

How does Vite decide chunks, what did manualChunks do, and how does CSS code splitting

interact with lazy components?

SIMPLE EXPLANATION

Everything statically imported from the entry goes into the main chunk; every dynamic `import()` becomes its own chunk; shared code between chunks is hoisted into common chunks. `manualChunks` lets you force certain modules into a named chunk, usually to improve caching.

HOW IT WORKS

Rollup builds the module graph and assigns modules to chunks based on import boundaries. CSS imported by a lazily loaded module is emitted as a separate CSS file that the chunk loader injects before the component renders, so you do not pay for styles of code you have not loaded. The entry HTML gets `modulepreload` tags for the entry's static chunks. A long-term caching strategy separates code that changes often (your app) from code that rarely changes (frameworks) so returning visitors reuse the stable chunk.

IN THIS PROJECT

Output: an app chunk (55 KB raw / 23 KB gz), a React chunk forced by `manualChunks` (143 KB / 46 KB), the Lottie player (204 KB / 54 KB, dynamic), ColorThief (9 KB / 4 KB, dynamic), the two GitHub cards with their own CSS (dynamic). The React chunk's hash stays stable across content edits. On GitHub Pages the ten-minute cache limits the benefit, but it costs nothing and pays off on any host with immutable caching.

How to say it: "Static imports form the main chunk, dynamic imports split, and I pinned React into its own chunk for long-term caching; lazy components bring their own CSS."

How do analytics tags normally load, what was wrong with the old setup, and how did you fix it without losing data?

SIMPLE EXPLANATION

Google's gtag snippet queues commands in an array and loads a script that processes the queue. The old page loaded a tag for a property that had been dead for two years, and that tag then loaded a second one. I load the live tag once, after the page has finished loading.

HOW IT WORKS

`window.dataLayer` is a plain array; `gtag()` pushes its arguments. The gtag.js script, when it arrives, reads the array and sends hits. Because the queue is in memory, the script can load at any time without losing `config` or events queued before it. Universal Analytics stopped processing hits in July 2023; a UA tag with a "connected site tag" to GA4 still downloads and then downloads the GA4 script, so the page paid twice (293 KB and two long tasks on mobile).

IN THIS PROJECT

The inline snippet queues `gtag('js', new Date())` and `gtag('config', 'G-001DL7NH6R')` immediately, and a `window.addEventListener('load', …)` creates the single GA4 script element. Analytics never competes with first paint or hydration. The only data lost is a visitor who leaves before `load`, which on the new site is about a second on mobile.

FOLLOW-UPS

- "Why not `requestIdleCallback`?" Also fine; `load` is simpler and available everywhere.

- "How would you measure vitals too?" The web-vitals library sending events through the same gtag queue.

How to say it: "gtag queues in an array, so loading it after the load event loses nothing. The old page loaded a dead UA tag that chained a GA4 tag; now one tag loads late."

What is a passive event listener, why throttle with requestAnimationFrame, and what was wrong with the old scroll handler?

SIMPLE EXPLANATION

When you listen to scroll or touch events, the browser must wait to see whether your handler cancels the scroll before it moves the page. "Passive" promises you will not cancel, so scrolling stays smooth. And since scroll events can fire faster than frames, you should do your work once per frame, not once per event.

HOW IT WORKS

`addEventListener('scroll', fn, { passive: true })` tells the browser `preventDefault()` will not be called, so it can start scrolling on the compositor without waiting. Inside the handler, reading `scrollTop` forces a layout flush if styles are dirty, and writing `style.visibility` dirties them again; alternating read/write across many events per frame causes layout thrashing. The fix is to set a flag and schedule one `requestAnimationFrame` callback that does the read and the write once before the next paint.

IN THIS PROJECT

The old back-to-top button assigned `window.onscroll` (overwriting any other handler) and `window.onload` inside render, and did DOM reads and writes on every event. The new hook adds a passive listener in an effect, coalesces into one rAF per frame, applies the same 20 px rule, and removes the listener on unmount.

How to say it: "Passive lets the browser scroll without waiting for me; rAF batching does one read and one write per frame. Same behaviour, no thrash, no clobbered globals."

How do you make a single-page React site indexable and shareable, and what did you change in the document head?

SIMPLE EXPLANATION

Crawlers and link previews read the HTML they receive. If it is an empty root, Google may render it later with JavaScript, but social networks and many crawlers will not. Pre-rendering gives them real content; good meta tags tell them how to describe and preview it.

HOW IT WORKS

Googlebot queues JavaScript rendering separately and can take days; Bing and most others index the initial HTML. Open Graph tags (`og:title` , `og:description` , `og:image` , `og:url`) drive link previews on LinkedIn, Slack and WhatsApp; Twitter cards add a format hint. A canonical link prevents duplicate-URL confusion (with and without trailing slash, query strings). A `description` meta tag is the snippet most engines show. The manifest describes the installable app. `robots.txt` allows or blocks crawling.

IN THIS PROJECT

The old head had a meta tag literally named `name="Portfolio"` (which no engine reads), no description, no canonical, no image, and a manifest naming the template author with two missing icons. The new head has a description, canonical, full Open Graph set with a 1200 px preview image, a Twitter card, a corrected manifest, and, because of pre-rendering, the full page content in the HTML.

How to say it: "Pre-rendering puts real content in the HTML for crawlers; I added description, canonical and Open Graph tags and fixed a manifest that pointed at missing icons."

How do responsive images work with srcset and sizes, and why did you not resize the project images?

SIMPLE EXPLANATION

`srcset` lists several copies of an image at different widths; `sizes` tells the browser how wide the image will be displayed; the browser picks the smallest copy that is sharp on this screen. A 2× screen needs twice the CSS pixels in image pixels.

HOW IT WORKS

`<img srcset="a-400.webp 400w, a-800.webp 800w" sizes="(max-width: 600px) 100vw, 250px">`: the browser evaluates `sizes` to a CSS width, multiplies by device pixel ratio, and chooses the candidate at or above that width. `<picture>` with `<source type="image/avif">` adds format fallbacks. Always include width/height (or CSS aspect ratio) so space is reserved.

IN THIS PROJECT

The project images are 512 px and displayed at 250 CSS px; on a 2× phone or laptop that needs 500 px, so 512 is already the right size and resizing would have blurred or changed pixels on high-DPR screens. Lighthouse's "properly sized images" hint assumes 1×. Format conversion (lossless WebP) gave the saving without touching pixels. For the redesign, multiple sizes and formats are a straightforward addition.

How to say it: "srcset plus sizes lets the browser pick by display width and DPR. My 512 px images serve 250 px at 2×, so I changed format, not dimensions."

How do performance and accessibility overlap in this project, and what accessibility debt remains?

SIMPLE EXPLANATION

Fast pages and accessible pages share foundations: real HTML content, stable layout, respect for user preferences, and not making people wait on JavaScript. Pre-rendering was a win for both.

HOW IT WORKS

Screen readers and keyboard users get content as soon as the HTML exists, so a pre-rendered page is usable earlier than a client-rendered one. Zero layout shift prevents mis-taps. `prefers-reduced-motion` is a user signal that animation can harm; `prefers-color-scheme` similarly. Semantic headings, alt text, visible focus, and correct roles are independent of speed but cheap.

IN THIS PROJECT

Preserved for parity but on the redesign list: the Lottie box has `role="button"` and `tabindex="0"` inherited from react-lottie though it does nothing; both carousel arrows have `alt="left"` ; inputs are visually scaled to 50%; headings are h1 and h5 without h2–h4; reduced motion is not honoured (Lottie keeps looping). Fixing these changes markup or motion, so they were documented rather than done.

How to say it: "Pre-rendering helped accessibility as much as speed. The remaining debt is markup-level and belongs with the redesign because fixing it changes what renders."

10.5 Advanced: the mechanics behind the decisions

Break LCP into its sub-parts. What did each look like before and after, and how do you read them in the tools?

SIMPLE EXPLANATION

LCP is not one thing; it is a chain of four waits. If you know which link is long, you know what to fix. For text the middle two links are zero, so the only two that matter are how fast the server answered and how long the text waited to be drawn.

HOW IT WORKS

LCP = **Time to First Byte** (navigation start to first byte of HTML) + **resource load delay** (first byte until the browser starts fetching the LCP resource, zero for text) + **resource load time** (download of that resource, zero for text) + **element render delay** (from the resource being available until the element is painted). Render delay for text is caused by render-blocking CSS, by JavaScript that must run before the element exists (client rendering), by fonts in a block period, or by the element being hidden (opacity 0, display none) until something reveals it. Lighthouse shows the breakdown in the "Largest Contentful Paint element" audit; the Performance panel's LCP marker plus the Network waterfall shows what the paint waited for; in the field, the web-vitals attribution build reports the same four numbers per user.

IN THIS PROJECT

Sub-part	Before (live, mobile)	After (local, mobile)
TTFB	0.83 s (GitHub Pages)	~0.1 s locally; Pages adds ~0.8 s in production
Load delay / load time	0 / 0 (text)	0 / 0 (text)
Render delay	7.1 s: waiting for 255 KB gz of JS plus three cross-origin stylesheets, then the hero's opacity-0 fade	Roughly the CSS round trip plus font swap; the hero is in the HTML and starts its fade at first paint
LCP	8.0 s	2.1 s

With render delay removed, TTFB is now the largest remaining term in production, which is the argument for a faster host.

FOLLOW-UPS

- "For an image LCP, what changes?" Load delay and load time dominate: discover the image early (no lazy loading, `fetchpriority="high"` , preload), serve a right-sized modern format, and keep it same-origin.

How to say it: "TTFB plus load delay plus load time plus render delay. Mine was almost all render delay, 7.1 seconds of waiting for JavaScript; pre-rendering took that to near zero."

How does Chrome choose the LCP element, how does opacity 0 affect it, and how did the hero fade interact with LCP before and after?

SIMPLE EXPLANATION

Chrome tracks the largest visible thing painted so far and reports the final one once the user interacts or the page settles. Things that are painted fully transparent do not count until they become visible.

HOW IT WORKS

Candidates are `<img>`, `<image>` inside SVG, `<video>` poster frames, elements with a CSS `background-image` loaded via `url()`, and block-level elements containing text nodes. Size is the visible area inside the viewport (overflow clipped, and for images the smaller of displayed and intrinsic size, with very low-entropy images ignored since Chrome 112). Each time a larger candidate paints, a new `largest-contentful-paint` entry is emitted; measurement stops on the first user input (scroll, tap, key). Elements rendered with `opacity: 0` are not considered painted content; when such an element's opacity becomes non-zero, that paint is when it can become a candidate. Removed-from-DOM elements still count in Chrome once painted (so image carousels do not reset LCP).

IN THIS PROJECT

Before: the hero section mounted with inline `opacity: 0` (react-reveal) after the JavaScript ran, then faded in over one second; its first visible paint, and therefore its LCP eligibility, came after hydration plus animation start. The mobile LCP element ended up being the header logo text, which was not inside a fade. After: the hero is in the HTML with its animation styles inline and static keyframes, so the fade begins at first paint; its first non-zero-opacity frame is within tens of milliseconds of the CSS arriving, and the LCP candidate is the hero subtitle text on desktop or the logo on mobile, both rendered from HTML.

FOLLOW-UPS

- "Is a fade-in bad for LCP?" A short fade from first paint is fine; a fade that waits for JavaScript is the problem.
- "Why does LCP stop on input?" After the user interacts, later paints are responses, not loading.

How to say it: "Opacity-zero paints do not count. The old hero was hidden until JavaScript faded it in; the new one fades from first paint, so LCP registers almost immediately."

Explain the state-flip plus layout-effect pattern you used to swap the Lottie placeholder without a single bad frame.

SIMPLE EXPLANATION

The problem: React removes the placeholder, then the player builds the real SVG. If anything paints in between, the box is empty for a frame and the page jumps. The solution is to make sure the real SVG exists before React lets the browser paint, which means having the data in memory and doing the insertion in a layout effect.

HOW IT WORKS

Sequence per box: (1) placeholder rendered; (2) proximity observer fires; (3) `Promise.all` fetches the player module and the JSON; (4) `setReady({ lottie, data })`; (5) React re-renders the component without the placeholder and commits the DOM mutation; (6) *synchronously*, before yielding to the browser, the layout effect runs `lottie.loadAnimation({ container, animationData: data, … })`. Because lottie-web is given the parsed data rather than a URL, it constructs the `<svg>` and its initial

frame synchronously inside that call. (7) The browser paints with the real SVG in place. Had I used `useEffect`, step 6 would run after paint, exposing one empty frame. Had I used the player's `path` option, the SVG would appear only when the player's own network request completed, exposing many empty frames. Because the placeholder and the SVG occupy the same inline box, the swap is geometry-neutral.

IN THIS PROJECT

Measured on the new build: the hero box is 605 wide and 609 tall before and after the swap at 1366 px; CLS in Lighthouse is 0; the full-page screenshots match the old build to the pixel outside the animated regions.

FOLLOW-UPS

- "Could `contain-intrinsic-size` do this?" It reserves size for `content-visibility` skipped content; it does not reproduce the descender gap and does not help with the JS-driven insertion timing.
- "What if the fetch fails?" The placeholder stays, the box keeps its size, and an error is logged; nothing shifts.

How to say it: "Load player and data first, flip state, insert the SVG synchronously in a layout effect with animationData, so the browser never paints an empty box."

Give a complete catalogue of hydration mismatch causes and the strategy for each.

SIMPLE EXPLANATION

Anything that makes the first client render differ from the server render will mismatch. The fix is always the same in spirit: make the first render deterministic and move the browser-dependent part into an effect, or tell React explicitly where differences are intentional.

HOW IT WORKS

Cause	Example	Strategy
Browser-only storage	theme from `localStorage`	Render a default on both sides; apply stored value in `useEffect`; or inline script sets a root attribute and components stop depending on it
Viewport or device	`window.innerWidth` to choose a layout	CSS media queries instead of JS; or render mobile-first and correct after mount
Time and locale	`new Date().toLocaleString()`	Render a stable value or placeholder; update in effect; `suppressHydrationWarning` on that text node only
Randomness and ids	`Math.random()` keys, incrementing ids	`useId`; deterministic keys from data
External stores	Redux state from the browser, media queries	`useSyncExternalStore` with a `getServerSnapshot`
Third-party DOM mutation	Extensions or scripts injecting nodes before hydration	Hydrate as early as possible; isolate injected UI outside the React root
Invalid HTML nesting	`<p>` inside `<p>`, `<div>` in `<table>`	Browser repairs it differently from React's output; write valid HTML (React 19 reports these clearly)

Cause	Example	Strategy
Conditional rendering on `typeof window`	`window ? <A/> : <B/>`	Never branch markup on environment; use a mounted flag set in an effect
Imperative DOM owned by a library	Lottie SVG inserted by lottie-web	Server and client render an empty container; the library attaches in an effect

In React 18 production a mismatch surfaces through `onRecoverableError` with a minified code (418: text mismatch, 423: hydration failed then client-rendered, 425: text content differs) and React re-renders from the nearest Suspense boundary. Development mode prints a readable diff.

IN THIS PROJECT

Theme (default light, effect applies storage), Lottie (placeholder on both sides, library attaches later), reveal state (deterministic initial style), banner colours (static data), and module-scope `window` reads (removed) cover every row that applied. The functional test asserted no console output after hydration.

How to say it: "Storage, viewport, time, randomness, external stores, third-party DOM, invalid nesting, environment branches, imperative libraries. Each has a known fix; I applied the deterministic-first-render rule everywhere and tested for a clean console."

If the site grew into a data-driven app, how would React 18 streaming SSR and selective hydration change your approach?

SIMPLE EXPLANATION

With `renderToString` the server must finish rendering everything before sending anything, and the client must download all the JavaScript before anything is interactive. Streaming sends the page in pieces as they are ready, and selective hydration makes the piece the user is touching interactive first.

HOW IT WORKS

`renderToPipeableStream` (Node) or `renderToReadableStream` (edge runtimes) starts sending HTML immediately. A `<Suspense>` boundary whose data is not ready is sent as its fallback with a placeholder marker; when the data resolves, the server streams the real HTML plus a tiny inline script that swaps it into place, all in the same response. On the client, React hydrates boundaries independently and out of order: if the user clicks inside a boundary that has not hydrated, React prioritises hydrating that boundary synchronously and replays the event. Code for a boundary can arrive via `React.lazy` without blocking others. This removes the "all or nothing" wait of classic SSR and makes TTI scale with what the user touches rather than the whole page.

IN THIS PROJECT

Unnecessary: one page, build-time content, no data fetching, 20 KB of markup rendered in milliseconds. If the GitHub section were enabled and fetched live, the right design would be a Suspense boundary around it with a fallback, data fetched at the edge or at build time with revalidation, and the rest of the page streamed first.

How to say it: "Streaming sends HTML as it is ready and selective hydration makes the touched part interactive first; for one static page renderToString at build time is simpler and equivalent."

Describe the full lifecycle of a web font request and how preload, font-display and metric overrides fit into it.

SIMPLE EXPLANATION

A font goes through: declared in CSS, needed by some text, requested, downloading while the browser decides what to show, arrived and swapped in. Each step is a place where you can lose time or cause a visible jump.

HOW IT WORKS

1. **Declaration.** `@font-face` parsed; nothing fetched.
2. **Need.** Style resolution assigns the family to an element whose text includes a character in the face's `unicode-range`. Only now does the fetch start, after the CSS has been applied and the element exists. For a client-rendered site that is after JavaScript runs.
3. **Preload short-circuits step 2.** `<link rel="preload" as="font" crossorigin>` starts the fetch when the HTML is parsed. It must be a CORS fetch (`crossorigin` attribute) to match how fonts are fetched; otherwise the browser fetches twice.
4. **Block period.** Governed by `font-display`: text is drawn invisible (but laid out) or in the fallback font.
5. **Swap.** The font arrives; text re-renders in it. If fallback and web font have different metrics, lines re-wrap and the page shifts (CLS). `size-adjust`, `ascent-override`, `descent-override` and `line-gap-override` on a local fallback face make the fallback occupy the same space, so the swap is shift-free.
6. **Observation.** `document.fonts.ready` resolves when all pending loads settle; `document.fonts.load('1em Montserrat')` can force one; `FontFaceSet` events report loading and loaded.

IN THIS PROJECT

Step 2 explains the header bug: the logo text existed only after React rendered, so the Agustina request began at mount and the header was measured with the fallback. Step 3 was broken (404 preloads) and is now correct for all five fonts. Step 5 is unchanged (`swap`, no metric overrides) for parity, and I added a re-measure at step 6 so the sticky header's spacer always reflects the real font.

How to say it: "Fonts are fetched only when text needs them, which is late on a client-rendered page; preload moves that to HTML parse time, font-display decides what shows meanwhile, and metric overrides make the swap shift-free."

Why exactly do same-origin font preloads need the crossorigin attribute?

SIMPLE EXPLANATION

The browser caches requests by URL *and* by how they were requested. Fonts are always requested in "CORS mode". A preload without `crossorigin` is a plain request, so when the CSS later asks for the font in CORS mode, the cached copy does not match and the browser downloads it again.

HOW IT WORKS

The CSS Fonts specification requires font fetches to use the CORS protocol with anonymous credentials, even for same-origin URLs, partly to prevent fonts being used as a cross-site fingerprinting or exfiltration vector. The preload cache matches on URL, destination (`as`) and request mode/credentials. `crossorigin` (equivalent to `crossorigin="anonymous"`) sets the preload to CORS mode with no

credentials, which is what the later font request uses, so it is a hit. Without it Chrome logs "A preload for … is found, but is not used because the request credentials mode does not match". The same logic applies to `fetch()` -ed resources preloaded with `as="fetch"` .

IN THIS PROJECT

All five font preloads carry `crossorigin` and `type="font/woff2"` ; the functional test, which captures console warnings, showed none about unused preloads.

How to say it: "Font requests are always CORS-mode; a preload must match the mode or it is a cache miss and a double download. Same-origin does not exempt you."

What is HTTP cache partitioning, why was it introduced, and what did it change for front-end architecture?

SIMPLE EXPLANATION

Browsers used to have one shared cache: a file downloaded on one website could be reused on another. That allowed sites to detect what other sites you had visited by timing cache hits. Browsers now keep a separate cache per top-level site, which closes the leak and removes the performance argument for sharing library URLs across sites.

HOW IT WORKS

The cache key became (top-level site, frame site, URL) in Chrome 86 (October 2020); Safari partitioned earlier; Firefox 85 did the same for all network state. A jQuery or Font Awesome file fetched from a shared CDN while on site A is a cache miss on site B. Consequences: shared CDN URLs no longer save bytes for first-time visitors to your site; each third-party origin is pure overhead on a cold visit; bundling or self-hosting popular libraries is correct; and cross-site resource timing attacks are mitigated.

IN THIS PROJECT

This is the principled reason the icon fonts and emailjs script moved from jsDelivr and cdnjs onto the site's own origin. The only shared benefit that survives partitioning is within your own site across pages, which self-hosting also gives you.

How to say it: "Since 2020 caches are keyed per top-level site, so shared CDN URLs buy nothing on a cold visit and still cost a connection. Self-hosting is the modern default."

Design a flash-free, hydration-safe dark mode for the redesign.

SIMPLE EXPLANATION

Decide the theme before the page paints, in a few lines of inline JavaScript, by setting an attribute on the root element. Make all colours CSS variables that depend on that attribute. Then no React component needs to know the theme, so nothing can mismatch and nothing flashes.

HOW IT WORKS

```
<script>
  try {
    var t = localStorage.getItem('theme');
    if (!t) t = matchMedia('(prefers-color-scheme: dark)').matches ? 'dark' : 'light';
    document.documentElement.dataset.theme = t;
```

```
  } catch (e) {}
</script>
```

```
:root { --bg: #ede6ea; --fg: #1f1a24; color-scheme: light; }
:root[data-theme="dark"] { --bg: #171c28; --fg: #fff; color-scheme: dark; }
body { background: var(--bg); color: var(--fg); }
```

The inline script is tiny, runs before any paint because it is in the head before the stylesheet, and is allowed under a CSP with a nonce or hash. A toggle writes `localStorage` and the attribute; `color-scheme` makes form controls and scrollbars follow. For per-request SSR, mirror the preference in a cookie so the server can set the attribute in the HTML; for SSG, the inline script is enough. Components render identical markup regardless of theme, so hydration cannot mismatch and no second render is needed. `prefers-color-scheme` changes at runtime can be observed with `matchMedia().addEventListener('change')`.

IN THIS PROJECT

Not done, because the current components compute class names from `isDark` and changing when the theme applies would alter behaviour for system-dark visitors. It is the first item I would do in the redesign, because it also removes a full-tree re-render and the only remaining "flash" on the site.

How to say it: "Inline script sets data-theme before paint, colours are CSS variables, components are theme-agnostic. No flash, no hydration risk, no re-render."

What is the per-frame cost model of a Lottie SVG animation, and what are the alternatives for heavy animations?

SIMPLE EXPLANATION

Every frame, the player recalculates every animated property and writes it into the SVG. The browser then has to recompute styles, re-layout and repaint the changed area. Cost grows with the number of shapes and the area they cover, sixty times a second, for as long as the animation runs.

HOW IT WORKS

Per frame: interpolate keyframes and evaluate expressions (JavaScript, main thread) → set attributes such as `transform`, `d`, `opacity`, `fill` on potentially hundreds of SVG nodes (DOM mutation) → style recalculation for affected nodes → layout of SVG geometry → paint of the invalidated region → composite. SVG paths are rasterised on the CPU; large or blurred areas are the worst. Alternatives: the **canvas renderer** (one element, draws directly, no DOM churn, no per-part CSS; cost is paint only); **OffscreenCanvas in a Web Worker** (lottie-web has a worker build) to take the work off the main thread; **dotLottie** (a zipped container, roughly 80% smaller to download, same runtime cost); **reducing the file** (fewer layers, simpler paths, no expressions) in After Effects; **frame-rate capping** (`setSubframe(false)`, or playing at 30 fps); **pausing when invisible**; and replacing decorative loops with a static image or a short CSS animation, especially under `prefers-reduced-motion`.

IN THIS PROJECT

Four SVG animations ran continuously, two of them invisible. Now only visible ones run, and only once visible. The SVG renderer was kept for parity (canvas would change anti-aliasing and the DOM). The main-thread breakdown on mobile fell from 3.5 s to 1.8 s during load, and steady-state work is roughly halved on phones.

How to say it: "Per frame it is JS interpolation, DOM writes, style, layout, paint. I cut the running animations in half and deferred them; canvas, workers and dotLottie are the next levers if motion stays heavy in the redesign."

Explain compositor layers, will-change and how to inspect paint and layer work in DevTools.

SIMPLE EXPLANATION

The browser can draw parts of the page onto separate layers, like sheets of glass, and then slide or fade a sheet without redrawing what is on it. That is why transform and opacity animations are cheap. Too many sheets, however, eat memory and time to composite.

HOW IT WORKS

Elements get their own compositor layer when they have 3D transforms, active transform/opacity animations, `will-change: transform/opacity`, `position: fixed` in some cases, video, canvas, or overlap something composited (overlap-induced promotion). Each layer is rasterised into GPU textures; compositing combines them per frame on the compositor thread, which keeps running even when the main thread is busy, so animations stay smooth during JavaScript. Costs: memory per layer (width × height × 4 bytes), upload time, and "layer explosion" when many elements get promoted by overlap. `will-change` should be applied shortly before an animation and removed after; permanently promoting everything is counter-productive. Inspect with DevTools: Performance panel (Paint and Composite Layers events, Paint flashing and Layer borders in Rendering settings), the Layers panel (why each layer exists, its memory), and Lighthouse's non-composited animations audit.

IN THIS PROJECT

Reveal animations use `translate3d`, which also hints layer promotion for the duration, and opacity. The experience cards have a permanently blurred 5 px layer per card, which is paint-heavy on hover; the contact form transitions all properties. Both were preserved for parity and are listed for the redesign.

How to say it: "Layers let the compositor move and fade without repainting; promote deliberately with will-change just before animating, and read the Layers and Performance panels to see what you are paying for."

What are CSS containment and content-visibility, when do they help, and why did you not use them here?

SIMPLE EXPLANATION

Containment tells the browser that a box's contents cannot affect the outside, so it can skip or isolate work. `content-visibility: auto` goes further: the browser does not even render off-screen sections until they are about to scroll into view.

HOW IT WORKS

`contain: layout paint size style` (or `strict` / `content`) lets style and layout engines limit invalidation to the box. `content-visibility: auto` implies containment and skips rendering (style, layout, paint) for content outside the viewport, which can cut initial render time dramatically on long pages. Because skipped content has no laid-out size, you must give the box `contain-intrinsic-size` so scrollbars and anchor positions are stable; find-in-page and accessibility trees still see the content but some behaviours (anchor scrolling, `scrollIntoView`) need care. It is best for long lists and articles where the fold hides most of the content.

IN THIS PROJECT

The page is about 4,600 px tall at desktop width with eight sections; initial style and layout for the whole page is a few milliseconds and not where time went. Guessing intrinsic sizes would have risked scrollbar and anchor differences against the parity baseline for a negligible gain. For a long blog index in the redesign, `content-visibility: auto` on each article card group is a cheap win.

How to say it: "content-visibility skips rendering off-screen content; it shines on long pages. Mine is short, so the gain was marginal and the parity risk real."

How do you optimise INP in practice: input delay, processing time and presentation delay?

SIMPLE EXPLANATION

When a user taps, three things take time: waiting for the main thread to be free (input delay), running your handler (processing), and getting the next frame on screen (presentation). INP measures all three. You shrink it by keeping the main thread free, keeping handlers short, and avoiding heavy work before the frame.

HOW IT WORKS

Input delay comes from long tasks already running (third-party scripts, hydration, heavy effects). Fix: split and defer work, lazy-load, use `scheduler.yield()` or `setTimeout(0)` between chunks, move computation to workers. **Processing time** is your handlers: avoid synchronous layout reads after writes, avoid large state updates that re-render big trees, use `startTransition` for non-urgent updates so the urgent visual feedback renders first. **Presentation delay** is style/layout/paint of the result: keep DOM updates small, avoid layout thrashing, avoid huge re-renders, and do not block with `requestAnimationFrame` callbacks that do heavy work. Measure with the web-vitals attribution build in the field and the Performance panel's Interactions track in the lab.

IN THIS PROJECT

Interactions are few: carousel arrows, tooltips, the form, the top button. The main-thread cleanup (no analytics or Apollo during load, lazy Lottie, rAF-throttled scroll handler, paused off-screen animations) reduces input delay, which is the dominant INP component on content sites. TBT, the lab proxy, went from 330 ms to under 90 ms.

How to say it: "INP is input delay plus processing plus presentation. On a content site input delay dominates, so freeing the main thread during and after load is the main lever."

How do you avoid leaks and stale callbacks with observers, timers and animations in React effects?

SIMPLE EXPLANATION

Anything you start in an effect you must stop in its cleanup: observers disconnected, timers cleared, animations destroyed, listeners removed. And when an async result arrives after the component is gone, you must ignore it.

HOW IT WORKS

An effect returns a cleanup function that React runs before the effect runs again and on unmount. Patterns: a `cancelled` flag captured in the closure that async continuations check; refs for timer handles so they survive re-renders; `AbortController` for fetches; `observer.disconnect()`; `anim.destroy()` for

Lottie, which also removes its DOM. In StrictMode development React runs mount → cleanup → mount to prove the cleanup is complete. Missing cleanup shows up as detached DOM nodes in the Memory panel, listeners accumulating in the Event Listeners pane, or callbacks firing on unmounted components.

IN THIS PROJECT

DisplayLottie disconnects both observers, destroys the animation and sets a cancelled flag so a late `Promise.all` result does not call `setState` on an unmounted component. Reveal disconnects its observer. Tooltip stores its timeout in a ref (the original re-declared a local variable each render, so the handle was lost) and clears it on unmount. Top removes its scroll listener. Contact clears the two-second reset timer on unmount. The Header's fonts.ready subscription uses a cancelled flag.

How to say it: "Every effect returns a cleanup: disconnect, clear, destroy, remove, and a cancelled flag for async results. StrictMode's double-invoke is the test."

How does Vite's asset pipeline handle images, fonts and JSON, including ?url imports, the public directory and base-path rewriting?

SIMPLE EXPLANATION

Anything you import from source is hashed, copied to the output and referenced by its final URL; anything in `public/` is copied as-is and referenced by a stable path. Vite rewrites URLs in HTML, CSS and JS so they include the deployment sub-path.

HOW IT WORKS

`import img from './a.png'` returns a URL string to `assets/a-<hash>.png` (or a data URI if under the inline limit). `import url from './anim.json?url'` forces the asset path for a file type Vite would otherwise bundle. `?raw` gives the contents as a string. CSS `url()` references to source files are rewritten the same way. In `index.html`, `<link href>`, `<script src>` and `<img src>` pointing at source files are processed and hashed; references to `public/` files are rewritten to include `base`. Hashes are content hashes, so the same file imported from CSS and preloaded from HTML gets the same name. The SSR build uses the same hashing, so server-rendered `<img src>` values match the client build's files.

IN THIS PROJECT

`base: '/udit-portfolio/'` replaced CRA's `homepage` and fixed the preload paths that had been hard-coded without the sub-path. Fonts are imported by CSS and preloaded by HTML and resolve to the same hashed file. Lottie JSON uses `?url`. Favicon, manifest, robots, Open Graph image and `.nojekyll` sit in `public/`.

How to say it: "Imported assets are content-hashed and URL-rewritten with the base path, in HTML, CSS and both client and SSR builds, which is why my preloads and CSS can never disagree again."

How does Chrome prioritise resource downloads, and how did that shape the new head?

SIMPLE EXPLANATION

The browser does not download everything at once; it assigns each request a priority based on what it is and where it is. CSS and fonts for the current page come first, scripts next depending on position and attributes, images later unless they are in the viewport.

HOW IT WORKS

Chrome's defaults: HTML Highest; CSS in head Highest; fonts (when needed) Highest; synchronous scripts before the first image High, after Medium; `async` / `defer` scripts Low; module scripts High when in head; preload inherits by `as`; images Low, raised to High when found in the viewport after layout; `fetchpriority` nudges up or down. With HTTP/2, priorities are signalled to the server, and Chrome also limits the number of in-flight low-priority requests until the critical ones finish. The preload scanner discovers resources in the raw HTML before the DOM is built, so putting critical hints early in the head matters.

IN THIS PROJECT

Head order: meta and title, five font preloads (Highest, discovered before the stylesheet is even parsed), the inline GA queue (no network), then the module script and modulepreload (High) and the stylesheet (Highest). Analytics is injected only after `load`, so it never takes bandwidth from any of these. The Lottie player and JSON are requested from JavaScript after hydration and after the proximity observer fires, so they queue behind the critical path naturally.

How to say it: "CSS and fonts first, scripts by position and attribute, images when visible. My head preloads fonts, defers scripts as modules, and keeps analytics out of the loading window entirely."

What are Subresource Integrity and Content Security Policy, and how does self-hosting change the security posture?

SIMPLE EXPLANATION

SRI lets the browser verify a third-party file's hash so a compromised CDN cannot inject code. CSP lets the page declare which origins may supply scripts, styles, fonts and connections, so injected code cannot load or run. Self-hosting shrinks the list of parties you must trust.

HOW IT WORKS

SRI: `<script src integrity="sha384-…" crossorigin="anonymous">`; the browser hashes the response and refuses it on mismatch. It only works for immutable files you pin by version. CSP: an HTTP header or meta tag with directives such as `script-src 'self' https://www.googletagmanager.com 'nonce-…'`, `style-src 'self'`, `font-src 'self'`, `connect-src 'self' https://api.emailjs.com https://www.google-analytics.com`. Inline scripts need a nonce or hash; `report-to` lets you collect violations before enforcing. React's escaping already protects against most XSS, but CSP protects against injected third-party scripts and supply-chain compromise of a CDN.

IN THIS PROJECT

The old page loaded Font Awesome from jsDelivr without SRI and font-mfizz from cdnjs with SRI, plus emailjs from jsDelivr without SRI. Now the only third party is Google Tag Manager plus the EmailJS API endpoint. A strict CSP would be a few lines on a host that allows headers (GitHub Pages does not), with a hash for the small inline GA snippet.

How to say it: "SRI pins third-party file hashes; CSP whitelists origins. Removing third parties made both easier: one script origin, one API endpoint, everything else self."

Why did you not add a service worker, and what is the update-flow problem?

SIMPLE EXPLANATION

A service worker can cache the whole site for offline use and instant repeat visits, but it introduces a hard problem: users keep seeing the old version until the new worker takes over, which by default happens only after every tab of the site is closed.

HOW IT WORKS

The worker intercepts fetches and can serve from its cache. On deploy, the browser detects the changed worker script, installs the new one in the background, and keeps it "waiting" until no clients use the old one; `skipWaiting()` plus `clients.claim()` can activate immediately, but then a page may run new assets against an old HTML or vice versa unless the precache is versioned carefully (Workbox handles this). You also take on cache size limits, storage eviction, debugging stale caches, and the fact that your HTML caching policy is now your own code. For a one-page portfolio with ten-minute HTTP caching on Pages, the benefit (offline portfolio) is small and the risk (users stuck on an old version after a deploy) is real.

IN THIS PROJECT

CRA had generated a service worker and a precache manifest in every build but the app called `unregister()`, so it was dead weight; both are gone. Pre-rendering already makes the page readable instantly on repeat visits.

How to say it: "Service workers buy offline and instant repeats at the cost of an update-flow problem. For a single page I chose HTTP caching and pre-rendering instead."

What do you know about how GitHub Pages serves a site, and what did it mean for this project?

SIMPLE EXPLANATION

GitHub Pages takes a branch of static files, optionally runs them through the Jekyll static site generator, and serves them through a CDN (Fastly) with fixed settings you cannot change.

HOW IT WORKS

Project sites live under `https://<user>.github.io/<repo>/`, so every absolute URL needs the repository sub-path (hence `base` in Vite and the old `homepage` in CRA). Jekyll ignores files and folders beginning with an underscore unless a `.nojekyll` file is present. Observed headers: `content-encoding: gzip` (no Brotli), `cache-control: max-age=600` on every file, HTTP/2, and in Lighthouse a mobile TTFB around 0.8 s. No custom headers, redirects or server logic. The `gh-pages` npm package pushes a build directory to the branch; it skips dotfiles unless told otherwise.

IN THIS PROJECT

The old font preloads failed precisely because they ignored the sub-path. The new build sets `base`, deploys with `--dotfiles` so `.nojekyll` is included, ships no source maps, and documents that Brotli, immutable caching and lower TTFB require a different host.

How to say it: "Fastly-backed static hosting under a sub-path, gzip only, ten-minute cache on everything, Jekyll unless you opt out. The code is host-agnostic; the remaining gains are hosting gains."

How do React events work, and why was onClick="null" wrong?

SIMPLE EXPLANATION

React does not attach your handler to each element. It listens once at the root and dispatches to your components. Handlers must be functions; passing a string is a template-era habit that React ignores with a warning.

HOW IT WORKS

Since React 17, event listeners are attached to the root container (not `document`), one per event type, and React builds a synthetic event that bubbles through the component tree in order. Props like `onClick` must be functions; a string value triggers "Expected onClick listener to be a function, instead got a value of string type" and no listener is attached. Passive listeners and `preventDefault` work as in the DOM, with the caveat that React registers some events (scroll, touch) as passive by default. Native events on the same element fire before React's delegated handler.

IN THIS PROJECT

The old DisplayLottie wrapped the player in `<div onClick="null">` to "override Lottie's click-to-pause"; it did nothing except warn, and react-lottie's own click handler remained. The new component has no click behaviour on the wrapper and the player is not clickable, matching what users actually experienced (clicking paused nothing visible because the wrapper intercepted nothing). Removing it also avoids a hydration warning.

How to say it: "React delegates at the root and expects function handlers; a string handler is ignored with a warning, so it was dead code I removed."

When is memoisation (React.memo, useMemo, useCallback) worth it, and why did you not add it?

SIMPLE EXPLANATION

Memoisation skips work when inputs have not changed, but it costs memory and comparisons on every render. It pays off when a render is expensive or a subtree is large and frequently re-rendered with the same props. Adding it everywhere makes code slower and harder to read.

HOW IT WORKS

`React.memo` shallow-compares props and skips the component's render when equal; it only helps if the parent re-renders often with unchanged props, and it is defeated by inline objects or functions unless those are memoised with `useMemo` / `useCallback`. The React Profiler shows commit durations and which components rendered and why; optimise what it shows as expensive. React 19's compiler automates much of this.

IN THIS PROJECT

Renders are rare (theme once, carousel paging, reveal state flips) and cheap (a few hundred elements). The Profiler would show sub-millisecond commits. Memoisation would add code for no measurable gain, and the parity constraint favoured minimal changes. The expensive work on this site was never React rendering; it was network bytes, third parties and animation.

How to say it: "Memoise what the Profiler proves expensive. Here React rendering was never the bottleneck, so I spent the effort on bytes, blocking resources and animation instead."

10.6 Expert: debugging stories, measurement and edge cases

Tell the complete story of the header spacer bug: symptom, investigation, root cause, fix and lesson.

SYMPTOM

After the first rebuild, the full-page screenshot diff showed zero differing pixels in the content, yet the new page was 12 px taller at desktop widths, 17 px at 1920, 2 px on phones. Identical content, different height: something invisible had changed size.

INVESTIGATION

1. A section-offset probe recorded the top and height of every major section in both builds: every section was shifted down by exactly 12 px from the first one. The difference was at the very top.
2. A probe of the sticky header's wrapper, run through the exact capture steps, showed the old build's react-headroom spacer at `height: 78px` after load and after scrolling, but `90px` after a full-page screenshot (which resizes the viewport and triggers react-headroom's resize re-measure). The header element itself was 90 px throughout. The new build read 90 px at every step.
3. A headless dump of the live production DOM confirmed `height: 78px` in production.
4. Explaining 78 versus 90: the logo uses the Agustina web font. Fonts are requested only when text needs them; the logo text exists only after React renders; react-headroom measures in `componentDidMount`, immediately, with the fallback font's metrics. Twelve pixels is the difference between the fallback and Agustina at the logo's size. The spacer never updated because nothing re-measured until a resize.

WHY THE NEW BUILD DIFFERED

Fonts are preloaded and the module script is deferred, so Agustina usually arrives before hydration; the measurement is correct by luck of ordering. To make it correct by design, the Header dispatches a synthetic `resize` event after `document.fonts.ready`, which react-headroom handles by re-measuring.

DECISION

The old 78 px was a race outcome, not a design: it varied with operating system (fallback fonts differ) and corrected itself on any window resize. A state that changes when you resize the window is not a stable pixel target. I kept the correct 90 px, normalised the baseline capture with the same synthetic resize so the diff compares like with like, and documented it as the one intentional deviation.

LESSON

Any measurement of text-dependent layout taken at mount is suspect until fonts have loaded. Wait for `document.fonts.ready`, or use a `ResizeObserver` on the element so the measurement follows reality, or avoid measuring by using CSS (sticky positioning) in the first place.

How to say it: "Twelve pixels of extra height with identical content pointed at the header. Probing the spacer through each step showed 78 before the font and 90 after any resize; the fix is to re-measure when fonts are ready, and I argued the old value was a race, not a design."

How did you locate the 185 px tablet discrepancy, and what does the method teach about

debugging layout regressions?

METHOD

The diff said 18.5% of pixels differed at 768 px and the page was 185 px taller. Pixel diffs tell you *that* something changed, not *what*. So I wrote a probe that loaded both builds, scrolled through them identically, and printed the bounding box of a list of landmark selectors: header wrapper, hero, hero text, hero image, each section, each card grid, the contact card, the footer. The first row that differed was the hero text: 336 px wide in the old build, 671 in the new; the hero image had moved from beside the text to below it. Everything after was simply shifted by the hero's extra height. That narrowed the cause to the hero's flex versus block layout at that breakpoint, which pointed straight at the deleted stylesheet with a `.greeting-main { display: flex }` rule.

LESSON

Pair every visual diff with a geometry dump. Record positions of a dozen landmarks alongside each screenshot; when the images disagree, the first landmark whose box differs is the culprit, and everything below it is consequence. Also: treat “duplicate” CSS as suspect until a cascade analysis or a diff proves it redundant.

How to say it: “Screenshots say ‘different’; a landmark geometry dump says ‘where’. The first differing box was the hero text at double width, which led straight to the cascade dependency.”

How do you design a deterministic visual-regression harness, and what did you get wrong the first time?

DETERMINISM CHECKLIST

- Same browser binary and version; fixed flags (`--font-render-hinting=none` , `--hide-scrollbars`); fixed device scale factor.
- Fresh browser context per capture so storage and caches start empty (the theme logic depends on it).
- Explicit media emulation (`prefers-color-scheme`) rather than relying on the machine.
- Wait for `networkidle0` and `document.fonts.ready` ; force `scroll-behavior: auto` .
- Drive scroll deterministically in fixed steps with pauses, so every observer fires; wait longer than the longest animation.
- Return to the top and settle; neutralise known races (the synthetic resize for the old header spacer).
- Mask regions that are legitimately time-dependent (Lottie boxes) using their measured bounding boxes rather than hard-coded rectangles.
- Record geometry (scroll height, landmark boxes, element counts, loaded fonts, console errors) next to each image.
- Compare with pixelmatch (YIQ colour distance, threshold 0.1, anti-aliasing detection off) and write a diff image.
- Sanity-check the comparator with a known difference before trusting a zero.

WHAT WENT WRONG, HONESTLY

Three things. My local static server had a path bug that returned 404 for the index page, so the first “baseline” was eight screenshots of an error page with zero elements; I caught it because the report showed `els=5` and `masks=0` . pixelmatch version 6 is ESM-only and my diff script was CommonJS, which failed at import; I pinned version 5. And the first diffs reported zero differing pixels on light-mode

captures that were 12 px taller: my padding filled the extra rows with transparent black, which pixelmatch blends to white, and white is within the 0.1 threshold of the page's pale background. The height comparison and the dark-mode captures exposed it. I then added the height check to the report and normalised the baseline so heights matched.

How to say it: "Fix every source of nondeterminism, mask what is legitimately dynamic, record geometry alongside pixels, and sanity-test the comparator. My first baseline was an error page and my first zero was a threshold artefact; the metadata caught both."

What does pixelmatch actually compute, and how do you choose a threshold?

HOW IT WORKS

For each pixel pair it converts RGBA to the YIQ colour space (luma plus two chroma axes), after blending alpha against white, and computes a weighted squared distance that approximates perceptual difference. If the distance exceeds `threshold² × maxDelta` (threshold 0.1 is the default; smaller is stricter), the pixel is marked different. With anti-aliasing detection on, a differing pixel surrounded by a gradient of neighbours on one side is treated as anti-aliasing and ignored. It writes a diff image with differing pixels in red and the rest faded, and returns the count.

CHOOSING A THRESHOLD

For "identical rendering" checks with the same browser and flags, 0.1 with anti-aliasing detection off is strict enough to catch a one-pixel shift of text (my sanity test: shifting a screenshot by 12 px produced 408,021 differing pixels) and lenient enough to ignore nothing meaningful. For cross-browser or cross-OS comparisons you raise it or enable anti-aliasing detection and accept small counts. Always report the count and the location (bounding box of red pixels) so a non-zero result can be judged: my one residual (842 px at 768 width) was inside the hero animation where it overflowed its masked box.

How to say it: "YIQ perceptual distance per pixel against a threshold, with optional anti-aliasing tolerance. I used strict settings for same-browser comparisons and located the single residual to a Lottie region."

How would you find out what is inside a JavaScript bundle, with and without source maps?

WITH SOURCE MAPS

Decode the `mappings` (base64 VLQ segments per generated line), sort by generated position, and credit the bytes up to the next segment to the segment's source file; group by package. Tools that do this: source-map-explorer, Lighthouse's Treemap view, Vite's `rollup-plugin-visualizer`, webpack-bundle-analyzer. This is how I measured that lottie-web was 261 KB of the 642 KB vendor chunk and Apollo plus graphql about 127 KB.

WITHOUT SOURCE MAPS

Search the minified code for library fingerprints (license banners, distinctive strings like "lottie", "ApolloClient"), use the DevTools Coverage tab to see how much of each file executed during load (Lighthouse's unused-JavaScript audit is the same idea), build locally with maps enabled, or use `npx vite build --sourcemap` in a throwaway branch. Package-size registries (bundlephobia) give upper bounds but not what survived tree shaking.

How to say it: "Source maps let you attribute minified bytes to packages; without them you rely on coverage data and fingerprints. Measure the built output, not the package registry."

How do you verify a font subset did not drop anything the page needs?

APPROACH

1. **Coverage check.** Extract every distinct character in the rendered text (from the pre-rendered HTML and content file) and confirm each is inside the subset's ranges; include punctuation and symbols such as ©, en dash and curly quotes.
2. **Runtime check.** After load, read `document.fonts` and confirm the expected families report `status === 'loaded'` and that no fallback family is being used for visible text (compare `getComputedStyle(el).fontFamily` and, more reliably, measure glyph widths).
3. **Geometry check.** Measure the bounding boxes of elements that depend on the font (icons, the logo, headings) in both builds; identical widths and heights mean identical glyphs and metrics.
4. **Pixel check.** The screenshot diff over the whole page.

IN THIS PROJECT

Loaded families matched (Agustina Regular, Font Awesome 5 Brands, Font Awesome 5 Free, FontMfizz, Montserrat), the ten skill icons and five social icons had identical pixel boxes, and the full-page diffs were zero outside animations. The regeneration script keeps the ranges and codepoints explicit so the check is repeatable.

How to say it: "Check character coverage, confirm the families loaded, compare glyph geometry, then diff pixels. All four agreed."

How do you compare Lighthouse results fairly, and why can two honest runs differ by five points?

SOURCES OF VARIANCE

Network jitter and server response time (especially on real hosts), CPU contention on the machine running Lighthouse, browser caching state, A/B variation in third-party scripts, and the simulation model's sensitivity to small timing changes. Scores near the steep part of the log-normal curve move several points for a few hundred milliseconds of LCP.

FAIR COMPARISON PROTOCOL

- Serve both versions from the same server with the same compression, on the same machine, back to back.
- Use the same Lighthouse version and preset; prefer simulated throttling for reproducibility.
- Run several times and compare medians; report ranges.
- Compare metrics, not just the score; the score hides which metric moved.
- Separate hosting effects (TTFB) from code effects by also running against the live host and attributing the difference.

IN THIS PROJECT

Old versus new were measured on the same local gzip server with Lighthouse 12.8.2; mobile went 63 → 98–99 across runs, desktop 97 → 100. The live baseline (53) is lower than the local baseline because of GitHub Pages' 0.8 s TTFB and network variance, which I reported separately rather than folding into the improvement.

How to say it: "Same server, same version, same preset, several runs, medians and ranges, and separate hosting from code. 63 to 98 is like for like; 53 is the live number with Pages' latency."

How do you debug a hydration mismatch that only appears in production?

APPROACH

1. Register `onRecoverableError` on `hydrateRoot` and log the error plus `info.componentStack`; production errors are minified codes (418, 423, 425) but the stack names the component.
2. Build the SSR output locally and diff the server HTML against the client's first render: render the same component with `renderToString` in a Node script and with `createRoot` in a headless browser, then compare the serialised DOM.
3. Look for the usual suspects: storage, time, viewport, random ids, environment branches, third-party DOM injection, invalid nesting.
4. Reproduce in development where React prints the exact attribute or text difference.
5. Fix by making the first render deterministic; use `suppressHydrationWarning` only for intentional text differences.

IN THIS PROJECT

I kept the production `onRecoverableError` logging on and had the functional test capture console output after load; it stayed empty. A deliberate experiment (rendering the theme from storage on the first render) would have produced class-name mismatches on every themed element, which is why the theme applies in an effect.

How to say it: "Log recoverable errors with component stacks, diff server HTML against client first render, hunt the known causes, and make the first render deterministic."

How did a functional test expose the contact form bug, and what is the general lesson about native form behaviour?

WHAT HAPPENED

The test typed an invalid email, clicked Send, and expected the button to read "Invalid Email!". Instead it read "Send", and unrelated assertions changed too: every revealed section had reset to opacity 0 and the hero was mid-animation. That pattern, all state reset at once, means a navigation happened. The form had submitted natively and reloaded the page with the field values in the query string, because `preventDefault()` was only called on the valid branch. The "Invalid Email!" state flashed for a few milliseconds before the reload, which is why nobody noticed in manual testing.

LESSON

A form's default action is to navigate. In a single-page app, always call `preventDefault()` first in the submit handler and then branch. Also: when many assertions fail together in ways that look unrelated, suspect a reload or a crash rather than many bugs.

How to say it: "Several unrelated assertions failed at once, which smelled like a reload; the form was submitting natively on the invalid path. Prevent default first, then branch."

How do you attribute long tasks in the Performance panel?

APPROACH

Record a load with CPU throttling matching your target device. In the Main track, long tasks have red triangles. Select one and use the Bottom-Up tab grouped by URL or by activity to see which script and which functions consumed the time; the Call Tree shows the path from the task root; Summary shows scripting versus rendering versus painting. Lighthouse's "Reduce JavaScript execution time" and "Minimize main-thread work" audits give per-URL totals, and "Avoid long main-thread tasks" lists the tasks with their start times. In the field, a `PerformanceObserver` on `longtask` entries gives attribution containers, and the `long-animation-frame` API (Chrome 123+) gives script attribution for frames that blocked.

IN THIS PROJECT

Old mobile trace: a 344 ms task in the main chunk (parsing inline Lottie JSON and bootstrapping React), 138 ms and 70 ms tasks in the two gtag scripts, and a 59 ms parse of the HTML. New: no gtag during load, JSON fetched not parsed as code, the player initialising in a shorter task after hydration.

How to say it: "Record with throttling, open the long task, Bottom-Up by URL. The old page's long tasks were inline Lottie JSON and two analytics scripts, all now deferred or gone."

Walk through how you triage a credential committed to a public repository.

STEPS

1. **Identify** what it is and what it can do: a GitHub personal access token in a code comment, scope unknown, public since September 2021.
2. **Assume compromise.** Anything pushed to a public repository has been scraped by bots within minutes.
3. **Revoke and rotate** at the provider (GitHub settings → tokens). GitHub's secret scanning auto-revokes its own token formats found in public pushes, but verify rather than assume. Never "test whether it still works" from an unrelated context.
4. **Remove from the working tree** so it stops being redistributed, then decide about history: rewriting with `git filter-repo` removes it from your repository but not from forks, clones, caches or scrapers; rotation is what actually closes the hole.
5. **Prevent recurrence:** a pre-commit secret scanner (gitleaks), GitHub push protection, and never putting tokens in client-side code at all; a fine-grained read-only token in a server-side function if the feature is ever enabled.
6. **Document** the incident and the pending action for the owner.

IN THIS PROJECT

The token was removed from the first line of the Header component; the report and README record that rotation is pending. The EmailJS identifiers in `.env` are public by design and were not treated as secrets.

How to say it: "Identify, assume compromised, revoke and rotate, remove from source, understand that history rewriting does not recall copies, and add scanning so it cannot happen again."

Which headless-browser settings affect screenshot comparisons, and how did you control them?

SETTINGS

- **Font hinting and anti-aliasing:** `--font-render-hinting=none` gives consistent glyph rasterisation across runs; still platform-specific (Windows versus macOS text rendering differ).
- **Scrollbars:** `--hide-scrollbars` prevents a 17 px layout difference and scrollbar pixels.
- **Device scale factor:** 1 for speed and simpler diffs; a 2× run would catch sub-pixel issues at the cost of 4× pixels.
- **Viewport size and full-page capture:** full-page screenshots temporarily resize the viewport, which fires `resize` events (that is how the old header spacer re-measured mid-capture).
- **Color scheme and reduced motion emulation:** set explicitly.
- **GPU:** `--disable-gpu` for stable software rasterisation in headless.
- **Network and timing:** wait for idle and fonts; animations given time to finish.

IN THIS PROJECT

All of the above were fixed in the capture script. The one surprise was the full-page resize side effect, which I turned into a feature by issuing an explicit resize before capturing so both builds were measured in the same post-resize state.

How to say it: "Hinting, scrollbars, DPR, viewport, media emulation and GPU all change pixels; fix them all, and know that full-page capture itself fires a resize."

A tall section never reaches 20% visibility on a short viewport and never reveals. How would you fix the reveal design for the redesign?

ANALYSIS

With threshold 0.2 and an element five times taller than the viewport, the intersection ratio maxes out at 20% exactly at one scroll position and the callback may never fire; react-reveal had the same flaw, which is why I preserved it rather than fixed it.

FIX OPTIONS

- Use a `rootMargin` of, say, `-10% 0px` with threshold 0, so "reveal when the element enters the central 80% of the viewport" regardless of its height.
- Use multiple thresholds (`[0, 0.1, 0.2]`) and reveal on the first positive ratio.
- Reveal when either the ratio exceeds 0.2 *or* the visible height exceeds some fraction of the viewport (`entry.intersectionRect.height > 0.2 * viewportHeight`).
- Reveal child items individually rather than whole sections.

- Fall back to revealed if the observer never fires within a timeout, so content is never permanently hidden; and honour `prefers-reduced-motion` by skipping the animation entirely.

How to say it: "A fixed 20% threshold fails for tall elements; use rootMargin or a visible-height rule and never leave content hidden if the observer does not fire."

What did the pixelmatch ESM-only error teach you about dependency management in scripts?

LESSON

Major versions of utility libraries increasingly ship as ESM-only, which a CommonJS script cannot `require`; Node reports `ERR_REQUIRE_ESM`. Options: use dynamic `import()` inside CommonJS, write the script as ESM (`.mjs` or `"type": "module"`), or pin the last CommonJS-compatible major. For a throwaway verification harness I pinned version 5; for the project itself I made the package ESM so build scripts and the SSR bundle run natively. More broadly: pin tooling versions in scripts you want to re-run later, and read the changelog when a major bumps.

How to say it: "ESM-only majors break require(); either go ESM yourself or pin the compatible major. I did the first for the project and the second for the throwaway harness."

10.7 System design

System design interviews reward a repeatable shape: clarify the goal, state requirements, draw the architecture, name the trade-offs, then anchor with what you actually did. Each answer below follows that shape. Say the headings out loud; interviewers like hearing the structure.

Design the delivery architecture for a static site that must load fast for users anywhere in the world.

CLARIFY

How much content and how often does it change? Any personalisation? Budget for hosting? Target: a content site (portfolio, docs, marketing) with build-time content, no login, updates a few times a week, must be fast on mobile in regions far from the origin.

REQUIREMENTS

Functional: serve HTML and assets; deploy on push; rollback. Non-functional: LCP under 2.5 s at p75 worldwide on mobile; CLS 0; cheap; secure; cacheable; observable.

ARCHITECTURE

1. **Build.** Pre-render HTML (SSG). Emit content-hashed assets. Precompress text with Brotli and gzip at build time, or let the CDN compress.
2. **Edge.** Put everything behind a CDN with points of presence near users (Cloudflare, Fastly, CloudFront). HTTP/2 and HTTP/3 on. TLS 1.3.
3. **Caching policy.** Hashed assets: `Cache-Control: public, max-age=31536000, immutable` . HTML: `no-cache` (revalidate with ETag) or a short max-age plus `stale-while-revalidate` , so deploys propagate within seconds and users never see a stale HTML referencing a missing hash.
4. **Deploy atomically.** Upload new assets first, then switch HTML; keep old assets for a while so in-flight pages still resolve. Keep previous deploys addressable for instant rollback.
5. **Critical path discipline.** Fonts preloaded and self-hosted; one small CSS; deferred module scripts; LCP image preloaded with `fetchpriority="high"` ; everything else lazy. No third party before first paint.
6. **Security headers at the edge.** CSP, HSTS, X-Content-Type-Options, Referrer-Policy.
7. **Observability.** RUM for vitals, synthetic Lighthouse on each deploy, CDN logs for 404s and status codes.

TRADE-OFFS

Pre-render versus SSR: pre-render is cheaper and faster for static content; SSR only if personalised. Immutable caching requires hashing discipline (never reuse a name with new content). A CDN in front of GitHub Pages (via a custom domain and Cloudflare) can add Brotli and header control without moving the build.

IN THIS PROJECT

The build side is done: pre-rendered HTML, hashed assets, React in a stable chunk, no source maps, self-hosted fonts, deferred analytics. GitHub Pages provides the CDN but fixes compression to gzip and cache lifetime to ten minutes; the next step is a host or proxy that honours the caching policy.

Whiteboard summary: "Build static, hash everything, cache assets forever and HTML briefly, deploy atomically, serve from the edge with Brotli and HTTP/3, keep third parties off the critical path, measure in the field."

Design a performance budget and a CI gate so the site cannot regress.

CLARIFY

Who merges code and how often? What is the current baseline? Answer: a small team or a solo developer, pull requests to main, baseline is the numbers in this report.

REQUIREMENTS

Catch regressions before deploy; low false-positive rate so people trust it; fast enough to run per pull request; produce evidence (diff images, reports) a reviewer can read.

ARCHITECTURE

1. **Resource budgets** (deterministic, fast): critical JS ≤ 80 KB gzipped, CSS ≤ 10 KB, fonts ≤ 60 KB, first-view transfer ≤ 500 KB, no new third-party origins. Enforced with a bundle-size check (size-limit or a script over the build output) that compares against the base branch and fails on growth beyond a tolerance.
2. **Timing budgets** (lab, noisier): Lighthouse CI against a local static server with assertions: LCP ≤ 2.5 s mobile, TBT ≤ 150 ms, CLS = 0, performance ≥ 95; run three times and assert on the median.
3. **Visual budget**: the screenshot harness against stored baselines; fail on any differing pixels outside masked regions or any height change; upload diff images as CI artefacts. Intentional visual changes require a baseline update commit reviewed in the same pull request.
4. **Functional smoke**: the headless interaction script (carousels, form validation, tooltip, headroom) and a hydration console check.
5. **Reporting**: post a summary comment on the pull request with before/after numbers; store trends in a small dashboard so slow creep is visible.

TRADE-OFFS

Lab timing budgets flake; use medians and generous thresholds, and let resource budgets be the strict gate. Visual regression tests need deterministic rendering (fixed browser, fonts, flags) and masked dynamic regions. Budgets should be revisited when the design changes intentionally.

IN THIS PROJECT

All the pieces exist as scripts (capture, diff, functional, Lighthouse); wiring them into GitHub Actions with stored baselines is the follow-up. The budgets I would set are the numbers the site now meets with margin.

Whiteboard summary: "Strict deterministic budgets on bytes and pixels, softer median-based budgets on lab timings, functional smoke, and evidence posted on every pull request."

Design real-user monitoring for Core Web Vitals.

CLARIFY

Volume (tens or millions of page views), privacy constraints, where data should land (existing analytics or own storage), and what decisions it must support (deploy regressions, device segments).

REQUIREMENTS

Collect LCP, INP, CLS, FCP, TTFB per page view with attribution; segment by device, connection, country, page and build version; compute p75; alert on regressions; respect consent; negligible performance cost.

ARCHITECTURE

1. **Collection.** The `web-vitals` library (attribution build, about 2 KB) loaded after hydration. It reports each metric when final (on visibility change for CLS and INP), with attribution: LCP element selector and sub-parts, INP target and phases, CLS sources.
2. **Transport.** `navigator.sendBeacon` (or `fetch` with `keepalive`) on `visibilitychange` to hidden, so the beacon survives tab close. Include a build id, page path, device class, effective connection type, and a random session id, no personal data.
3. **Sink.** GA4 custom events for simplicity, or a small endpoint writing to a columnar store (BigQuery, ClickHouse) for flexible queries.
4. **Processing.** Daily and hourly p75 per metric per segment; join to deploy markers.
5. **Alerting.** Regression if p75 worsens beyond a threshold for a sustained window after a deploy; include attribution so the alert says which element or interaction regressed.
6. **Sampling and consent.** Sample at high traffic; gate collection behind consent where required; strip URLs of query strings.

TRADE-OFFS

GA4 is easy but limited for percentiles and attribution; an own pipeline costs engineering but answers better questions. Beacons on unload are lossy on some browsers; `visibilitychange` is the reliable hook.

IN THIS PROJECT

Not yet done; it is the single most important next step because every number I have is lab. The implementation is a dozen lines in the client entry sending through the existing gtag queue.

Whiteboard summary: "web-vitals attribution build, sendBeacon on hide, build id on every beacon, p75 by segment joined to deploys, alert with attribution."

Design an image pipeline for a content-heavy redesign.

CLARIFY

Source of images (repository, CMS, user uploads), how many, how often they change, and the design's breakpoints and densities.

REQUIREMENTS

Right-sized images per viewport and DPR; modern formats with fallbacks; zero layout shift; the LCP image prioritised; lazy loading elsewhere; reproducible transforms; sensible cache behaviour.

ARCHITECTURE

1. **Build-time transforms** for repository images: sharp generates AVIF, WebP and JPEG/PNG at a set of widths (e.g. 400, 800, 1200, 1600), content-hashed. For CMS or user images, an image CDN resizes on demand with URL parameters and caches at the edge.
2. **Markup.** `<picture>` with AVIF and WebP `<source>` elements and a fallback `<img>`; `srcset` with width descriptors and an accurate `sizes`; explicit `width` and `height` for aspect ratio.

3. **Priority.** LCP image: no lazy loading, `fetchpriority="high"`, optionally preloaded with `imagesrcset` / `imagesizes`. Everything below the fold: `loading="lazy"`, `decoding="async"`.
4. **Quality policy.** Lossy for photos (AVIF around quality 50–60, WebP 75–80); lossless or PNG for logos and screenshots with text; cap DPR at 2 for huge screens.
5. **Placeholders.** Optional blurred low-quality placeholder or dominant colour background to improve perceived speed without shift.
6. **Component.** One `Image` component that enforces all of the above so authors cannot forget dimensions or lazy rules.

TRADE-OFFS

AVIF encodes slowly (build time) and has weaker lossless; too many breakpoints inflate build output; on-demand CDN transforms cost money per transform but handle arbitrary sources.

IN THIS PROJECT

Images were converted losslessly to WebP (pixel parity) and not resized because the 512 px sources already match 250 px at 2×; lazy loading was applied only below the fold in fixed-height containers. The full pipeline above is what the redesign should adopt once lossy compression is acceptable.

Whiteboard summary: "Generate sizes and formats at build or at the edge, picture plus srcset plus sizes with dimensions, prioritise the LCP image, lazy-load the rest, enforce it in one component."

Design a web font strategy for a multilingual site with several weights.

CLARIFY

Which scripts (Latin, Devanagari, CJK), which weights and styles the design really needs, brand requirements, and whether a system-font fallback is acceptable on first visit.

REQUIREMENTS

Fast first paint with readable text, no layout shift on swap, minimal bytes per visitor (only the scripts and weights they need), consistent rendering, cacheable, maintainable.

ARCHITECTURE

1. **Self-host** from the same origin as the page.
2. **Variable fonts** where available: one file covers all weights (and often width), replacing four to six static files.
3. **Split by script with `unicode-range`**: Latin, Latin Extended, Cyrillic, Devanagari, CJK ranges as separate files declared with the same family; the browser downloads only the subsets its text needs.
4. **Preload** only the subset and weight used above the fold (usually Latin regular); let the rest load on demand.
5. **`font-display: swap`** with a metric-matched local fallback (`size-adjust`, `ascent-override`, `descent-override`, `line-gap-override`) so the swap produces no shift; or `optional` for purely decorative faces.
6. **Reproducible subsetting** in the repository (HarfBuzz via a script), with the original font files stored and the ranges explicit.
7. **Immutable caching** for the hashed font files.

TRADE-OFFS

More subsets mean more small requests (fine on HTTP/2); variable fonts are larger than a single static weight but smaller than three; `optional` is the most stable but shows fallback on cold first visits.

IN THIS PROJECT

Single-language, two families, one weight each (the design synthesises bold): a single Latin subset per family, preloaded, with the regeneration script kept in the repository. For a multilingual redesign, the steps above are the extension.

Whiteboard summary: "Self-host, variable fonts, unicode-range subsets, preload only what the fold needs, metric-matched fallbacks for shift-free swap, reproducible subsetting in the repo."

How would you govern third-party scripts on a larger site?

CLARIFY

Which third parties exist (analytics, tag managers, chat, A/B testing, embeds, ads), who owns each, and what the business needs from them.

REQUIREMENTS

No third party on the critical rendering path; bounded impact on INP; consent compliance; security (no arbitrary script injection); visibility of cost per vendor; a process to remove dead ones.

ARCHITECTURE

1. **Inventory** with owner, purpose, bytes, main-thread time (from RUM or Lighthouse third-party summary), and consent category.
2. **Loading tiers.** Essential: after first paint via `defer`. Analytics: after `load`, sampled if heavy. Marketing: after consent, idle time. Embeds (video, maps, chat): facades that render a static preview and load the real widget on click.
3. **Isolation.** Run tag managers in a Web Worker with Partytown where compatible; sandboxed iframes for untrusted widgets.
4. **Security.** CSP whitelisting vendor origins; SRI for versioned static vendor files; no inline vendor snippets without nonces.
5. **Budgets.** A third-party byte and blocking-time budget in CI; alerts from RUM attribution when a vendor's share grows.
6. **Lifecycle.** Quarterly review; remove anything without an owner or without recent use.

TRADE-OFFS

Deferring analytics loses a small fraction of very short sessions; facades change the first interaction with embeds; Partytown is not compatible with every vendor script.

IN THIS PROJECT

The inventory had seven third-party origins; four were removable, one was dead (a Universal Analytics tag still loading two years after the service shut down), one was a duplicate of a bundled library. The survivor, GA4, loads after `load`.

Whiteboard summary: "Inventory, tiers by necessity, facades for embeds, isolation in workers or iframes, CSP and SRI, budgets and alerts, and a review that deletes the dead ones."

The portfolio grows into a multi-page site with a blog and project pages. How do you evolve the architecture?

CLARIFY

How many pages, who writes content and in what format, how often it changes, whether comments or search are needed, and whether the current components should be reused.

REQUIREMENTS

Keep the current performance profile per page (pre-rendered HTML, minimal JS), add routing, content authoring in Markdown, SEO per page (titles, descriptions, Open Graph images, sitemap, RSS), fast builds, and a migration path for existing components.

ARCHITECTURE

1. **Framework.** Astro for a content-first site: zero JavaScript by default, React components as islands only where interactive (carousels, form), Markdown/MDX content collections with typed frontmatter, built-in image optimisation, sitemap and RSS integrations. Alternative: Next.js static export if server features (auth, API routes) are anticipated.
2. **Content.** Markdown files in the repository under version control; frontmatter for title, date, tags, cover image; a build step validates schema.
3. **Layout and tokens.** One base layout; design tokens as CSS variables; shared head component for SEO tags generated per page; Open Graph images generated at build.
4. **Data.** GitHub stats fetched at build time with caching, never from the client with a token.
5. **Performance guardrails.** The same budgets and visual regression harness per page template; islands hydrate on `client:visible` so below-the-fold interactivity costs nothing until seen.
6. **Deployment.** Static output to the same host; incremental builds if the site becomes large; ISR only if content changes faster than builds.

TRADE-OFFS

Astro's island model changes how components are composed (props must be serialisable across the island boundary); Next brings a heavier runtime. Markdown is great for developers, less so for non-technical authors who may want a headless CMS.

IN THIS PROJECT

The components are already plain React 18 function components with no framework APIs, content is in a single data module, and fonts, icons and animations have documented pipelines, so porting into Astro islands is mechanical.

Whiteboard summary: "Content-first framework with islands, Markdown collections, build-time data, per-page SEO generated, same budgets and visual tests per template."

Compare GitHub Pages, Cloudflare Pages, Netlify and Vercel for this site and recommend one.

CRITERIA

Compression (Brotli), cache header control, edge network reach and TTFB, redirects and headers configuration, functions for a form backend, build integration, cost, custom domain and TLS, preview deploys.

Host	Strengths	Limits
GitHub Pages	Free, zero setup, repository-native, Fastly CDN	gzip only; fixed `max-age=600`; no headers, redirects or functions; measured ~0.8 s mobile TTFB; sub-path URLs for project sites
Cloudflare Pages	Free tier, very wide edge network, Brotli, HTTP/3, `_headers` and `_redirects` files, Pages Functions (Workers) for a form endpoint, preview deploys, analytics	Build minutes limits on free tier; Workers model to learn for functions
Netlify	Mature redirects/headers, Netlify Forms (could replace EmailJS without code), functions, preview deploys, plugins	Bandwidth and build minute limits on free tier; some features paid
Vercel	Best Next.js integration, edge functions, image optimisation, analytics with vitals	Optimised around Next; free tier limits; less compelling for a plain static site

RECOMMENDATION

Cloudflare Pages for this site: free, Brotli and HTTP/3 by default, immutable caching via a two-line `_headers` file, a Pages Function later for the contact form, and preview URLs per pull request for the visual regression job. Netlify is a close second if forms without code are attractive. Vercel if the redesign adopts Next.js. GitHub Pages remains fine as long as its caching and compression limits are accepted.

Whiteboard summary: "Pages is free but fixed; Cloudflare adds Brotli, HTTP/3, headers and functions for free; Netlify has forms; Vercel if Next. I would pick Cloudflare Pages."

Design an animation system for the redesign that stays smooth and cheap.

REQUIREMENTS

Consistent motion language; compositor-only animations; scroll-triggered entrances that work with pre-rendered HTML; respect for reduced motion; bounded cost for rich animations (Lottie); measurable.

ARCHITECTURE

1. **Tokens.** Durations (fast 150 ms, base 300 ms, slow 600 ms, reveal 1000 ms) and easings as CSS variables; keyframes for enter and exit defined once.
2. **Reveal utility.** One IntersectionObserver-based component or hook (the current Reveal), with `appear` for above-the-fold content so pre-rendered pages animate from first paint, a visible-height rule instead of a fixed 20% threshold for tall sections, and a timeout fallback so content is never stuck hidden.
3. **Reduced motion.** A global media query that disables non-essential motion and pauses Lottie on a representative frame; the Reveal utility renders the final state immediately.
4. **Lottie policy.** Lazy-load the player, pause when off-screen, prefer dotLottie or the canvas renderer for heavy files, cap animation JSON per page, prefer simple CSS for decorative motion.
5. **Only transform and opacity** in transitions; no `transition: all`; `will-change` applied just-in-time.
6. **Measurement.** Dropped-frame checks in the Performance panel, Lighthouse's non-composited animations audit in CI, and INP in the field.

IN THIS PROJECT

The Reveal component is the seed of the utility; the Lottie component already implements lazy loading and pausing. Reduced motion and the transition-all cleanup are the first additions.

Whiteboard summary: "Motion tokens, one reveal utility with appear and reduced-motion support, Lottie lazy and paused, compositor-only properties, audited in CI."

Design the component and styling architecture for the UI/UX revamp so that performance survives the redesign.

REQUIREMENTS

Themeable without flashes or hydration risk; consistent spacing and typography; no layout shift by construction; images and icons handled by components that enforce the rules; testable visually; small CSS.

ARCHITECTURE

1. **Design tokens as CSS custom properties** (colour, type scale, spacing, radius, shadow, motion) with light and dark values switched by a root attribute set by an inline script before paint. Components never read the theme in JavaScript.
2. **Layout primitives** (Stack, Cluster, Grid, Section, Container) that own spacing via `gap`, so component CSS has no margins to collide.
3. **Media components**: an Image component that requires dimensions and applies format, size and lazy rules; an Icon component backed by an SVG sprite; a Lottie component with the lazy/pause behaviour.
4. **Styling approach**: CSS Modules or vanilla CSS with cascade layers (`@layer reset, tokens, layout, components, utilities`) so order dependencies like the StarWars.css incident cannot happen; no runtime CSS-in-JS.
5. **Content as data**: one typed content module (or content collections) feeding templates.
6. **Testing**: Storybook or a simple harness per component with visual snapshots in both themes; the page-level screenshot harness per template; budgets in CI.
7. **Accessibility baked in**: semantic landmarks, heading order, focus styles, alt text required by the Image component, reduced motion respected.

TRADE-OFFS

Cascade layers and primitives have a learning curve; CSS Modules add build plumbing; strict components slow down one-off experiments but prevent regressions.

Whiteboard summary: "Tokens in CSS variables switched before paint, layout primitives with gap, enforcing media components, layered CSS, content as data, visual tests and budgets from day one."

Design observability for a static site.

REQUIREMENTS

Know when the site is broken, slow, or attacked; attribute problems to deploys; keep user privacy; near-zero runtime cost.

ARCHITECTURE

1. **Front-end errors.** An error monitoring SDK (Sentry or similar) capturing exceptions and unhandled rejections with release tags; source maps uploaded to the service at build time and never deployed publicly.
2. **Vitals.** RUM as designed above, joined to releases.

3. **Synthetic.** Lighthouse on every deploy against the production URL; uptime checks from several regions.
4. **Security signals.** CSP in report-only mode first, collecting violations (unexpected scripts, injected content), then enforce.
5. **Edge logs.** Status code distribution and 404 lists from the CDN to catch broken links, missing hashed assets after a bad deploy, and bot abuse.
6. **Deploy markers** in every dashboard so regressions map to commits.

IN THIS PROJECT

None of this exists yet beyond analytics; the pre-render step's assertion (markup must be non-trivial) is the only build-time guard. Error monitoring and RUM are the first two additions.

Whiteboard summary: "Errors with releases and private source maps, RUM joined to deploys, synthetic checks per deploy, CSP reports, CDN logs for 404s."

How should the site degrade when JavaScript fails, a third party is down, or the user is offline?

PRINCIPLES

Content first, enhancements second; every external dependency has a fallback; failures are visible to the user only where they matter.

DESIGN

- **JavaScript fails or is slow.** Pre-rendered HTML means all content is readable; anchors work; carousels show their first page; the Lottie boxes show blank placeholders (an acceptable degradation, or a static first-frame image in the redesign). The contact form should have a `mailto:` fallback link so the address is reachable without JavaScript.
- **EmailJS is down.** The form shows a clear error with the email address and a retry; nothing else is affected.
- **Analytics is down.** Invisible to the user by construction (loaded after `load`, failures ignored).
- **Fonts fail.** `font-display: swap` keeps text visible in the fallback font.
- **Offline.** Optional service worker precaching the page shell with a careful update flow (Workbox, skip-waiting on user confirmation); for a portfolio the value is modest and the update problem real, so I would only add it with a visible "new version available" prompt.
- **Partial deploys.** Atomic deploys and retained old assets so an HTML referencing a hash never 404s.

Whiteboard summary: "Pre-rendered content works without JS, every third party fails silently or with a clear local fallback, offline only with a deliberate update flow."

Design a contact form backend to replace EmailJS, with spam protection and rate limiting.

CLARIFY

Expected volume (a few messages a week), privacy requirements, hosting (static site with optional serverless functions), and acceptable friction (CAPTCHA or not).

ARCHITECTURE

1. **Endpoint.** A serverless function (Cloudflare Pages Function, Netlify Function) at `POST /api/contact`; the client sends JSON with name, email, message.
2. **Validation.** Server-side schema validation (lengths, email format), reject on failure with a clear message; the client validates too for fast feedback.
3. **Spam defences, layered.** A honeypot field hidden from humans; a minimum time-to-submit check (bots submit in milliseconds); a privacy-friendly challenge (Cloudflare Turnstile or hCaptcha) verified server-side; origin and referer checks; rate limiting per IP and per email via a KV store (e.g. 5 per hour).
4. **Delivery.** A transactional email API (Resend, SendGrid, SES) with the API key as a server secret, never in the client; or store messages in a database and notify.
5. **Abuse and privacy.** Strip and limit content; log minimally; no third-party scripts needed on the page except the challenge widget, loaded only when the form is focused.
6. **User experience.** Optimistic "Sending…" state, success and error messages, no page reload, accessible status announcements.

TRADE-OFFS

A challenge adds friction; honeypot plus timing plus rate limiting catch most spam for low-volume sites and can be tried first. EmailJS is simpler but exposes a public key that can be abused up to its quota.

IN THIS PROJECT

EmailJS was kept for parity of behaviour; the public identifiers were already public. The endpoint design above is the upgrade path once the site moves to a host with functions.

Whiteboard summary: "Serverless POST endpoint, server-side validation, honeypot plus timing plus challenge plus rate limit, email API with a server secret, no reload."

Design the deploy and cache-invalidation strategy with safe rollbacks.

REQUIREMENTS

Users never see a page referencing a missing asset; new deploys visible within seconds; rollback in one step; no cache purges needed in the common case.

ARCHITECTURE

1. **Content-hashed assets** with immutable caching; a changed file is a new URL, so no invalidation is ever needed for assets.
2. **HTML with short or no caching** (`no-cache` plus ETag, or `max-age=60, stale-while-revalidate=600`) so the new HTML is picked up quickly and references the new hashes.
3. **Atomic, additive deploys.** Upload new assets, then swap HTML. Keep old assets live for at least a day so tabs opened before the deploy can still lazy-load their chunks (otherwise dynamic imports fail with chunk-load errors).
4. **Immutable deploy snapshots** (every deploy has its own URL on hosts like Cloudflare Pages and Netlify); rollback is pointing production at a previous snapshot.
5. **Chunk-load error handling** in the client: on a failed dynamic import, reload once to pick up the new HTML.

6. **Versioning signal.** A build id in the HTML and in RUM beacons so you can see which version users are on.

IN THIS PROJECT

Hashing, a stable React chunk and a short-lived HTML name are in place; GitHub Pages' fixed ten-minute policy approximates the HTML rule but also applies it to assets. The `gh-pages` branch history provides crude rollback by reverting the branch.

Whiteboard summary: "Hash assets and cache forever, keep HTML fresh, deploy additively and atomically, keep old chunks alive, roll back by re-pointing to a snapshot."

How would you safely test whether a performance change actually helps real users?

REQUIREMENTS

Compare two variants under the same traffic; measure Core Web Vitals and business metrics per variant; detect harm quickly; avoid biasing the measurement.

ARCHITECTURE

1. **Assignment at the edge** (a Worker or edge middleware) by a sticky cookie or hash of a session id, serving variant A or B HTML so the client does not need JavaScript to pick a variant (client-side switching adds flicker and delay that pollute the measurement).
2. **Tagging.** Every RUM beacon and analytics event carries the variant id.
3. **Metrics.** p75 LCP, INP, CLS per variant plus conversion or engagement metrics; statistical test on the distributions, not the means.
4. **Guardrails.** Automatic rollback if error rate or p75 LCP in the new variant exceeds a threshold.
5. **Duration.** Long enough to cover weekly patterns and enough samples per device class.

TRADE-OFFS

Edge assignment needs a programmable edge; server-side variant generation doubles build outputs; with low traffic (like a portfolio) you will not reach significance and should rely on lab plus qualitative checks instead.

Whiteboard summary: "Assign at the edge with a sticky cookie, tag every beacon, compare p75s per variant with guardrails, and accept that low traffic cannot prove small effects."

Design adaptive loading for low-end devices and slow networks.

IDEA

Serve less to people who can afford less: fewer bytes, fewer animations, smaller images, when the device or connection is weak, without punishing everyone.

SIGNALS

`navigator.connection.effectiveType` (2g/3g/4g) and `saveData`; the `Save-Data` request header (server-side); `navigator.deviceMemory` and `hardwareConcurrency`; `prefers-reduced-motion` and `prefers-reduced-data` media queries; viewport size. All are hints and some are Chromium-only, so always have a sane default.

DESIGN

- Serve a static image instead of a Lottie animation when `saveData` is on or the effective connection is 2g/3g; skip autoplay video; smaller image breakpoints.
- Defer non-critical scripts further and reduce third parties on slow connections.
- Honour `prefers-reduced-motion` and `prefers-reduced-data` in CSS so no JavaScript is needed for the basic decisions.
- Keep the default path fast enough that adaptive paths are refinements, not rescues.
- Measure by segment in RUM to see whether the adaptive path helps.

IN THIS PROJECT

The default path was made light for everyone; the adaptive additions would be a static first frame for Lottie under `saveData` or reduced motion, which is also an accessibility win.

Whiteboard summary: "Read connection, memory and user-preference hints, degrade rich media to static alternatives, keep the default path fast, and verify by segment."

What are the performance implications of internationalising this site?

CONSIDERATIONS

- **Fonts.** Each script needs its own subset; CJK fonts are megabytes unless subset or served by `unicode-range` slices; consider system fonts for non-Latin scripts.
- **Text expansion.** German or Finnish strings run 30% longer; fixed-width layouts wrap or overflow; test visual regression per locale.
- **Direction.** RTL locales need logical CSS properties (`margin-inline-start`) and mirrored icons; the carousel arrows flip.
- **Bundles.** Load only the active locale's strings (per-locale chunk or static page per locale); avoid shipping all translations in one bundle.
- **Pre-rendering.** Generate one static page per locale with `hreflang` links and locale-specific Open Graph tags.
- **Dates and numbers.** `Intl` APIs are built in and free; avoid heavy locale data libraries.

Whiteboard summary: "Per-script font subsets, logical CSS for RTL, per-locale pages and string chunks, visual tests per locale, native Intl."

Design a lazy-loading strategy for a long content page with images, embeds, animations and third parties, with zero layout shift.

PRINCIPLES

Everything below the fold is deferred; everything that is deferred has a placeholder with exactly the final size; nothing deferred is needed for the LCP element.

DESIGN

1. **Classify the fold.** Above: eager, prioritised. Below: lazy.
2. **Images:** dimensions always; `loading="lazy"` ; a dominant-colour background as placeholder.

3. **Embeds (video, maps, tweets):** facades with the final aspect ratio; load the real iframe on click or when near the viewport.
4. **Animations:** the Lottie pattern from this project: inline placeholder with final geometry, load player and data on proximity, swap synchronously, pause when hidden.
5. **Components with heavy code:** dynamic import on proximity (`rootMargin`) rather than on visibility so the code arrives before the user does.
6. **Third parties:** after `load` or on interaction; never in placeholders that change size.
7. **Fonts:** preloaded; metric-matched fallbacks so late fonts do not shift text.
8. **Verification:** CLS in lab and field; a screenshot before and after all lazy content has loaded must have identical geometry.

Whiteboard summary: "Lazy below the fold, exact-size placeholders for everything lazy, load on proximity not visibility, swap without an empty frame, verify geometry before and after."

Design the GitHub pinned-repositories section properly, without shipping a token to the browser.

PROBLEM

The GitHub GraphQL API requires a token; anything in a client bundle is public; unauthenticated REST has low rate limits and no pinned-items query.

OPTIONS

1. **Build-time fetch.** The build runs with a server-side token (CI secret), fetches pinned repositories and profile, writes them into the content as JSON, and pre-renders the cards. Zero client cost, zero secrets exposed, data as fresh as the last build (schedule a nightly rebuild). Best fit for a portfolio.
2. **Serverless proxy.** A function with the token calls GitHub, caches the response at the edge for an hour, and serves a tiny JSON to the client. Fresh data, small runtime cost, token stays server-side.
3. **Unauthenticated REST** for public data (stars, description) without pinned ordering; rate-limited per IP (60/hour), risky under load.

DESIGN CHOICE

Option 1 with a nightly scheduled rebuild; the cards render in the pre-rendered HTML with no client fetch, no loading state and no failure mode; the lazy card components and Suspense fallback become unnecessary.

Whiteboard summary: "Fetch at build time with a CI secret and pre-render the cards; schedule rebuilds for freshness; never put a token in the client."

10.8 Behavioural questions and cross-examination

Behavioural answers work best in Situation, Task, Action, Result form with real specifics. Cross-examination questions are the sceptical follow-ups a strong interviewer asks after a polished story. The right response is almost always a direct, honest answer followed by what you would do about it.

BEHAVIOURAL

Tell me about a time you improved the performance of something significantly. (STAR)

Situation. My portfolio was a 2020 template-based React site. It looked fine, but a Lighthouse run showed a mobile score of 53 and an LCP of 8 seconds; the page was blank for five seconds on a simulated phone.

Task. Make it fast end to end, modernise the stack so a redesign could follow, and change nothing a visitor could see, because the redesign was a separate decision.

Action. I audited first: Lighthouse's LCP breakdown showed 7.1 seconds of render delay, meaning the content waited for JavaScript; source-map attribution showed 41% of the vendor bundle was an animation player and 20% a GraphQL client for a disabled section; four third-party resources were render-blocking. I ranked fixes by impact: pre-render HTML at build time, remove the blocking third parties, make the animations lazy, delete dead dependencies, subset and self-host fonts, fix images and analytics, move to Vite and React 18. I preserved the old build and wrote a headless-Chrome harness that screenshots both builds at six viewports and two colour schemes, masks the animations and diffs pixels, plus a functional test for every interaction.

Result. Mobile Lighthouse 63 → 98–99 under identical conditions, LCP 5.5 → 2.1 s, transfer 1,136 → 413 KB, 1,648 → 101 packages, zero differing pixels outside animations. The harness also caught two real regressions during the work and one pre-existing bug, which I fixed before shipping.

What was the hardest part, and how did you handle it?

Proving “nothing changed” rather than making it fast. Speed came from well-understood levers. Parity required building a measurement system I could trust and then believing it over my intuition. Twice the diff said my build was different when I was sure it was identical: once because a stylesheet I had deleted as a duplicate was actually overriding a media query (185 px of layout change on tablets), and once because the old site's header spacer depended on a font-loading race (12 px). In both cases I did not argue with the tool; I wrote a smaller probe to find where the difference started, found the cause, and either restored the behaviour or documented why the new behaviour was correct. The lesson I carry is that a verification harness is only useful if you are willing to be wrong in front of it.

How did you prioritise what to do first?

By measured impact on the user-facing metric, not by what was most interesting. The LCP breakdown said render delay was 7.1 seconds, so pre-rendering came first; Lighthouse's render-blocking audit said third-party CSS cost 4.3 seconds, so that was second; the bundle attribution said the Lottie player and JSON were the biggest bytes, so lazy loading was third. Dependency cleanup, fonts, images and analytics

followed because each was smaller individually. Toolchain modernisation was both a prerequisite (the SSR build needed Vite) and a long-term investment. I wrote the order down before starting and explained the reasoning in the audit so anyone could challenge it.

Describe a decision where you had to balance competing constraints.

The header spacer. Strict pixel parity said keep the old 78 px height; correctness said the header was really 90 px and the old value came from measuring before the logo font loaded. Keeping 78 would have meant deliberately reproducing a race whose result also varied by operating system and fixed itself on window resize. I chose correctness, but I did three things to respect the constraint: I proved the old value was a race with a probe through the exact load sequence and a check against production; I normalised my baseline capture so the comparison still isolated every other difference; and I documented it as the single intentional deviation with a one-line CSS override if anyone disagreed. That is how I try to handle constraint conflicts: pick, prove, make it reversible, and write it down.

Tell me about a bug you found that nobody had noticed.

The contact form reloaded the page when the email was invalid. The code only called `preventDefault()` on the valid path, so an invalid address triggered the browser's native submit and a reload with the fields in the URL; the “Invalid Email!” state flashed for a few milliseconds. It had been like that since the template was written. My functional test caught it indirectly: several unrelated assertions failed at once (the button said “Send”, every revealed section was hidden again), which is the signature of a page reload. I fixed it, re-ran the test, and added it to the report as a behaviour change, because it was outside the pixel brief and I wanted the owner to know.

How do you make sure a large refactor does not break things?

Three layers. First, preserve the old artefact so you always have a ground truth: I copied the old build before the first new build. Second, automate the comparison on the dimensions that matter: pixels (screenshot diffs with masked dynamic regions), geometry (landmark bounding boxes and page heights), behaviour (scripted interactions with assertions), and errors (console capture after hydration). Third, measure the goal under equal conditions (Lighthouse against both builds on the same server). Then make the checks cheap to re-run so every change is checked, not just the last one. I also keep a list of intentional deviations so “different” can be classified as “expected” or “regression” rather than argued about.

What did you learn that you did not know before?

Three concrete things. That browsers fetch a web font only when rendered text needs it, which makes any mount-time measurement of text layout unreliable. That an inline SVG adds descender space to its container, so a placeholder must be an inline element with the same sizing, not a CSS aspect-ratio box. And that “duplicate” CSS can be load-bearing because media queries do not add specificity, so order decides. More generally: a pixel diff tells you that something changed; a geometry dump tells you where; you need both.

How did you communicate the work and its risks?

With a written report that separates what was measured from what was inferred, lists every change with its reason, states the two intentional behaviour changes explicitly, lists the quirks preserved on purpose with their one-line removals, and keeps a “what I did not verify” list (field data, the GitHub section with a real token). In a team setting I would have shipped the audit as a document first, agreed the order and the parity rule, and posted the diff images with each pull request so reviewers see evidence rather than claims.

How would you scope this work for a team sprint?

As four tickets with a dependency order. (1) Verification harness and baseline capture, half a day; nothing merges without it. (2) Toolchain and pre-render, one day, including hydration checks. (3) Critical path: third parties, fonts, icons, analytics, half a day. (4) Lazy media and dead-code removal: Lottie, ColorThief, Apollo, images, half a day. Plus a review buffer for the discrepancies the harness will surface. Roughly three focused days for one engineer, with the harness paying for itself on day two.

If you had to onboard a teammate to maintain this, what would you tell them in ten minutes?

Content lives in one data file; images are imported at its top. Fonts are subset WOFF2 files regenerated by one script, so changing the logo text or adding an icon means editing that script and the icon stylesheet. Animations load lazily through one component; keep the placeholder logic. The reveal component mirrors react-reveal's props. Two visual quirks are preserved on purpose and documented with their removals. Run the capture-and-diff harness before merging any CSS change, and keep the console clean after hydration. Deploy is one command; the host sets caching you cannot change.

CROSS-EXAMINATION

“A Lighthouse score of 98 is a lab number. Did real users actually get faster?”

Honest answer: I have strong lab evidence and no field evidence yet. The lab comparison is fair because both builds were measured on the same machine behind the same server with the same Lighthouse version, so the difference is the code. The mechanisms are not speculative: the HTML paints before any JavaScript, 700 KB fewer bytes travel, and six fewer origins are contacted on the critical path. I expect the field to follow, but I would not claim it until the web-vitals library reports p75 LCP from real visitors, which is my next step and a dozen lines of code.

“Why not just use Next.js or Astro?”

Because the brief was zero visible change on a one-page site with build-time content. A framework migration changes file conventions, adds a runtime and can alter markup in small ways a pixel diff flags; a thirty-line pre-render using React's own `renderToString` and Vite's SSR build achieved the same first-paint benefit with far less surface area and risk. The components are now plain React 18 function components, so moving to Astro or Next for the redesign is cheap if routing or server features justify it. I would make that decision when there is a second page.

“You promised pixel parity and then changed the header by 12 pixels. Isn't that a contradiction?”

It would be if the old height were a design. It was the outcome of a race: react-headroom measured the header before the logo font loaded, so the spacer was 78 px while the header was 90 px, the overlap differed by operating system because fallback fonts differ, and any window resize corrected it. A state that changes when you resize the window is not a stable pixel target. I kept the deterministic correct value, proved the race with a probe and a production check, neutralised it in the baseline so every other pixel could still be compared, and documented it as the one intentional deviation with a one-line override. I would rather defend a documented correction than silently reproduce a bug.

“Why keep two obvious visual bugs, the stray slashes and the half-size inputs?”

Because the owner's instruction was that pixels must not change and a redesign was already planned. Fixing them would have been scope creep that also invalidates the parity test as evidence. I documented both with their one-line removals so the redesign removes them deliberately. If I were the owner I would fix them; as the engineer executing a stated constraint, I flag rather than override.

“Pinning lottie-web to a version from 2020 looks like technical debt.”

It is a parity decision with a known exit. The animation files use expressions and effects, and a newer player could render edge cases differently; pinning the version already in production guarantees frame-identical output. The player only processes trusted files from the repository, so the security exposure is small. Upgrading needs a frame-by-frame comparison, which is a bounded task I would schedule with the redesign, when visual changes are expected anyway.

“You removed Apollo. Did you test the GitHub section with a real token?”

No, and I say so in the report. The section is disabled in configuration and was already failing in production with an undefined token, so there was no working behaviour to preserve. I replaced the client with a small fetch wrapper mirroring the two queries and the error handling, kept the lazy card components, and documented that enabling the section requires a real token and a test. I would rather state the gap than imply coverage I do not have. In a proper design the data would be fetched at build time with a CI secret anyway.

“What if a visitor needs a glyph outside your font subset?”

The page's text is fixed, English, with a handful of punctuation characters, and the subset covers Basic Latin, Latin-1, Latin Extended-A, general punctuation, currency and arrows, so every character on the page is covered with margin. If the content changes to another script, the fallback font renders those characters and adding the range to the regeneration script is one line. For user-generated content I would split by `unicode-range` instead of guessing.

“Why not inline the critical CSS?”

The entire stylesheet is 7 KB gzipped, same-origin, discovered in the head, over the same HTTP/2 connection as the HTML. Inlining saves one round trip on a cold visit but makes the CSS uncacheable across visits and grows the HTML. With a host that caches assets immutably the external file is the better trade; on GitHub Pages with ten-minute caching the difference is small either way. It is on my follow-up list as a measured experiment, not a default.

“Loading analytics after the load event loses data.”

Only sessions that end before `load`, which on the new site is about a second on mobile. The `config` call is queued inline so the page view fires once the script arrives. For a portfolio this is clearly the right trade; on a site where early abandonment is the metric you care about, you would load earlier or send a minimal first beacon yourself.

“Lossless WebP is a half measure. Why not lossy or AVIF?”

Because lossy changes pixels and the brief forbade it. Lossless WebP halved the bytes with provably identical visible pixels. Lossy WebP or AVIF would cut another 60% and is the right choice for the redesign; I listed it with the expected saving.

“Your dependency count dropped from 1,648 to 101. Isn't that mostly removing dev tooling, not a user-facing win?”

Partly, yes: react-scripts brought jest, eslint, workbox and dozens of loaders. The user-facing part is the runtime dependencies: 17 → 7, with Apollo, graphql, react-reveal, react-lottie, react-easy-emoji, react-twitter-embed, dotenv and gsap gone from the bundle. The dev-tooling part still matters for maintenance: install time, security advisories, and the ability to upgrade React, which react-reveal had blocked.

“What broke while you were doing this?”

Four things, all caught by the harness rather than by eye. Deleting a stylesheet that held a cascade dependency changed the tablet layout by 185 px. The header spacer race first looked like a regression in my build. The contact form's native submit surfaced when my test expected an error state. And two tooling issues of my own: a path bug in the local server that made my first baseline an error page, and an ESM-only dependency my CommonJS diff script could not load. Each one is in the report with how it was found and fixed.

“What would you do differently next time?”

Build the screenshot harness before touching any code instead of after the first rebuild, so every change is checked incrementally. Add real-user monitoring on day one so field data accumulates while the work happens. Push back earlier on strict parity for the two visible bugs, because fixing them is cheap and clearly right, and get that decision from the owner up front rather than preserving them by default.

> **"How much of this was you versus tools?"**
>
> Tools measured; I decided. Lighthouse and source maps told me where time and bytes went; I chose the order, wrote the pre-render pipeline, the Reveal replacement, the Lottie component and the verification harness, read react-reveal's and react-lottie's source to extract exact behaviour, and diagnosed the three discrepancies with custom probes. The point of a verification harness is that it does not care whose intuition is wrong; twice it was mine.

> **"Explain hydration to a junior developer in one minute."**
>
> Imagine the server sends a photograph of a fully furnished room: you can see everything instantly but you cannot open a drawer. Hydration is React walking into the room and connecting handles to the drawers that are already there, without rebuilding the furniture. It only works if the furniture in the photo is exactly what React expected to build; if a chair is in a different place, React gets confused, tears that part out and rebuilds it, which is slow and flickers. So the rule is: render the same thing on the server and on the client's first pass, and do anything browser-specific afterwards.

10.9 Rapid-fire revision glossary

One to three sentences each, for the night before.

LCP	Render time of the largest text block or image in the viewport. Good ≤ 2.5 s. Sub-parts: TTFB, load delay, load time, render delay. Mine was 8 s, almost all render delay.
INP	Worst interaction latency across the visit (input delay + processing + presentation). Good ≤ 200 ms. Field-only; TBT is the lab proxy.
CLS	Sum of unexpected shift scores (impact × distance) in the worst 5-second window. Good ≤ 0.1. Prevent with reserved space and metric-matched fonts.
FCP	First paint of any content. Mine went 3.8 → 1.4 s on mobile.
TBT	Sum of long-task time over 50 ms between FCP and TTI. 30% of the Lighthouse score. Mine went 330 → 60–90 ms.
Speed Index	How quickly the visible area fills, from a filmstrip.
TTI	First 5-second quiet window on the main thread after FCP.
Long task	Main-thread task over 50 ms; blocks input.
Critical rendering path	DOM → CSSOM → render tree → layout → paint → composite. Blocked by stylesheets and sync scripts.
Render-blocking	Must finish before first paint: head stylesheets, @imports, sync scripts.
Preload scanner	A lookahead parser that starts downloads for resources found in raw HTML before the DOM exists.
defer / async / module	defer: after parse, in order. async: on arrival, any order. module: deferred by default, strict, ESM.
preload	High-priority fetch for the current page; needs `as`; fonts need `crossorigin`.
modulepreload	Preload plus compile for an ES module and its graph.

preconnect / dns-prefetch	Warm a connection / resolve a name for another origin.
font-display	block (hide ~3 s), swap (show fallback, swap), fallback, optional (no swap).
Font lazy loading	A web font is fetched only when rendered text needs it. Root of the header spacer race.
size-adjust / ascent-override	Make a fallback font occupy the same space as the web font so swaps do not shift layout.
WOFF2	Lossless font container: Brotli plus glyph-table transform.
Subsetting	Removing unused glyphs with glyph closure; outlines of kept glyphs unchanged. Montserrat 246 → 38 KB.
unicode-range	Declares which code points a face covers so the browser downloads only needed subsets.
gzip vs Brotli	Both lossless; Brotli 15–25% smaller on text. Pages serves gzip only.
Cache-Control	max-age, immutable, no-cache (revalidate), no-store, stale-while-revalidate. Hash assets and cache forever; keep HTML fresh.
Cache partitioning	Since Chrome 86 the cache is per top-level site; shared CDN URLs no longer save bytes.
HTTP/2 / HTTP/3	Multiplexing over one connection / QUIC over UDP with per-stream loss recovery.
CORS mode for fonts	Font fetches are always CORS; preloads must match or they double-download.
CSR / SSR / SSG / ISR	Browser-rendered / per-request server-rendered / build-time rendered / build-time with regeneration.
Pre-render	My SSG: client build, SSR build, renderToString once, inject into index.html.
Hydration	React adopting server DOM; requires identical first render; mismatches re-render from the nearest Suspense boundary.
onRecoverableError	hydrateRoot option that reports hydration problems; I log it in production too.
useLayoutEffect	Runs after DOM mutation, before paint; used to insert the Lottie SVG without an empty frame.
cloneElement	Copy an element with extra props; how Reveal adds class, style and ref without a wrapper div.
Keys	Identity for list items across renders; index keys reuse instances for different data.
React 18 root API	createRoot / hydrateRoot; automatic batching; streaming SSR; selective hydration.
IntersectionObserver	Async visibility callbacks on threshold crossings plus one initial callback; rootMargin expands the root.
Passive listener	Promise not to preventDefault so scrolling need not wait for the handler.
rAF throttling	Do at most one read/write per frame in requestAnimationFrame.
Compositor-only properties	transform and opacity: animate without layout or paint.
will-change	Pre-promote to a layer; apply just before animating, remove after.
Cascade	Origin, importance, specificity, then source order. Media queries add no specificity.
Inline SVG descender gap	Inline elements sit on the baseline; the line box adds descender space below. 4–5 px here.

display: none vs visibility vs opacity	No box / box but unpainted and non-interactive / painted transparent and interactive; only display: none never intersects.
content-visibility: auto	Skip rendering off-screen content; needs contain-intrinsic-size.
Tree shaking	Dropping unused ESM exports; CSS imports are side effects and stay.
Source maps	Map minified positions to sources; use for attribution, never deploy publicly.
Data URI inlining	Base64 is +33% and compresses badly; inline only tiny assets, never fonts that are preloaded.
manualChunks	Force modules into a named chunk; React in its own chunk for long-term caching.
?url import	Vite: get the hashed URL of a file instead of bundling it; used for Lottie JSON.
ssr.noExternal	Bundle a dependency into the SSR build to avoid Node's CJS default-import problem.
Lottie builds	full (all renderers + expressions), lottie_svg (SVG + expressions), lottie_light (SVG, no expressions). Expressions forced lottie_svg.
ColorThief	Canvas read-back plus median-cut quantisation; needs non-tainted canvas (CORS).
Lossless WebP	Identical visible pixels; differences only under alpha 0 (libwebp exact flag).
srcset / sizes	Browser picks an image candidate by display width × DPR.
SRI / CSP	Hash-pin third-party files / whitelist origins for scripts, styles, fonts, connections.
Service worker update flow	New worker waits until all tabs close unless skipWaiting; the reason I did not add one.
pixelmatch	YIQ perceptual pixel diff with threshold and optional anti-aliasing tolerance.
Lighthouse weights	TBT 30, LCP 25, CLS 25, FCP 10, SI 10; log-normal scoring; simulated slow 4G and 4× CPU on mobile.
Lab vs field	Reproducible tool runs vs real users (CrUX, RUM). I have lab; RUM is next.
web-vitals library	2 KB script reporting LCP, INP, CLS with attribution; send via sendBeacon on visibilitychange.
GitHub Pages	Fastly-backed static hosting; gzip only; max-age=600; Jekyll unless .nojekyll; sub-path for project sites.
Atomic deploy	Upload assets, then switch HTML; keep old assets so open tabs can still lazy-load.
Numbers	53 → 98–99 mobile; 8.0 → 2.1 s LCP; 258 → 69 KB critical JS; 1,136 → 413 KB; 7 → 1 origins; 0 px diff.

11. Appendices

A. File map

```
index.html                      Vite entry; meta, font preloads, deferred GA4, module script
vite.config.mjs                 base path, asset inlining policy, React chunk, SSR noExternal
scripts/prerender.mjs           injects renderToString output into build/index.html
scripts/subset-fonts.mjs        regenerates WOFF2 subsets (subset-font / HarfBuzz)
tools/fonts-src/                original Montserrat TTF, Agustina WOFF, FA WOFF2s, mfizz TTF
public/                         favicon, profile_pic, manifest.json, robots.txt, og-image.jpg, .nojekyll
src/entry-client.jsx            hydrateRoot (prod) / createRoot (dev), hydration error logging
src/entry-server.jsx            render() for the pre-render step
src/App.jsx, src/index.css      root component; fonts and base styles (+ icons.css import)
src/styles/icons.css            Font Awesome + font-mfizz rules for the 18 icons in use
src/portfolio.js                all content; image imports; bannerColor values
src/lib/github.js               fetch-based GraphQL helper
src/components/reveal/          Reveal.jsx (Fade, Slide) + reveal.css static keyframes
src/components/displayLottie/   lazy, pausing, layout-stable Lottie box
src/components/header/          Headroom header; fonts.ready re-measure; ToggleSwitch.css import
src/components/experienceCard/  bannerColor with lazy ColorThief fallback
src/containers/Main.jsx         section order, theme state, preserved "//" text node
src/containers/greeting/        hero (Fade appear), Greeting.css then StarWars.css
src/containers/contact/         EmailJS form (VITE_* env), unconditional preventDefault
src/containers/topbutton/       rAF-throttled scroll handler
src/containers/profile|projects GitHub section (disabled) via fetch, lazy cards
```

B. Dependency table

Package	Before	After	Minified bytes it contributed (old vendor chunk)
lottie-web	5.7.3 full build, eager	5.7.3 lottie_svg, lazy	261,521
react-dom	16.13.1	18.3.1	117,315
apollo-* + graphql + graphql-tag + zen-observable	yes	removed	~137,000
react-reveal	1.2.2	removed (in-repo Reveal)	34,540
babel-runtime / @babel/runtime	yes	removed	~18,500
react-easy-emoji	1.4.0	removed	9,508
react-headroom	3.0.0	3.2.1	8,226
colorthief	eager	2.7.0 lazy fallback	6,385
react-twitter-embed + scriptjs	yes	removed	10,826
react-lottie	1.2.3	removed	3,571
dotenv + path-browserify + process	yes	removed	~5,600
emailjs-com	3.1.0 (+ CDN copy)	3.2.0	1,635

Package	Before	After	Minified bytes it contributed (old vendor chunk)
email-validator	2.0.4	2.0.4	324
gsap	dependency, tree-shaken	removed	0

C. Glossary

FCP	First Contentful Paint: first text or image painted.
LCP	Largest Contentful Paint: render time of the largest text block or image in the viewport; good ≤ 2.5 s at the 75th percentile.
CLS	Cumulative Layout Shift: sum of unexpected layout-shift scores in the worst 5-second window; good ≤ 0.1.
INP	Interaction to Next Paint: latency of the slowest interaction (roughly); replaced FID in March 2024; good ≤ 200 ms.
TBT	Total Blocking Time: sum of the portion of long tasks over 50 ms between FCP and TTI; lab proxy for interactivity.
Hydration	Attaching React to server-rendered DOM so it becomes interactive without re-creating it.
SSG / pre-render	Producing the HTML at build time rather than per request.
Render-blocking	A resource the browser must finish before it can paint (stylesheets, synchronous scripts).
Subsetting	Removing glyphs a font will never need.
Compositor-only animation	Animating transform/opacity so frames are produced without layout or paint.

D. Key numbers reference

Montserrat	245,708 → 38,460 bytes
Agustina	96,932 → 9,424 bytes
FA Brands / FA Solid / mfizz	76,576 / 79,464 / 65,864 → 2,684 / 760 / 1,324 bytes
ambulance / kassandra / logo512	142,688 / 135,065 / 93,789 → 76,536 / 61,628 / 42,932 bytes (lossless)
Lottie JSON (hero, skills, engine, email)	168,098 / 125,859 / 11,081 / 5,630 bytes raw; 21.1 / 9.8 / 1.6 / 1.1 KB gzipped
Old vendor chunk / main chunk	642,434 / 405,039 bytes raw; 171.8 / 83.1 KB gzipped
New app / React / Lottie player chunks	55,324 / 143,054 / 203,770 bytes raw; 23.2 / 45.8 / 53.6 KB gzipped